Crabs

#Introduction #Dataset description

The crabs dataset is part of the R package MASS and consists of 200 observations with 8 variables listed below:

1. sp: Species (B,O), "B" or "O" for blue or orange
2. sex: Sex(M,F), "male" or "female"
3. index: just enumerates the number of crabs in each of the 4 groups from 1 to 50.
4. FL: Frontal Lobe Size (mm) (FL, mm)
5. RW: Rear Width (mm) (RW, mm)
6. CL: Carapace Length (mm) (CL, mm)
7. CW: Carapace Width (mm) (CW,mm)
8. BD: Body Depth (mm) (BD,mm)

The libraries that are used, are as following:

```r
# loading all libraries
library(gamlss)
```

```
## Loading required package: splines

## Loading required package: gamlss.data

##
## Attaching package: 'gamlss.data'

## The following object is masked from 'package:datasets':
##
##     sleep

## Loading required package: gamlss.dist

## Loading required package: MASS

## Loading required package: nlme

## Loading required package: parallel

##  **********   GAMLSS Version 5.4-3  **********

## For more on GAMLSS look at https://www.gamlss.com/

## Type gamlssNews() to see new features/changes/bug fixes.
```

```r
library(gamlss.mx)
```

```
## Loading required package: nnet
```

```r
library(corrplot)
```

```
## corrplot 0.92 loaded
```

```r
library(factoextra)
```

```
## Loading required package: ggplot2

## Warning: package 'ggplot2' was built under R version 4.2.3

## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```r
library(hopkins)
library(NbClust)
library(mclust)
```

```
## Package 'mclust' version 5.4.10
## Type 'citation("mclust")' for citing this R package in publications.

##
## Attaching package: 'mclust'

## The following object is masked from 'package:gamlss.data':
##
##     acidity
```

```r
library(e1071)
library(ggplot2)
library(psych)
```

```
##
## Attaching package: 'psych'

## The following object is masked from 'package:mclust':
##
##     sim

## The following objects are masked from 'package:ggplot2':
##
##     %+%, alpha

## The following object is masked from 'package:gamlss':
##
##     cs
```

```r
library(clustertend)
```

```
## Package ‘clustertend‘ is deprecated.  Use package ‘hopkins‘ instead.

##
## Attaching package: ’clustertend’

## The following object is masked from ’package:hopkins’:
##
##     hopkins
```

```r
library(fpc)
library(ggplot2)
library(cluster)
library(clValid)
library(knitr)
library(tinytex)
```

#Dataset structure

First of all, the dataset is loaded.

```r
# install.packages("MASS")
library(MASS)
data(crabs)
```

To gain a clearer understanding of the dataset's structure, we utilize the "str()" command.

```r
str(crabs)
```

```
## ’data.frame’:    200 obs. of  8 variables:
##  $ sp   : Factor w/ 2 levels "B","O": 1 1 1 1 1 1 1 1 1 1 ...
##  $ sex  : Factor w/ 2 levels "F","M": 2 2 2 2 2 2 2 2 2 2 ...
##  $ index: int  1 2 3 4 5 6 7 8 9 10 ...
##  $ FL   : num  8.1 8.8 9.2 9.6 9.8 10.8 11.1 11.6 11.8 11.8 ...
##  $ RW   : num  6.7 7.7 7.8 7.9 8 9 9.9 9.1 9.6 10.5 ...
##  $ CL   : num  16.1 18.1 19 20.1 20.3 23 23.8 24.5 24.2 25.2 ...
##  $ CW   : num  19 20.8 22.4 23.1 23 26.5 27.1 28.4 27.8 29.3 ...
##  $ BD   : num  7 7.4 7.7 8.2 8.2 9.8 9.8 10.4 9.7 10.3 ...
```

The dataset consists of 200 objects and 8 variables, as evident from the output of the "str()" command.

However, since some values in the output of "str()" are rounded, we can use the "head()" command to obtain a clearer view of the values. By executing this command, we retrieve the first six rows of the "crabs" dataset.

```r
head(crabs)
```

```
##   sp sex index   FL  RW   CL   CW  BD
## 1  B   M     1  8.1 6.7 16.1 19.0 7.0
## 2  B   M     2  8.8 7.7 18.1 20.8 7.4
## 3  B   M     3  9.2 7.8 19.0 22.4 7.7
## 4  B   M     4  9.6 7.9 20.1 23.1 8.2
## 5  B   M     5  9.8 8.0 20.3 23.0 8.2
## 6  B   M     6 10.8 9.0 23.0 26.5 9.8
```

To check for the presence of non available values in our dataset, we can use the following command:

```
any(is.na(crabs))
```

```
## [1] FALSE
```

Since there are no NA values in the data, there is no need to remove any data points, and we can proceed with performing statistical operations.

The initial analysis begins with univariate analysis. Following that, a preliminary analysis is conducted to determine if it would be beneficial to run a Principal Component Analysis (PCA) and a Cluster Analysis on the dataset. Subsequently, the PCA and Cluster Analysis are performed, and the results are validated through Cluster Validation. Finally, Model Based Clustering is implemented.

(1) Univariate Analysis

In this section of the report, we will examine each variable in the dataset individually. Specifically, we will analyze the frequency distributions by calculating both the absolute and relative frequencies for each variable.

#sp:

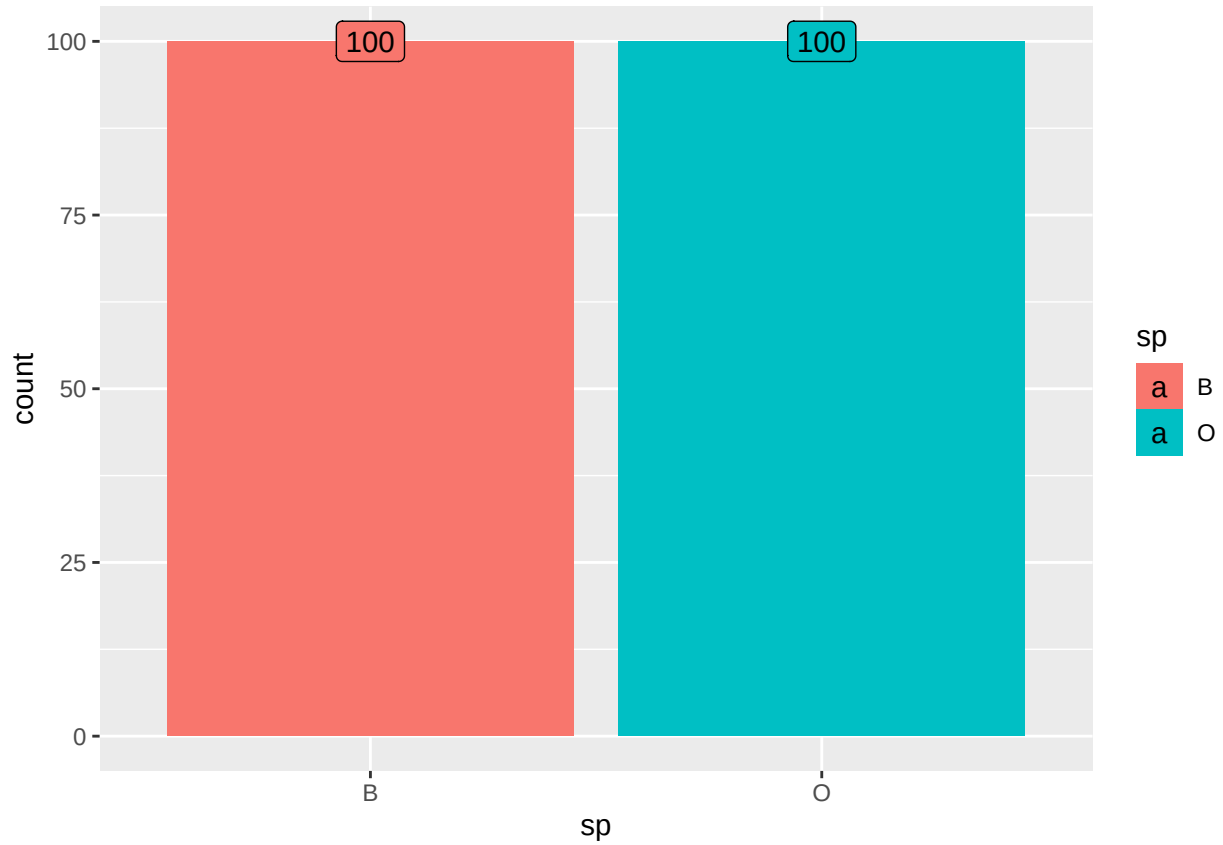
The variable "sp" is a categorical variable that indicates the species number, represented by the color "B" for blue or "O" for orange.

```
summary(crabs$sp)
```

```
##   B   O
## 100 100
```

```
ggplot(crabs, aes(x = sp, fill=sp, )) + geom_bar()+
geom_label(stat = "count", aes(label = ..count..))
```

```
## Warning: The dot-dot notation (`..count..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(count)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



#sex

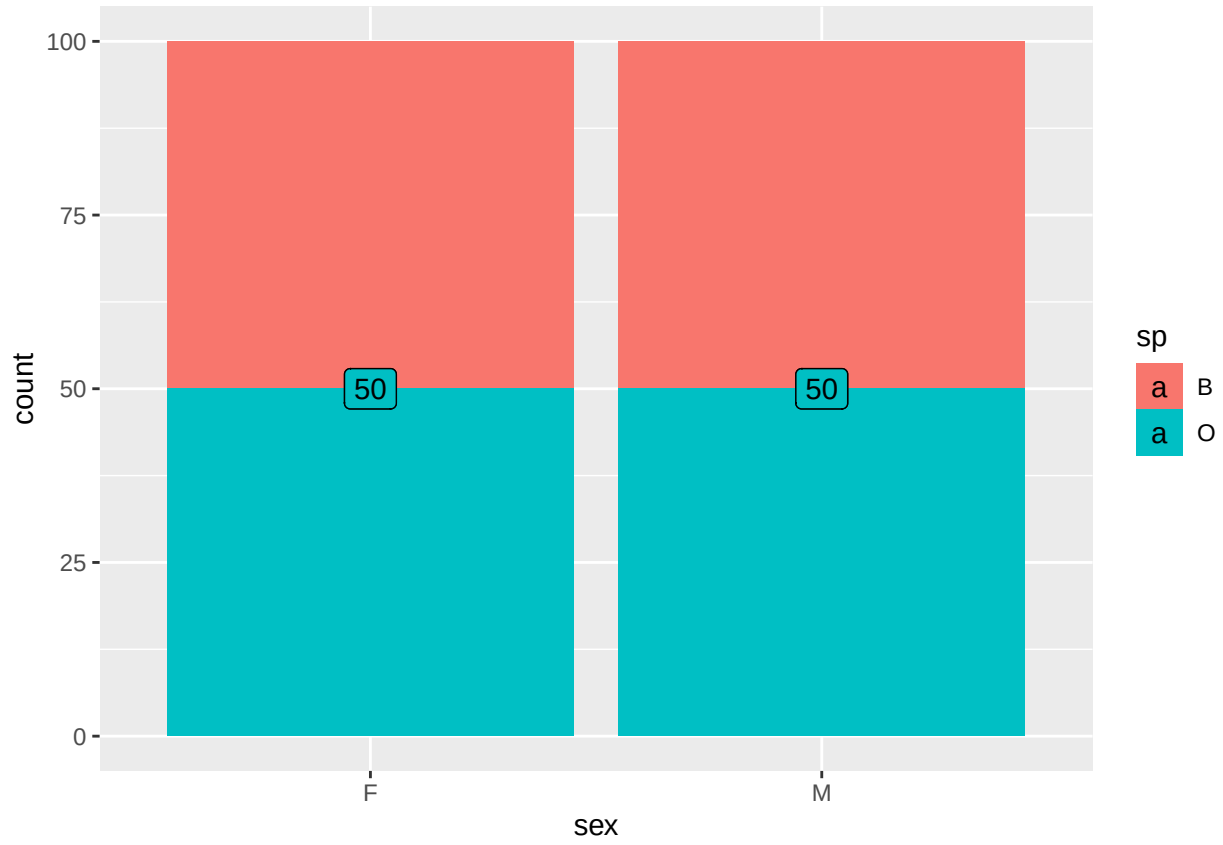
sex is a categorical variable showing the number of male or female crabs as "M" or "F".

By using different colors, we can attempt to identify specific characteristics associated with different combinations of Species and Sex.

```
summary(crabs$sex)
```

```
##   F   M
## 100 100
```

```
ggplot(crabs, aes(x = sex, fill=sp, )) + geom_bar()+
geom_label(stat = "count", aes(label = ..count..))
```



#FL

Frontal Lobe Size is a numerical variable.

```
summary(crabs$FL)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    7.20   12.90   15.55   15.58   18.05   23.10
```

```
length(crabs$FL)
```

```
## [1] 200
```

The length of FL variable is 200 and range is from 7.2 to 23.1.

```
sd(crabs$FL)
```

```
## [1] 3.495325
```

```
var(crabs$FL)
```

```
## [1] 12.2173
```

Standard deviation and variance are respectively 3.49 and 12.21.

```r
# Create the function to find mode.
getmode <- function(v) {
uniqv <- unique(v)
uniqv[which.max(tabulate(match(v, uniqv)))]
}
# Create the vector with numbers.
v <- c(crabs$FL)
# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
## [1] 15
```

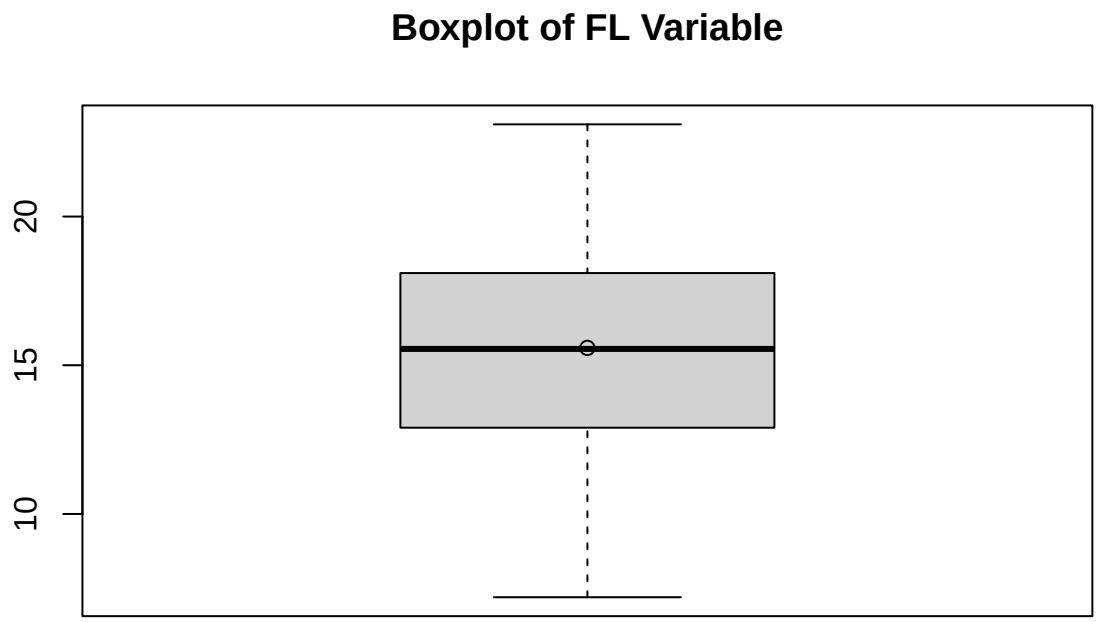
Based on the results, we observe that the mean and median are almost equal, indicating a symmetrical distribution without skewness. This is confirmed by the nearly zero skewness value obtained from the following command.

```r
# install.packages("moments")
# library(moments)
skewness(crabs$FL)
```

```
## [1] 0.01081884
```

#Boxplot

```r
boxplot(crabs$FL, main = "Boxplot of FL Variable")
points(mean(crabs$FL))
```

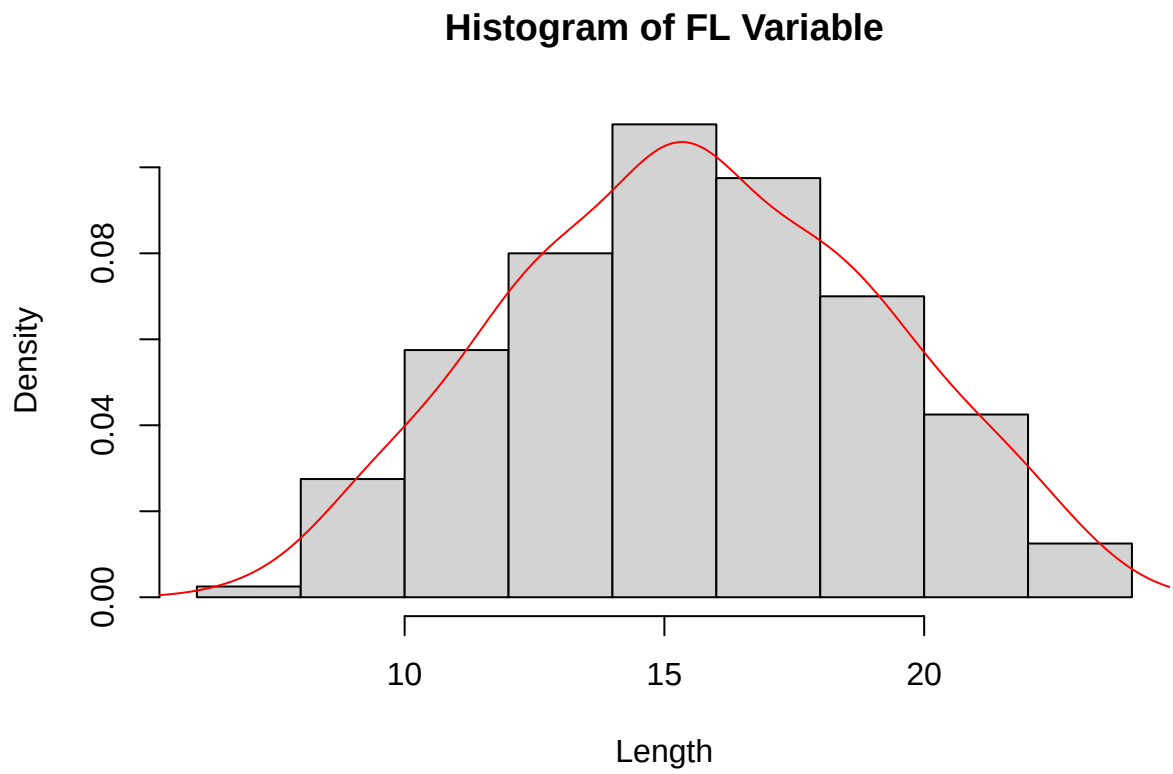


A boxplot is a visual depiction used to illustrate the distribution of a variable. It displays the first and third quartiles as the lower and upper boundaries of a rectangle, with the median dividing the box into two parts. Based on the boxplot generated for our data, we can conclude that this variable does not contain any outliers, as no dots lie outside the whiskers.

Now, we can proceed to plot a histogram and identify the model that best fits the distribution.

#Histogram

```
hist(crabs$FL,xlab = "Length" ,freq=FALSE, main = "Histogram of FL Variable")
lines(density(crabs$FL), col="red")
```



Based on the histogram of the FL, we observe that the distribution of data above and below the mean (15.58) is symmetric, suggesting an equal distribution on both sides. Additionally, the histogram displays a single peak, indicating a unimodal distribution. Furthermore, the symmetry of the histogram's tails confirms our earlier observation of zero skewness, as the right and left tails are equivalent.

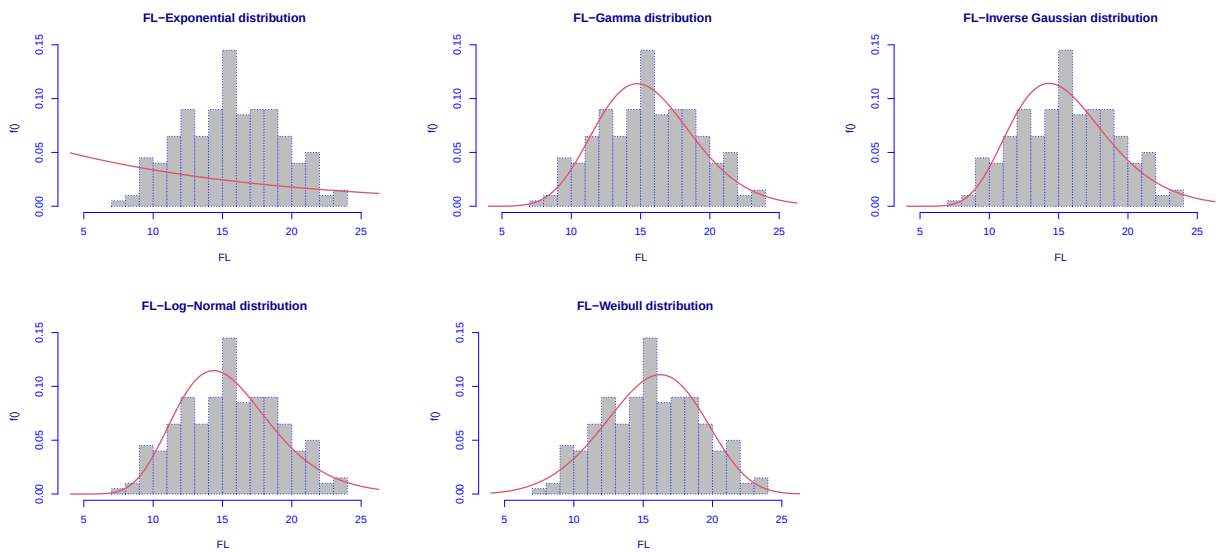
Data fitting for FL

In the following codes, we aim to determine the best-fitting model for the FL variable using three different criteria: maximizing the log-likelihood value, minimizing the Akaike Information Criterion (AIC), and minimizing the Bayesian Information Criterion (BIC).

```
round(2*nrow(crabs)**(1/3))
```

```
## [1] 12
```

```
FL.fit.EXP <- histDist(crabs$FL, family=EXP, xlab= "FL", nbins =12
                        ,main="FL-Exponential distribution")
FL.fit.GA <- histDist(crabs$FL, family=GA, xlab= "FL ", nbins =12,
                               main="FL-Gamma distribution")
FL.fit.IG <- histDist(crabs$FL, family=IG, xlab= " FL ", nbins =12,
                               main="FL-Inverse Gaussian distribution")
FL.fit.LOGNO <- histDist(crabs$FL, family=LOGNO, xlab= "FL ", nbins =12,
                                  main="FL-Log-Normal distribution")
FL.fit.WEI <- histDist(crabs$FL, family=WEI, xlab= "FL ", nbins =12,
                                main="FL-Weibull distribution")
```



A summary table of the Bic, the Aic and Log-likelihood values for each model is shown.

```
FL.fitted<-matrix(c(FL.fit.EXP$df.fit, logLik(FL.fit.EXP),
                                AIC(FL.fit.EXP), FL.fit.EXP$sbc,
FL.fit.GA$df.fit, logLik(FL.fit.GA), AIC(FL.fit.GA),
                                FL.fit.GA$sbc,
FL.fit.LOGNO$df.fit, logLik(FL.fit.LOGNO),
                                AIC(FL.fit.LOGNO), FL.fit.LOGNO$sbc,
FL.fit.IG$df.fit, logLik(FL.fit.IG), AIC(FL.fit.IG),
                               FL.fit.IG$sbc,
FL.fit.WEI$df.fit, logLik(FL.fit.WEI),
                               AIC(FL.fit.WEI), FL.fit.WEI$sbc), ncol=4, byrow=TRUE)
colnames(FL.fitted)<-c("df","LogLik", "AIC", "BIC")
rownames(FL.fitted)<-c("EXP", "GA", "LOGNO", "IG", "WEI")
FL.fitted<-as.table(FL.fitted)
FL.fitted
```

```
##              df    LogLik       AIC       BIC
## EXP      1.0000 -749.2361 1500.4722 1503.7705
## GA       2.0000 -535.4610 1074.9221 1081.5187
## LOGNO    2.0000 -538.6856 1081.3712 1087.9679
## IG       2.0000 -538.7339 1081.4678 1088.0644
## WEI      2.0000 -533.9393 1071.8787 1078.4753
```

Based on the Maximum Likelihood method, the BIC criterion, and the AIC criterion, the best-fitting model among the fitted ones is the Weibull distribution. This model has the highest Log-likelihood value (-533.9393) and the lowest values for both AIC (1071.8787) and BIC (1078.4753).

Likelihood-ratio test

The Likelihood-ratio test for goodness of fit was conducted between the Exponential model (null hypothesis) and the Weibull model (alternative hypothesis).

```r
#install.packages("lmtest")
library(lmtest)
```

```
## Loading required package: zoo

## Warning: package 'zoo' was built under R version 4.2.2

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
lrtest(FL.fit.EXP, FL.fit.WEI)
```

```
## Likelihood ratio test
##
## Model 1: gamlssML(formula = crabs$FL, family = "EXP")
## Model 2: gamlssML(formula = crabs$FL, family = "WEI")
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1   1 -749.24
## 2   2 -533.94  1 430.59  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The null hypothesis is rejected at almost all significance levels, indicating that the Weibull model significantly improves the accuracy of our model compared to the Exponential model. Therefore, it is evident that we should utilize the Weibull model for better accuracy.

```r
LR.test(FL.fit.EXP, FL.fit.WEI)
```

```
##  Likelihood Ratio Test for nested GAMLSS models.
##  (No check whether the models are nested is performed).
##
##        Null model: deviance= 1498.472 with  1 deg. of freedom
##  Altenative model: deviance= 1067.879 with  2 deg. of freedom
##
##  LRT = 430.5936 with 1 deg. of freedom and p-value= 0
```

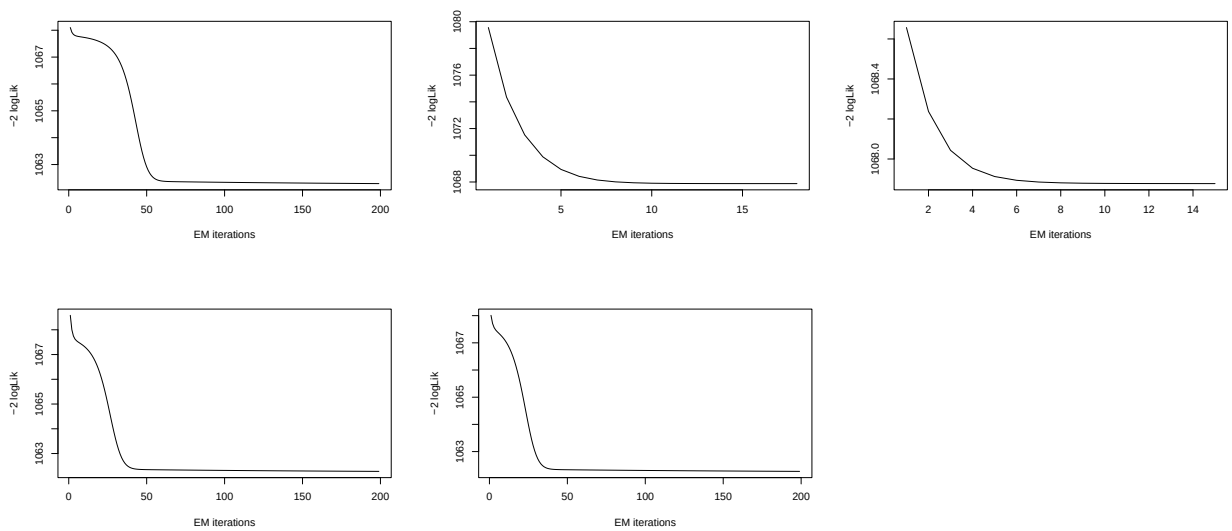
When comparing the Exponential distribution (null hypothesis) to the Weibull distribution (alternative hypothesis), the obtained p-value of 0 is less than the significance level of 0.05. As a result, we reject the null hypothesis, which suggests that the Exponential distribution is the appropriate model for the data. Instead, we conclude that the Weibull distribution is the better-fitting model, supported by its lower AIC and BIC values. The p-value of 0 confirms the rejection of the null model, which is the Exponential distribution.

Mixture of distributions

To find the best mixture of two Weibull distributions, the algorithm is repeated five times. This iterative process aims to determine the optimal parameters for the mixture model by evaluating different combinations and selecting the one that yields the best fit to the data

```
# install.packages("gamlss")
# install.packages("gamlss.mx")
library(gamlss.mx)
FL.mix.WEI <- gamlssMXfits(n = 5, crabs$FL~1, family = WEI,
                                    K = 2, data = crabs)
```

```
## model= 1

## model= 2

## model= 3

## model= 4

## model= 5
```




```
FL.mix.WEI$aic
```

```
## [1] 1072.271
```

```
FL.mix.WEI$sbc
```

```
## [1] 1088.763
```

```r
logLik(FL.mix.WEI)
```

```
## 'log Lik.' -531.1357 (df=5)
```

```r
FL.mix.WEI$prob
```

```
## [1] 0.5728299 0.4271701
```

```r
fitted(FL.mix.WEI, "mu")[1]
```

```
## [1] 16.65149
```

```r
fitted(FL.mix.WEI, "sigma")[2]
```

```
## [1] 16.65149
```

By using the mixture of two Weibull distributions, we observe that the AIC value is now 1072.234, which is slightly higher than the previous value of 1071.8787. Similarly, the BIC value is now 1088.725, which is higher than the previous value of 1078.4753. The log-likelihood value of the mixture model is -531.1206, which is greater than the log-likelihood value computed for a single Weibull distribution (-533.9393). This indicates that in general the mixture model provides a better fit to the data compared to the single Weibull distribution.

```r
mu.hat1 <- exp(FL.mix.WEI[["models"]][[1]][["mu.coefficients"]])
sigma.hat1 <- exp(FL.mix.WEI[["models"]][[1]][["sigma.coefficients"]])

mu.hat2 <- exp(FL.mix.WEI[["models"]][[2]][["mu.coefficients"]])
sigma.hat2 <- exp(FL.mix.WEI[["models"]][[2]][["sigma.coefficients"]])

hist(crabs$FL, breaks = 50,freq = FALSE, xlab = "FL",
     main="FL-Mixture of two Weibull distributions")

lines(seq(min(crabs$FL),max(crabs$FL),FL=FL(crabs$FL)),
      FL.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$FL),
max(crabs$FL),FL=FL(crabs$FL)), mu = mu.hat1, sigma = sigma.hat1),
lty=2,lwd=3,col=2)
```

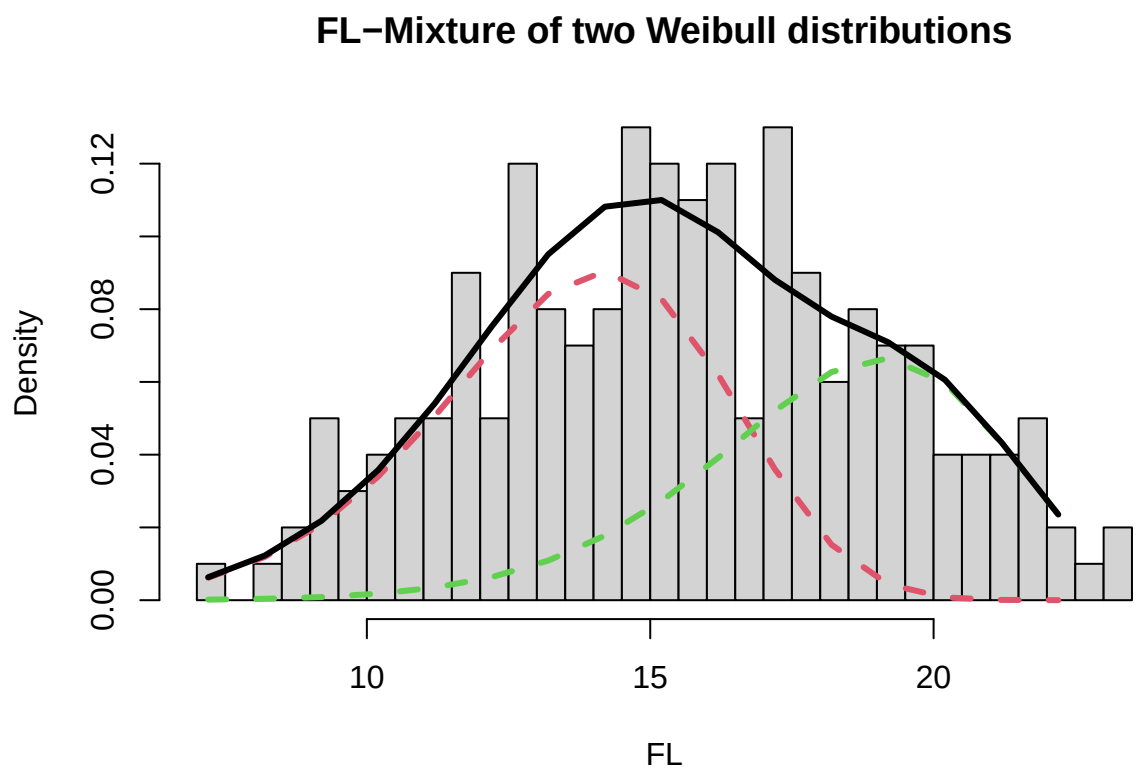
```
## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded

## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded
```

```r
lines(seq(min(crabs$FL),max(crabs$FL),FL=FL(crabs$FL)),
 FL.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$FL),max(crabs$FL)
,FL=FL(crabs$FL)), mu = mu.hat2, sigma = sigma.hat2),lty=2,lwd=3,col=3)
```

```
## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded

## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded
```

```r
lines(seq(min(crabs$FL),max(crabs$FL),FL=FL(crabs$FL)),
 FL.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$FL),max(crabs$FL),
 FL=FL(crabs$FL)), mu = mu.hat1, sigma = sigma.hat1) +
 FL.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$FL),
 max(crabs$FL),FL=FL(crabs$FL)), mu = mu.hat2, sigma = sigma.hat2),
 lty = 1, lwd = 3, col = 1)
```

```
## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded

## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded

## Warning: In seq.default(min(crabs$FL), max(crabs$FL), FL = FL(crabs$FL)) :
##  extra argument 'FL' will be disregarded
```



By selecting K=2, we can observe two peaks in the plot, each corresponding to a distribution. The dotted red line represents the first distribution, the dotted green line represents the second distribution, and the solid black line represents the overall mixture model for all the data. In this mixture model, the first Weibull distribution accounts for 60.76% of the data, while the second Weibull distribution accounts for 39.23%.

#RW

Rear Width is a numerical variable.

```r
summary(crabs$RW)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    6.50   11.00   12.80   12.74   14.30   20.20
```

```r
length(crabs$RW)
```

```
## [1] 200
```

The length of RW variable is 200 and range is from 6.5 to 20.2.

```r
sd(crabs$RW)
```

```
## [1] 2.57334
```

```r
var(crabs$RW)
```

```
## [1] 6.622078
```

```r
# Create the function to find mode.
getmode <- function(v) {
uniqv <- unique(v)
uniqv[which.max(tabulate(match(v, uniqv)))]
}
# Create the vector with numbers.
v <- c(crabs$RW)
# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
## [1] 11
```

Standard deviation and variance are respectively 2.5 and 6.6.

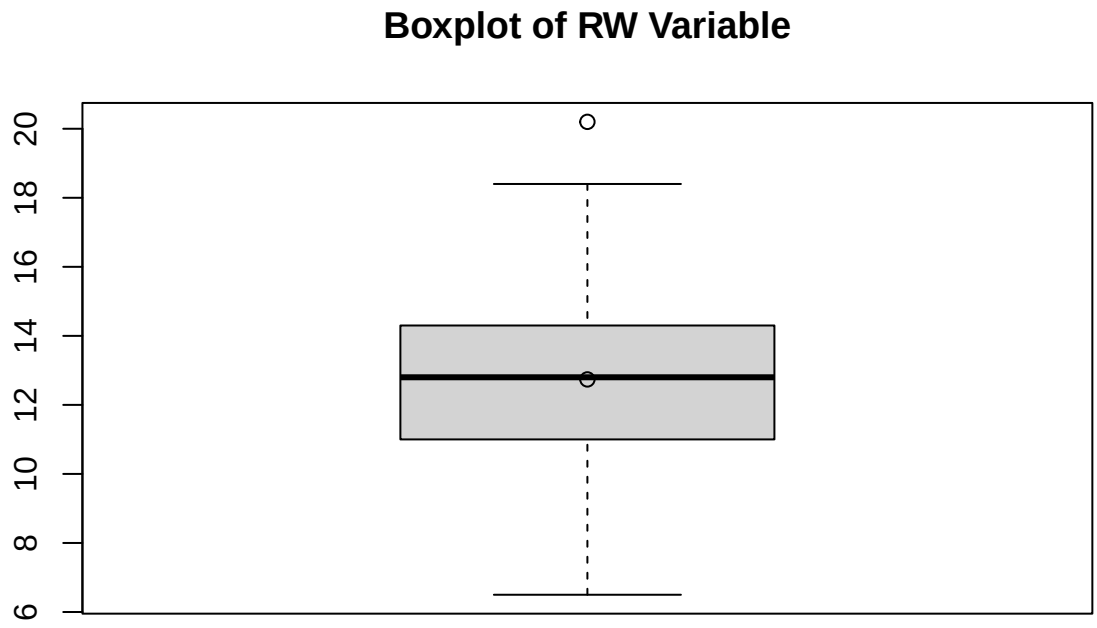
Based on the results, the Mean and Median are approximately equal, indicating a nearly symmetrical and non-skewed distribution. The similarity between the mean and median supports the conclusion that the distribution is almost equal. The skewness value, close to zero in the command shown, confirms the accuracy of the prediction.

```r
# install.packages("moments")
# library(moments)
skewness(crabs$RW)
```

```
## [1] 0.02556633
```

#Boxplot

```r
boxplot(crabs$RW, main = "Boxplot of RW Variable")
points(mean(crabs$RW))
```

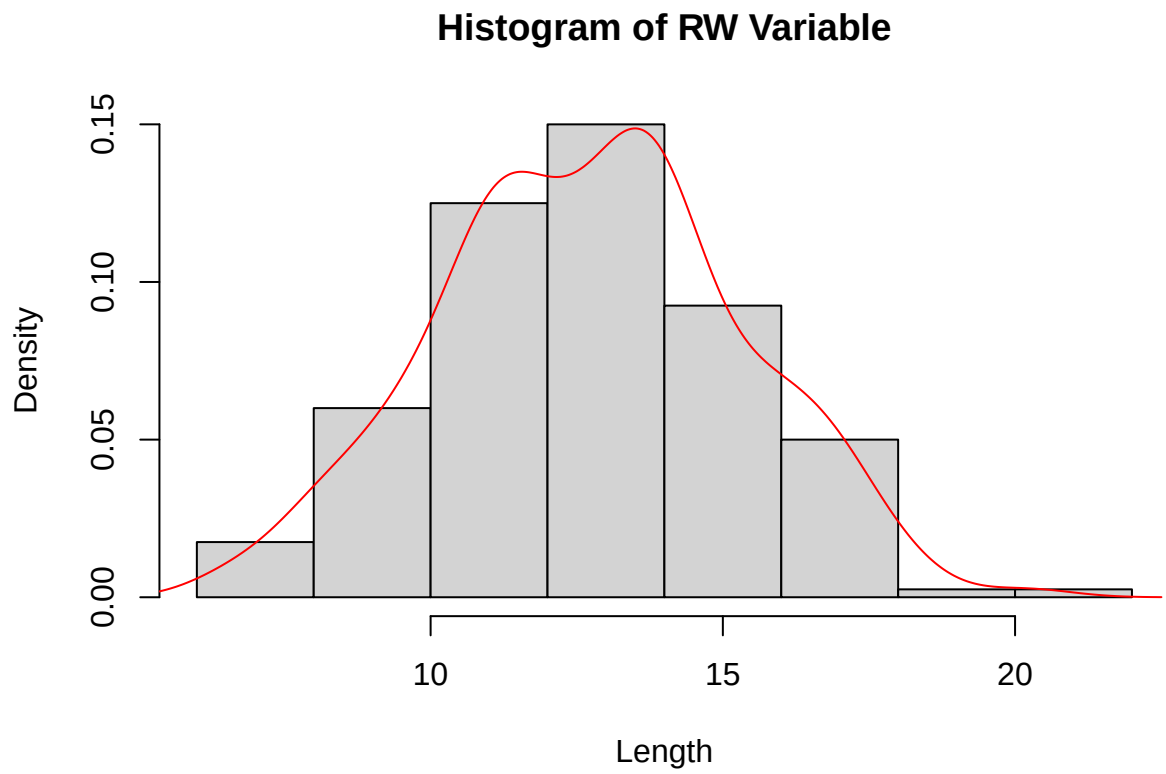


An outlier is present in the variable as indicated by a data point lying outside the expected range.

Next, we can generate a histogram to assess which model best matches the distribution.

#Histogram

```r
hist(crabs$RW,xlab = "Length" ,freq=FALSE, main = "Histogram of RW Variable")
lines(density(crabs$RW), col="red")
```



Based on the analysis of the RW variable, the plot shows that the data above and below the mean of 12.80 are evenly distributed, suggesting symmetry. Furthermore, the histogram displays two distinct peaks, indicating a bimodal distribution. Additionally, the symmetry of the distribution is evident from the fact that the right tail is equal to the left tail, which aligns with the previously determined skewness value of zero.

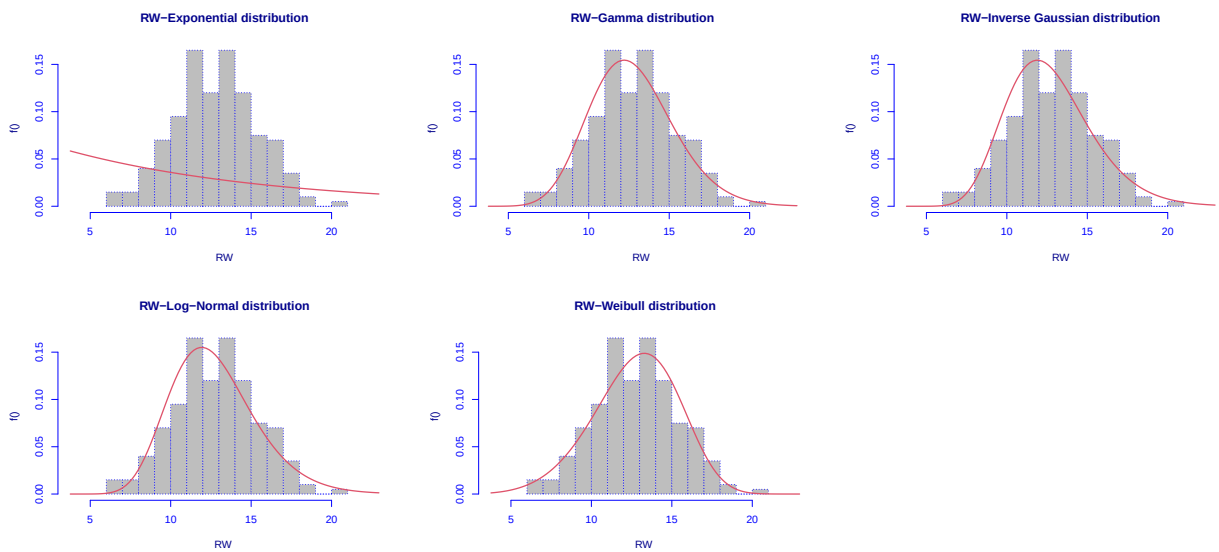
Data fitting for RW

In the upcoming codes, we will attempt to find the best-fitting model for the RW data using three criteria: maximizing the Log-likelihood value and minimizing the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC).

```
round(2*nrow(crabs)**(1/3))
```

```
## [1] 12
```

```
RW.fit.EXP <- histDist(crabs$RW, family=EXP, xlab= "RW", nbins =12
                        ,main="RW-Exponential distribution")
RW.fit.GA <- histDist(crabs$RW, family=GA, xlab= "RW ", nbins =12,
                               main="RW-Gamma distribution")
RW.fit.IG <- histDist(crabs$RW, family=IG, xlab= " RW ", nbins =12,
                               main="RW-Inverse Gaussian distribution")
RW.fit.LOGNO <- histDist(crabs$RW, family=LOGNO, xlab= "RW ", nbins =12,
                                  main="RW-Log-Normal distribution")
RW.fit.WEI <- histDist(crabs$RW, family=WEI, xlab= "RW ", nbins =12,
                                main="RW-Weibull distribution")
```



A summary table of the Bic, the Aic and Log-likelihood values for each model is shown.

```
RW.fitted<-matrix(c(RW.fit.EXP$df.fit, logLik(RW.fit.EXP),
                                AIC(RW.fit.EXP), RW.fit.EXP$sbc,
RW.fit.GA$df.fit, logLik(RW.fit.GA), AIC(RW.fit.GA),
                                RW.fit.GA$sbc,
RW.fit.LOGNO$df.fit, logLik(RW.fit.LOGNO),
                                AIC(RW.fit.LOGNO), RW.fit.LOGNO$sbc,
RW.fit.IG$df.fit, logLik(RW.fit.IG), AIC(RW.fit.IG),
                               RW.fit.IG$sbc,
RW.fit.WEI$df.fit, logLik(RW.fit.WEI),
                               AIC(RW.fit.WEI), RW.fit.WEI$sbc), ncol=4, byrow=TRUE)
colnames(RW.fitted)<-c("df","LogLik", "AIC", "BIC")
rownames(RW.fitted)<-c("EXP", "GA", "LOGNO", "IG", "WEI")
RW.fitted<-as.table(RW.fitted)
RW.fitted
```

```
##              df    LogLik       AIC       BIC
## EXP      1.0000 -708.9258 1419.8516 1423.1499
## GA       2.0000 -474.2640  952.5280  959.1246
## LOGNO    2.0000 -477.2675  958.5349  965.1316
## IG       2.0000 -477.4070  958.8140  965.4106
## WEI      2.0000 -474.4331  952.8662  959.4629
```

Based on the Maximum Likelihood method, the BIC criterion, and the AIC criterion, the best-fitting model among the ones tested is the Gamma distribution with two parameters. This model has the highest Log-likelihood value (-474.2640) and the lowest values for both AIC (952.5280) and BIC (959.1246).

Likelihood-ratio test

A likelihood-ratio test for goodness of fit was conducted between the Exponential model (under the null hypothesis) and the Gamma model (under the alternative hypothesis).

```r
#install.packages("lmtest")
#library(lmtest)
lrtest(RW.fit.EXP, RW.fit.GA)
```

```
## Likelihood ratio test
##
## Model 1: gamlssML(formula = crabs$RW, family = "EXP")
## Model 2: gamlssML(formula = crabs$RW, family = "GA")
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1   1 -708.93
## 2   2 -474.26  1 469.32  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Based on the results, the null hypothesis is rejected at almost every significance level. This suggests that the Gamma model significantly improves the accuracy of the model compared to the Exponential model. Therefore, it is advisable to use the Gamma model as it provides a substantial increase in accuracy.

```r
LR.test(RW.fit.EXP, RW.fit.GA)
```

```
##  Likelihood Ratio Test for nested GAMLSS models.
##  (No check whether the models are nested is performed).
##
##        Null model: deviance= 1417.852 with  1 deg. of freedom
##  Altenative model: deviance= 948.528 with  2 deg. of freedom
##
##  LRT = 469.3236 with 1 deg. of freedom and p-value= 0
```

Upon comparing the Exponential distribution (null hypothesis) with the Gamma distribution (alternative hypothesis), the p-value is determined to be 0, which is less than the significance level of 0.05. Therefore, the null hypothesis, suggesting the Exponential distribution as the appropriate model for the data, is rejected. It can be concluded that the Gamma distribution is the correct model choice as it exhibits lower AIC and BIC values compared to the Exponential distribution.

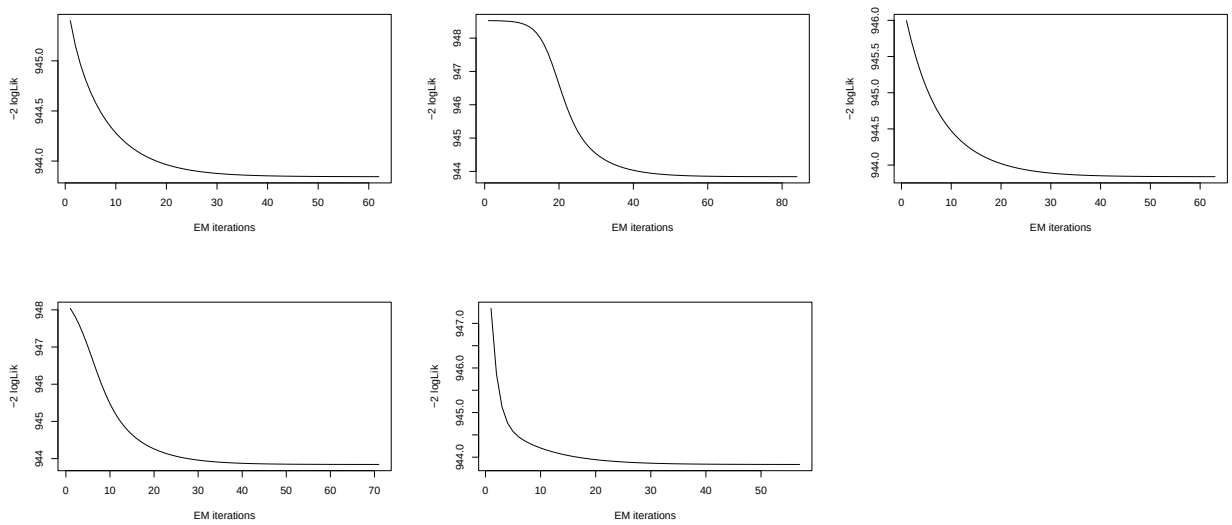
Mixture of distributions

To determine the best mixture of two Gamma distributions, the algorithm will be repeated five times. This process involves computing and evaluating different mixtures to identify the optimal solution.

```r
# install.packages("gamlss")
# install.packages("gamlss.mx")
library(gamlss.mx)
RW.mix.GA <- gamlssMXfits(n = 5, crabs$RW~1, family = GA,
                                    K = 2, data = crabs)
```

```
## model= 1

## model= 2

## model= 3
```

```
## model= 4

## model= 5
```




```
RW.mix.GA$aic
```

```
## [1] 953.837
```

```
RW.mix.GA$sbc
```

```
## [1] 970.3286
```

```
logLik(RW.mix.GA)
```

```
## 'log Lik.' -471.9185 (df=5)
```

```
RW.mix.GA$prob
```

```
## [1] 0.5400071 0.4599929
```

```
fitted(RW.mix.GA, "mu")[1]
```

```
## [1] 12.7386
```

```
fitted(RW.mix.GA, "sigma")[2]
```

```
## [1] 12.7386
```

By using the mixture of two Weibull distributions, we observe that the AIC value is now 953.84, which is slightly higher than the previous value of 952.52 Similarly, the BIC value is now 970.33, which is higher than the previous value of 959.12 log-likelihood value of the mixture model is -471.9205, which is higher than the log-likelihood value obtained from fitting a single Gamma distribution (-474.2640). This suggests that the mixture model provides a better fit to the data.

```r
mu.hat1 <- exp(RW.mix.GA[["models"]][[1]][["mu.coefficients"]])
sigma.hat1 <- exp(RW.mix.GA[["models"]][[1]][["sigma.coefficients"]])

mu.hat2 <- exp(RW.mix.GA[["models"]][[2]][["mu.coefficients"]])
sigma.hat2 <- exp(RW.mix.GA[["models"]][[2]][["sigma.coefficients"]])

hist(crabs$RW, breaks = 50,freq = FALSE, xlab = "RW",
     main="RW-Mixture of two Gamma distributions")

lines(seq(min(crabs$RW),max(crabs$RW),RW=RW(crabs$RW)),
      RW.mix.GA[["prob"]][1]*dGA(seq(min(crabs$RW),
max(crabs$RW),RW=RW(crabs$RW)), mu = mu.hat1, sigma = sigma.hat1),
lty=2,lwd=3,col=2)
```

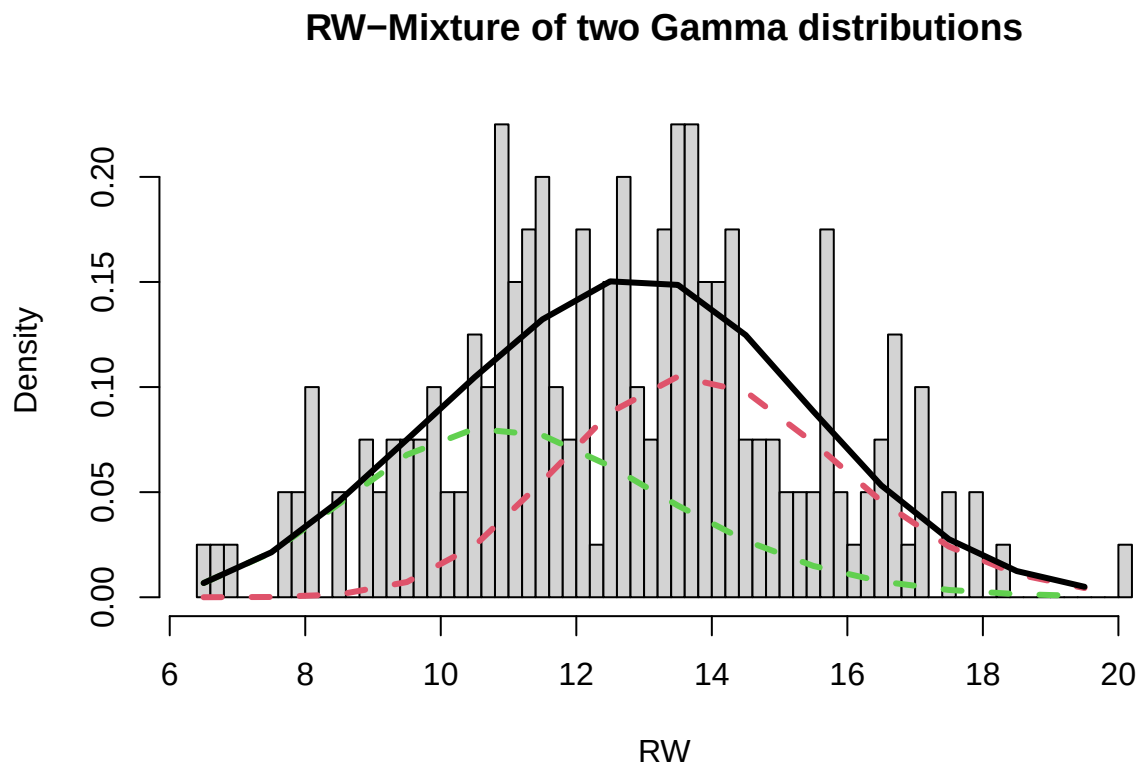
```
## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded

## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded
```

```r
lines(seq(min(crabs$RW),max(crabs$RW),RW=RW(crabs$RW)),
 RW.mix.GA[["prob"]][2]*dGA(seq(min(crabs$RW),max(crabs$RW)
,RW=RW(crabs$RW)), mu = mu.hat2, sigma = sigma.hat2),lty=2,lwd=3,col=3)
```

```
## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded

## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded
```

```r
lines(seq(min(crabs$RW),max(crabs$RW),RW=RW(crabs$RW)),
 RW.mix.GA[["prob"]][1]*dGA(seq(min(crabs$RW),max(crabs$RW),
 RW=RW(crabs$RW)), mu = mu.hat1, sigma = sigma.hat1) +
 RW.mix.GA[["prob"]][2]*dGA(seq(min(crabs$RW),
 max(crabs$RW),RW=RW(crabs$RW)), mu = mu.hat2, sigma = sigma.hat2),
 lty = 1, lwd = 3, col = 1)
```

```
## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded

## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded

## Warning: In seq.default(min(crabs$RW), max(crabs$RW), RW = RW(crabs$RW)) :
##  extra argument 'RW' will be disregarded
```



With K=2 selected for the mixture model, the plot displays two peaks corresponding to each distribution. The dotted red line represents the first distribution, the dotted green line represents the second distribution, and the solid black line represents the overall mixture model for all the data. In this mixture model, the first Gamma distribution accounts for 46.12% while the second Gamma distribution accounts for 53.88%.

#CL

Carapace Length is a numerical variable.

```
summary(crabs$CL)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   14.70   27.27   32.10   32.11   37.23   47.60
```

```
length(crabs$CL)
```

```
## [1] 200
```

The length of CL variable is 200 and range is from 14.7 to 47.6.

```
sd(crabs$CL)
```

```
## [1] 7.118983
```

```r
var(crabs$CL)
```

```
## [1] 50.67992
```

```r
# Create the function to find mode.
getmode <- function(v) {
uniqv <- unique(v)
uniqv[which.max(tabulate(match(v, uniqv)))]
}
# Create the vector with numbers.
v <- c(crabs$CL)
# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
## [1] 34.5
```

Standard deviation and variance are respectively 7.12 and 50.68.

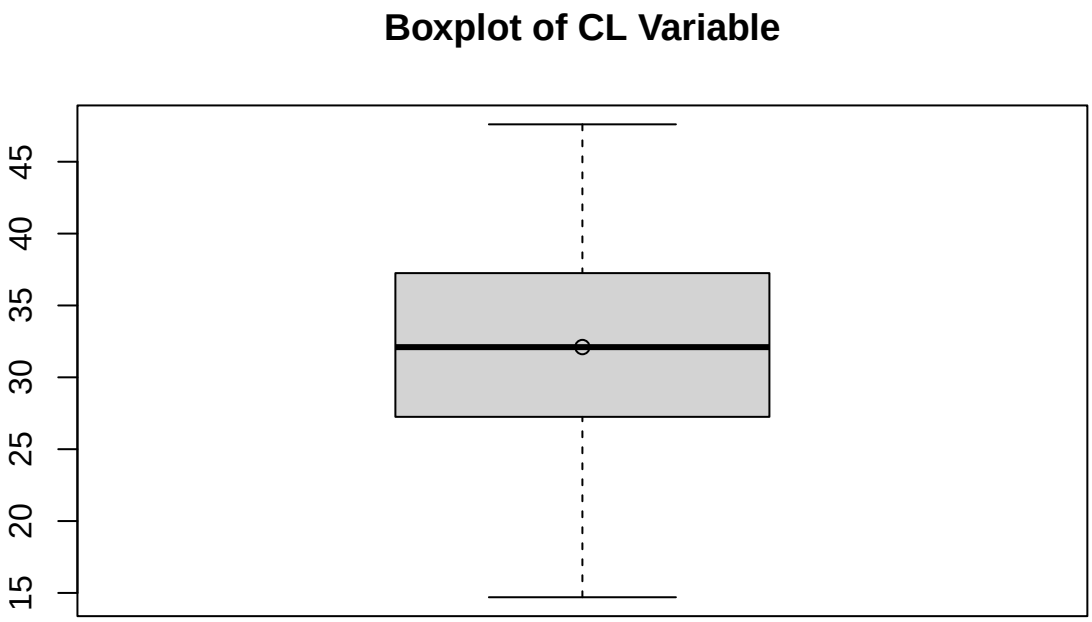
Based on the results, the mean and median values are equal, indicating a symmetrical distribution without skewness. The similarity between the mean and median further supports the conclusion of an equal distribution. Additionally, the skewness value, as shown in the subsequent command, is close to zero, further confirming the lack of skewness in the distribution.

```r
# install.packages("moments")
# library(moments)
skewness(crabs$CL)
```

```
## [1] -0.07013991
```

#Boxplot

```r
boxplot(crabs$CL, main = "Boxplot of CL Variable")
points(mean(crabs$CL))
```

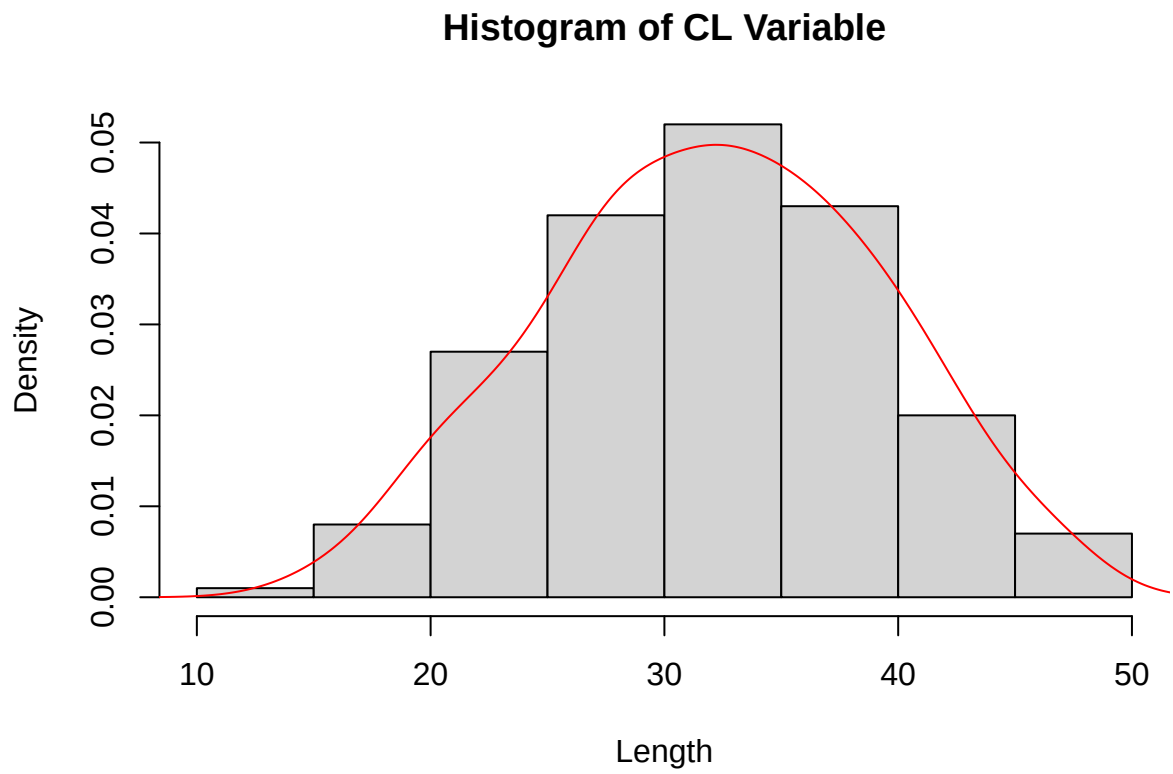


We can say that in this variable there is no outliers.

Now, we can plot the histogram and look for the model that fits the distribution better.

#Histogram

```
hist(crabs$CL,xlab = "Length" ,freq=FALSE, main = "Histogram of CL Variable")
lines(density(crabs$CL), col="red")
```



Based on the analysis of the CL variable, the plot shows that the data above and below the mean of 32.1 are evenly distributed, suggesting symmetry. Furthermore, the histogram displays a single peak, indicating a unimodal distribution. Additionally, the symmetry of the distribution is evident from the fact that the right tail is equal to the left tail, which aligns with the previously determined skewness value of zero.

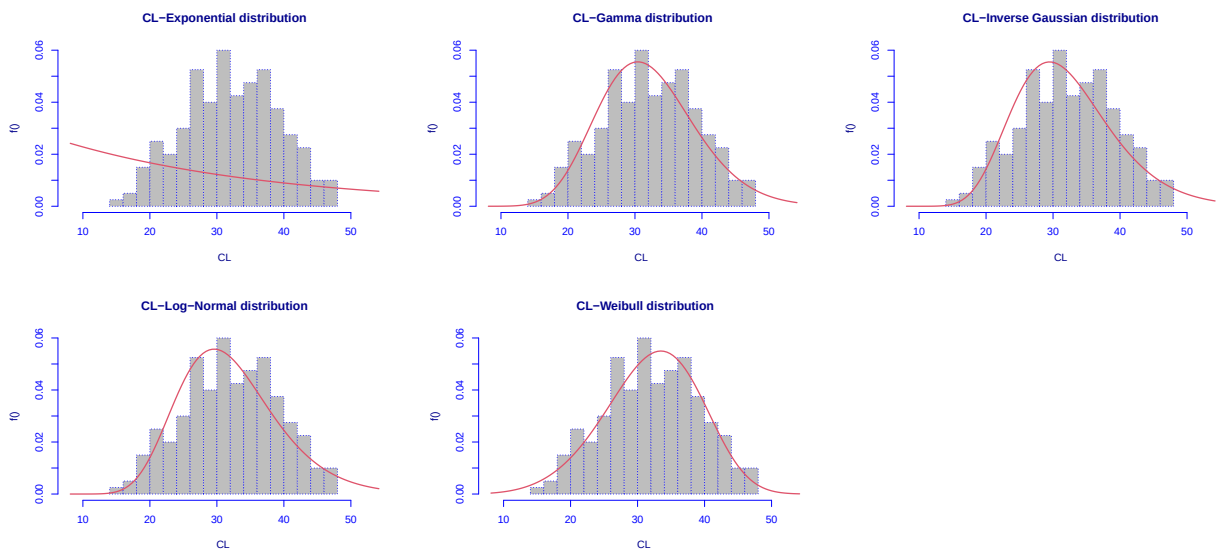
Data fitting for CL

In the following codes, we will try to establish the model to fit the CL data, by looking at three different criterions: the Log-likelihood value (which has to be maximized), the Akaike Information Criterion (AIC) and the Bayesian Information criterion (BIC) (which have to be minimized).

```
round(2*nrow(crabs)**(1/3))
```

```
## [1] 12
```

```
CL.fit.EXP <- histDist(crabs$CL, family=EXP, xlab= "CL", nbins =12
                        ,main="CL-Exponential distribution")
CL.fit.GA <- histDist(crabs$CL, family=GA, xlab= "CL ", nbins =12,
                               main="CL-Gamma distribution")
CL.fit.IG <- histDist(crabs$CL, family=IG, xlab= " CL ", nbins =12,
                               main="CL-Inverse Gaussian distribution")
CL.fit.LOGNO <- histDist(crabs$CL, family=LOGNO, xlab= "CL ", nbins =12,
                                  main="CL-Log-Normal distribution")
CL.fit.WEI <- histDist(crabs$CL, family=WEI, xlab= "CL ", nbins =12,
                                main="CL-Weibull distribution")
```



A summary table of the Bic, the Aic and Log-likelihood values for each model is shown.

```
CL.fitted<-matrix(c(CL.fit.EXP$df.fit, logLik(CL.fit.EXP),
                                AIC(CL.fit.EXP), CL.fit.EXP$sbc,
CL.fit.GA$df.fit, logLik(CL.fit.GA), AIC(CL.fit.GA),
                                CL.fit.GA$sbc,
CL.fit.LOGNO$df.fit, logLik(CL.fit.LOGNO),
                                AIC(CL.fit.LOGNO), CL.fit.LOGNO$sbc,
CL.fit.IG$df.fit, logLik(CL.fit.IG), AIC(CL.fit.IG),
                               CL.fit.IG$sbc,
CL.fit.WEI$df.fit, logLik(CL.fit.WEI),
                               AIC(CL.fit.WEI), CL.fit.WEI$sbc), ncol=4, byrow=TRUE)
colnames(CL.fitted)<-c("df","LogLik", "AIC", "BIC")
rownames(CL.fitted)<-c("EXP", "GA", "LOGNO", "IG", "WEI")
CL.fitted<-as.table(CL.fitted)
CL.fitted
```

```
##              df    LogLik       AIC       BIC
## EXP      1.0000 -893.8055 1789.6109 1792.9093
## GA       2.0000 -679.1146 1362.2292 1368.8258
## LOGNO    2.0000 -683.0714 1370.1428 1376.7394
## IG       2.0000 -683.2188 1370.4376 1377.0342
## WEI      2.0000 -675.4328 1354.8657 1361.4623
```

According to the Maximum Likelihood method, the BIC criterion and the AIC criterion, the best model between the fitted ones is the Weibull distribution (two parameters' distributions). The model has the maximum value of Log-likelihood (-675.4328) and the minimum value both AIC (1354.8657) and BIC (1361.4623).

Likelihood-ratio test

The Likelihood-ratio test goodness of fit performed between the Exponential model (under the null hypothesis) and the Weibull model (under the alternative hypothesis).

```r
#install.packages("lmtest")
#library(lmtest)
lrtest(CL.fit.EXP, CL.fit.WEI)
```

```
## Likelihood ratio test
##
## Model 1: gamlssML(formula = crabs$CL, family = "EXP")
## Model 2: gamlssML(formula = crabs$CL, family = "WEI")
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1   1 -893.81
## 2   2 -675.43  1 436.75  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The null hypothesis would be rejected at nearly every significance level. Thus, we know that we should definitely use the Weibull model as it increases the accuracy of our model by a substantial amount.

```r
LR.test(CL.fit.EXP, CL.fit.WEI)
```

```
##  Likelihood Ratio Test for nested GAMLSS models.
##  (No check whether the models are nested is performed).
##
##        Null model: deviance= 1787.611 with  1 deg. of freedom
##  Altenative model: deviance= 1350.866 with  2 deg. of freedom
##
##  LRT = 436.7452 with 1 deg. of freedom and p-value= 0
```

Upon comparing the Exponential distribution (null hypothesis) with the Weibull distribution (alternative hypothesis), the p-value is determined to be 0, which is less than the significance level of 0.05. Therefore, the null hypothesis, suggesting the Exponential distribution as the appropriate model for the data, is rejected. It can be concluded that the Weibull distribution is the correct model choice as it exhibits lower AIC and BIC values compared to the Exponential distribution.

Mixture of distributions

It is possible to compute a mixture of two Weibull distributions. In order to find the best mixture, the algorithm is repeated five times.

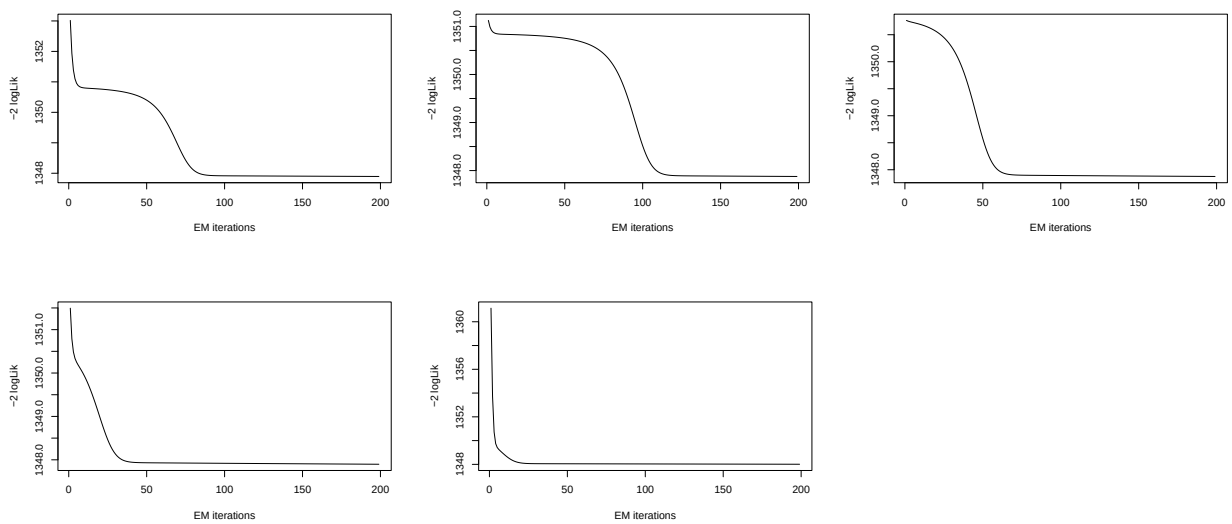
```r
# install.packages("gamlss")
# install.packages("gamlss.mx")
library(gamlss.mx)
CL.mix.WEI <- gamlssMXfits(n = 5, crabs$CL~1, family = WEI,
                                    K = 2, data = crabs)
```

```
## model= 1
```

```
## model= 2
```

```
## model= 3
```

```
## model= 4

## model= 5
```




```
CL.mix.WEI$aic
```

```
## [1] 1357.875
```

```
CL.mix.WEI$sbc
```

```
## [1] 1374.367
```

```
logLik(CL.mix.WEI)
```

```
## 'log Lik.' -673.9377 (df=5)
```

```
CL.mix.WEI$prob
```

```
## [1] 0.4489679 0.5510321
```

```
fitted(CL.mix.WEI, "mu")[1]
```

```
## [1] 34.40277
```

```
fitted(CL.mix.WEI, "sigma")[2]
```

```
## [1] 34.40277
```

We can see that with the mixture of two Weibull distributions the AIC and BIC values have increased. The log-likelihood value of the mixture is -673.9299, greater than the log-likelihood value computed on a single Weibull distribution (-675.4328).

```r
mu.hat1 <- exp(CL.mix.WEI[["models"]][[1]][["mu.coefficients"]])
sigma.hat1 <- exp(CL.mix.WEI[["models"]][[1]][["sigma.coefficients"]])

mu.hat2 <- exp(CL.mix.WEI[["models"]][[2]][["mu.coefficients"]])
sigma.hat2 <- exp(CL.mix.WEI[["models"]][[2]][["sigma.coefficients"]])

hist(crabs$CL, breaks = 50,freq = FALSE, xlab = "CL",
     main="CL-Mixture of two Weibull distributions")

lines(seq(min(crabs$CL),max(crabs$CL),CL=CL(crabs$CL)),
      CL.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$CL),
max(crabs$CL),CL=CL(crabs$CL)), mu = mu.hat1, sigma = sigma.hat1),
lty=2,lwd=3,col=2)
```

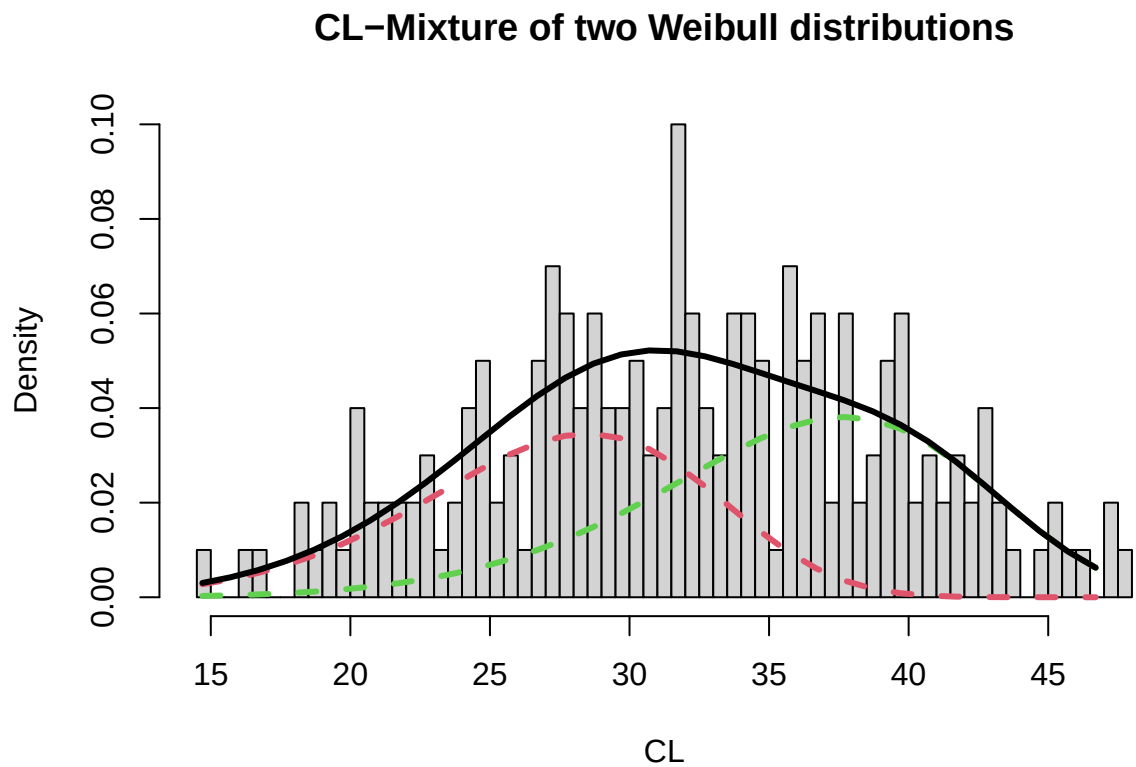
```
## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded

## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded
```

```r
lines(seq(min(crabs$CL),max(crabs$CL),CL=CL(crabs$CL)),
 CL.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$CL),max(crabs$CL)
,CL=CL(crabs$CL)), mu = mu.hat2, sigma = sigma.hat2),lty=2,lwd=3,col=3)
```

```
## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded

## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded
```

```r
lines(seq(min(crabs$CL),max(crabs$CL),CL=CL(crabs$CL)),
 CL.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$CL),max(crabs$CL),
 CL=CL(crabs$CL)), mu = mu.hat1, sigma = sigma.hat1) +
 CL.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$CL),
 max(crabs$CL),CL=CL(crabs$CL)), mu = mu.hat2, sigma = sigma.hat2),
 lty = 1, lwd = 3, col = 1)
```

```
## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded

## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded

## Warning: In seq.default(min(crabs$CL), max(crabs$CL), CL = CL(crabs$CL)) :
##  extra argument 'CL' will be disregarded
```



Since we have selected K=2, we can see in the plot 2 peaks, each corresponding to a distribution. The dotted red line is relative to the first distribution, the dotted green one to the second distribution, while the black line is relative to the mixture, i.e. the overall model for all data. In this mixture (solid black line), the first Weibull distribution (dotted red line) accounts for 37.2% the second Weibull distribution (dotted green line) accounts for 62.8%.

#CW

Carapace Width is a numerical variable.

```
summary(crabs$CW)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   17.10   31.50   36.80   36.41   42.00   54.60
```

```
length(crabs$CW)
```

```
## [1] 200
```

The length of CW variable is 200 and range is from 17.1 to 54.6.

```
sd(crabs$CW)
```

```
## [1] 7.871955
```

```r
var(crabs$CW)
```

```
## [1] 61.96768
```

```r
# Create the function to find mode.
getmode <- function(v) {
uniqv <- unique(v)
uniqv[which.max(tabulate(match(v, uniqv)))]
}
# Create the vector with numbers.
v <- c(crabs$CW)
# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
## [1] 31.8
```

Standard deviation and variance are respectively 7.87 and 61.97.

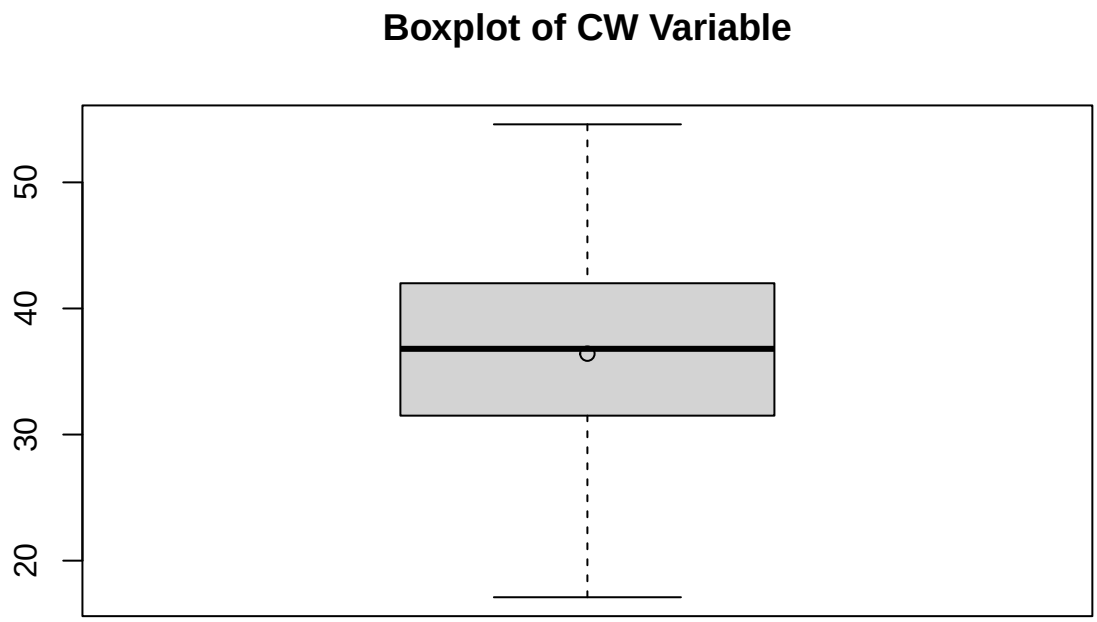
Based on the results, the mean and median values are approximately equal, indicating a nearly symmetrical distribution without skewness. The similarity between the mean and median further supports the conclusion that the distribution is nearly equal. Additionally, the skewness value, as shown in the subsequent command, is close to zero.

```r
# install.packages("moments")
# library(moments)
skewness(crabs$CW)
```

```
## [1] -0.09834458
```

#Boxplot

```r
boxplot(crabs$CW, main = "Boxplot of CW Variable")
points(mean(crabs$CW))
```

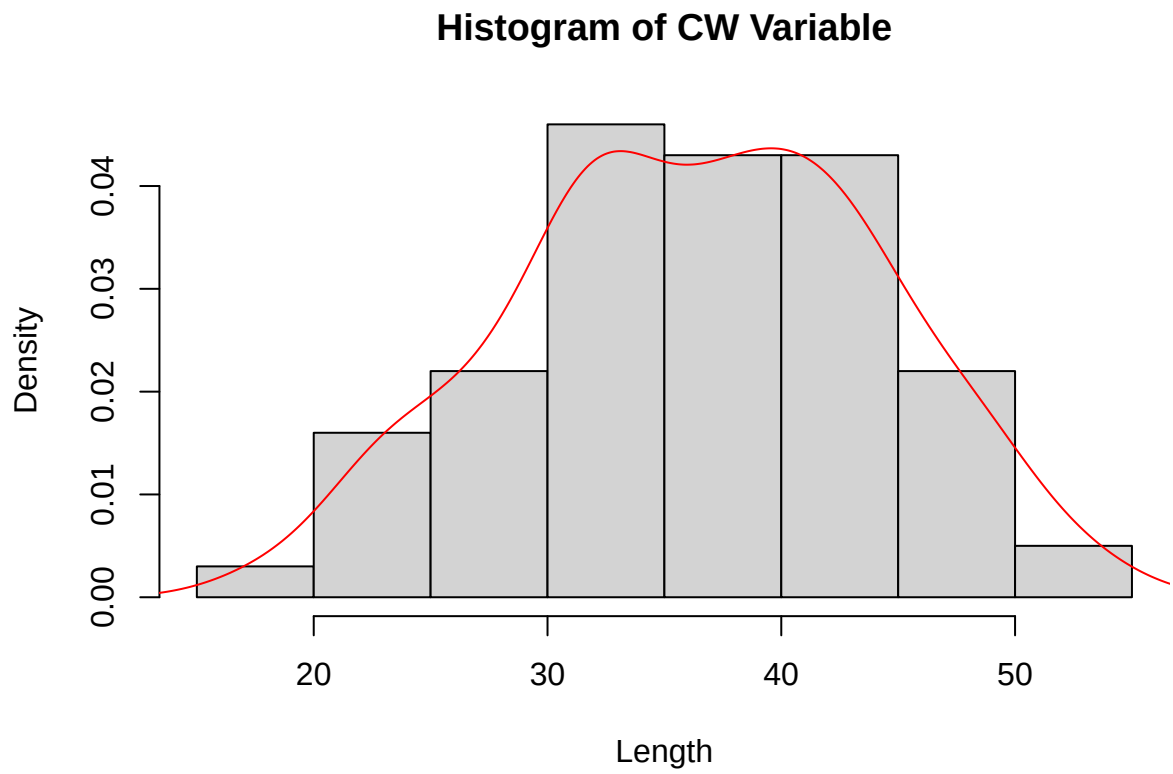


Since no outliers are present in this variable, we can proceed to plot the histogram and determine the best-fitting model for the distribution.

Now, we can plot the histogram and look for the model that fits the distribution better.

#Histogram

```
hist(crabs$CW,xlab = "Length" ,freq=FALSE, main = "Histogram of CW Variable")
lines(density(crabs$CW), col="red")
```



Based on the analysis of the plot and histogram, it is observed that the distribution of data above and below the mean of 36.4 is evenly distributed, indicating symmetry. Additionally, the histogram displays two peaks, suggesting a bimodal distribution. Furthermore, the symmetry of the distribution is evident from the fact that the right tail is equal to the left tail, which aligns with the previously determined skewness value of zero.

Data fitting for CW

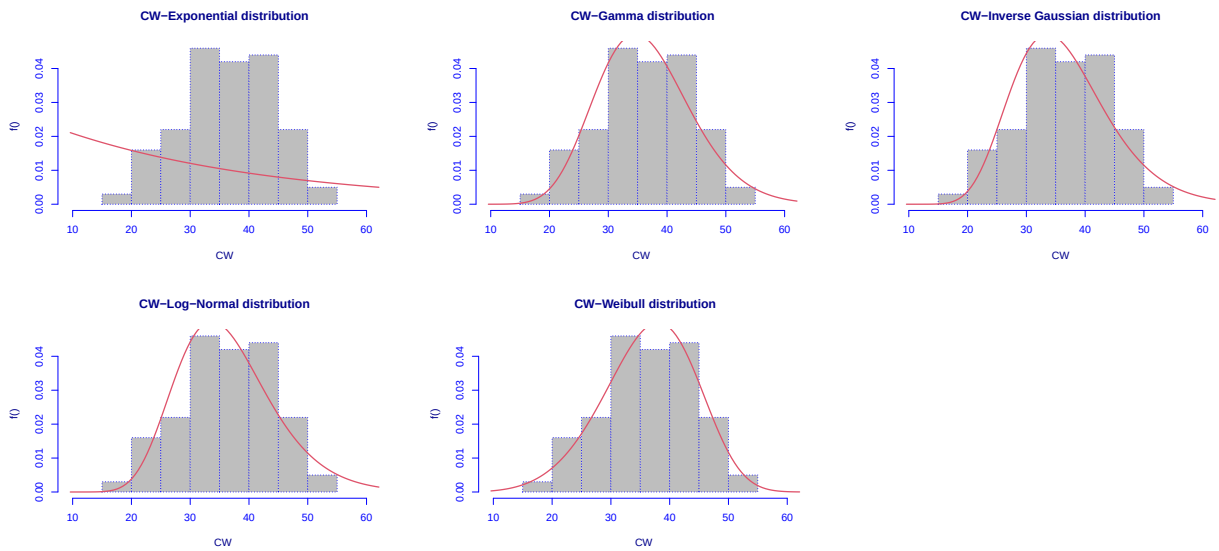
In the upcoming codes, we will attempt to find the best-fitting model for the CW data by considering three criteria: maximizing the Log-likelihood value and minimizing the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC).

```
round(2*nrow(crabs)**(1/3))
```

```
## [1] 12
```

```
CW.fit.EXP <- histDist(crabs$CW, family=EXP, xlab= "CW", nbins =12
                        ,main="CW-Exponential distribution")
CW.fit.GA <- histDist(crabs$CW, family=GA, xlab= "CW ", nbins =12,
                               main="CW-Gamma distribution")
CW.fit.IG <- histDist(crabs$CW, family=IG, xlab= " CW ", nbins =12,
                               main="CW-Inverse Gaussian distribution")
CW.fit.LOGNO <- histDist(crabs$CW, family=LOGNO, xlab= "CW ", nbins =12,
                                  main="CW-Log-Normal distribution")
```

```
CW.fit.WEI <- histDist(crabs$CW, family=WEI, xlab= "CW ", nbins =12,
                                  main="CW-Weibull distribution")
```



A summary table of the Bic, the Aic and Log-likelihood values for each model is shown.

```
CW.fitted<-matrix(c(CW.fit.EXP$df.fit, logLik(CW.fit.EXP),
                              AIC(CW.fit.EXP), CW.fit.EXP$sbc,
CW.fit.GA$df.fit, logLik(CW.fit.GA), AIC(CW.fit.GA),
                              CW.fit.GA$sbc,
CW.fit.LOGNO$df.fit, logLik(CW.fit.LOGNO),
                              AIC(CW.fit.LOGNO), CW.fit.LOGNO$sbc,
CW.fit.IG$df.fit, logLik(CW.fit.IG), AIC(CW.fit.IG),
                             CW.fit.IG$sbc,
CW.fit.WEI$df.fit, logLik(CW.fit.WEI),
                             AIC(CW.fit.WEI), CW.fit.WEI$sbc), ncol=4, byrow=TRUE)
colnames(CW.fitted)<-c("df","LogLik", "AIC", "BIC")
rownames(CW.fitted)<-c("EXP", "GA", "LOGNO", "IG", "WEI")
CW.fitted<-as.table(CW.fitted)
CW.fitted
```

```
##              df    LogLik       AIC       BIC
## EXP      1.0000 -918.9934 1839.9868 1843.2851
## GA       2.0000 -699.5006 1403.0012 1409.5979
## LOGNO    2.0000 -703.4746 1410.9491 1417.5458
## IG       2.0000 -703.6143 1411.2285 1417.8251
## WEI      2.0000 -695.3094 1394.6188 1401.2154
```

Based on the Maximum Likelihood method, the BIC criterion, and the AIC criterion, the best-fitting model among the fitted ones is the Weibull distribution with two parameters. This model exhibits the highest Log-likelihood value (-695.3094) and the lowest values for both AIC (1394.6188) and BIC (1401.2154).

Likelihood-ratio test

The Likelihood-ratio test goodness of fit performed between the Exponential model (under the null hypothesis) and the Weibull model (under the alternative hypothesis).

```
#install.packages("lmtest")
#library(lmtest)
lrtest(CW.fit.EXP, CW.fit.WEI)
```

```
## Likelihood ratio test
##
## Model 1: gamlssML(formula = crabs$CW, family = "EXP")
## Model 2: gamlssML(formula = crabs$CW, family = "WEI")
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1   1 -918.99
## 2   2 -695.31  1 447.37  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The null hypothesis would be rejected at nearly every significance level. Thus, we know that we should definitely use the Weibull model as it increases the accuracy of our model by a substantial amount.

```
LR.test(CW.fit.EXP, CW.fit.WEI)
```

```
##  Likelihood Ratio Test for nested GAMLSS models.
##  (No check whether the models are nested is performed).
##
##        Null model: deviance= 1837.987 with  1 deg. of freedom
##  Altenative model: deviance= 1390.619 with  2 deg. of freedom
##
##  LRT = 447.3681 with 1 deg. of freedom and p-value= 0
```

Upon comparing the Exponential distribution (null hypothesis) with the Weibull distribution (alternative hypothesis), the p-value is determined to be 0, which is less than the significance level of 0.05. Therefore, the null hypothesis, suggesting the Exponential distribution as the appropriate model for the data, is rejected. It can be concluded that the Weibull distribution is the correct model choice as it exhibits lower AIC and BIC values compared to the Exponential distribution.

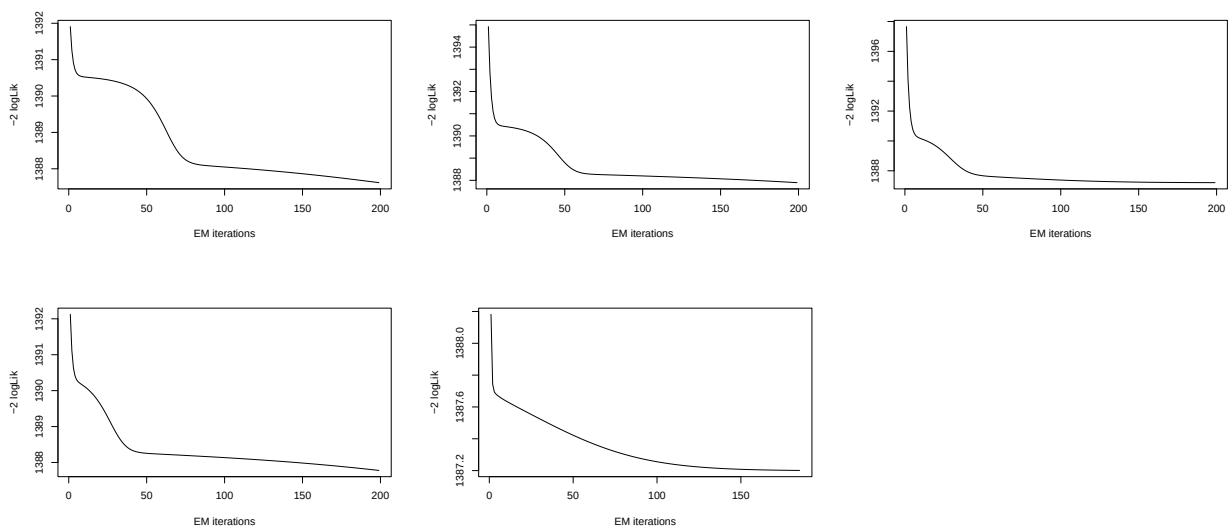
Mixture of distributions

It is possible to compute a mixture of two Weibull distributions. In order to find the best mixture, the algorithm is repeated five times.

```
# install.packages("gamlss")
# install.packages("gamlss.mx")
library(gamlss.mx)
CW.mix.WEI <- gamlssMXfits(n = 5, crabs$CW~1, family = WEI,
                                    K = 2, data = crabs)
```

```
## model= 1
```

```
## model= 2

## model= 3

## model= 4

## model= 5
```



By using a mixture of two Weibull distributions, it is observed that the AIC and BIC values have improved compared to those obtained with a single Weibull distribution. The lower AIC and BIC values indicate a better fit and suggest that the mixture model provides a more accurate representation of the data.The log-likelihood value of the mixture model is -693.6036, which is higher than the log-likelihood value obtained from fitting a single Weibull distribution (-695.3094). This suggests that the mixture model provides a better fit to the data.

```r
CW.mix.WEI$aic
```

```
## [1] 1397.201
```

```r
CW.mix.WEI$sbc
```

```
## [1] 1413.693
```

```r
logLik(CW.mix.WEI)
```

```
## 'log Lik.' -693.6005 (df=5)
```

```r
CW.mix.WEI$prob
```

```
## [1] 0.2508362 0.7491638
```

```r
fitted(CW.mix.WEI, "mu")[1]
```

```
## [1] 39.07297
```

```r
fitted(CW.mix.WEI, "sigma")[2]
```

```
## [1] 39.07297
```

We can see that with the mixture of two Weibull distributions the AIC and BIC values have increased. The log-likelihood value of the mixture is -693.6005, greater than the log-likelihood value computed on a single Weibull distribution (-695.3094).

```r
mu.hat1 <- exp(CW.mix.WEI[["models"]][[1]][["mu.coefficients"]])
sigma.hat1 <- exp(CW.mix.WEI[["models"]][[1]][["sigma.coefficients"]])

mu.hat2 <- exp(CW.mix.WEI[["models"]][[2]][["mu.coefficients"]])
sigma.hat2 <- exp(CW.mix.WEI[["models"]][[2]][["sigma.coefficients"]])

hist(crabs$CW, breaks = 50,freq = FALSE, xlab = "CW",
     main="CW-Mixture of two Weibull distributions")

lines(seq(min(crabs$CW),max(crabs$CW),CW=CW(crabs$CW)),
      CW.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$CW),
max(crabs$CW),CW=CW(crabs$CW)), mu = mu.hat1, sigma = sigma.hat1),
lty=2,lwd=3,col=2)
```

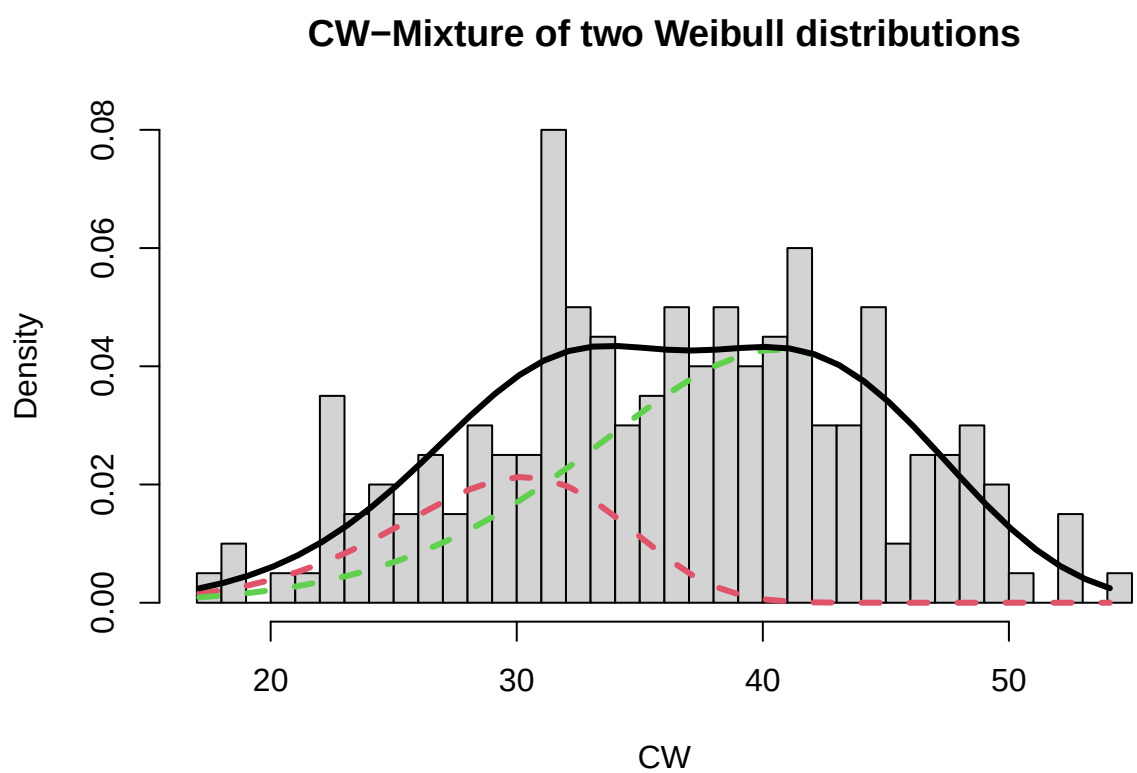
```
## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded

## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded
```

```r
lines(seq(min(crabs$CW),max(crabs$CW),CW=CW(crabs$CW)),
 CW.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$CW),max(crabs$CW)
,CW=CW(crabs$CW)), mu = mu.hat2, sigma = sigma.hat2),lty=2,lwd=3,col=3)
```

```
## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded

## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded
```

```r
lines(seq(min(crabs$CW),max(crabs$CW),CW=CW(crabs$CW)),
 CW.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$CW),max(crabs$CW),
 CW=CW(crabs$CW)), mu = mu.hat1, sigma = sigma.hat1) +
 CW.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$CW),
 max(crabs$CW),CW=CW(crabs$CW)), mu = mu.hat2, sigma = sigma.hat2),
 lty = 1, lwd = 3, col = 1)
```

```
## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded

## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded

## Warning: In seq.default(min(crabs$CW), max(crabs$CW), CW = CW(crabs$CW)) :
##  extra argument 'CW' will be disregarded
```



With K=2 selected for the mixture model, the plot displays two peaks corresponding to each distribution. The dotted red line represents the first distribution, the dotted green line represents the second distribution, and the solid black line represents the overall mixture model for all the data. In this mixture model, the first Weibull distribution accounts for 74.91% while the second Weibull distribution accounts for 25.09%.

#BD

Body Depth is a numerical variable.

```r
summary(crabs$BD)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    6.10   11.40   13.90   14.03   16.60   21.60
```

```r
length(crabs$BD)
```

```
## [1] 200
```

The length of BD variable is 200 and range is from 6.1 to 21.6.

```
sd(crabs$BD)
```

```
## [1] 3.424772
```

```
var(crabs$BD)
```

```
## [1] 11.72907
```

```
# Create the function to find mode.
getmode <- function(v) {
uniqv <- unique(v)
uniqv[which.max(tabulate(match(v, uniqv)))]
}
# Create the vector with numbers.
v <- c(crabs$BD)
# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
## [1] 13.8
```

Standard deviation and variance are respectively 3.42 and 11.72.

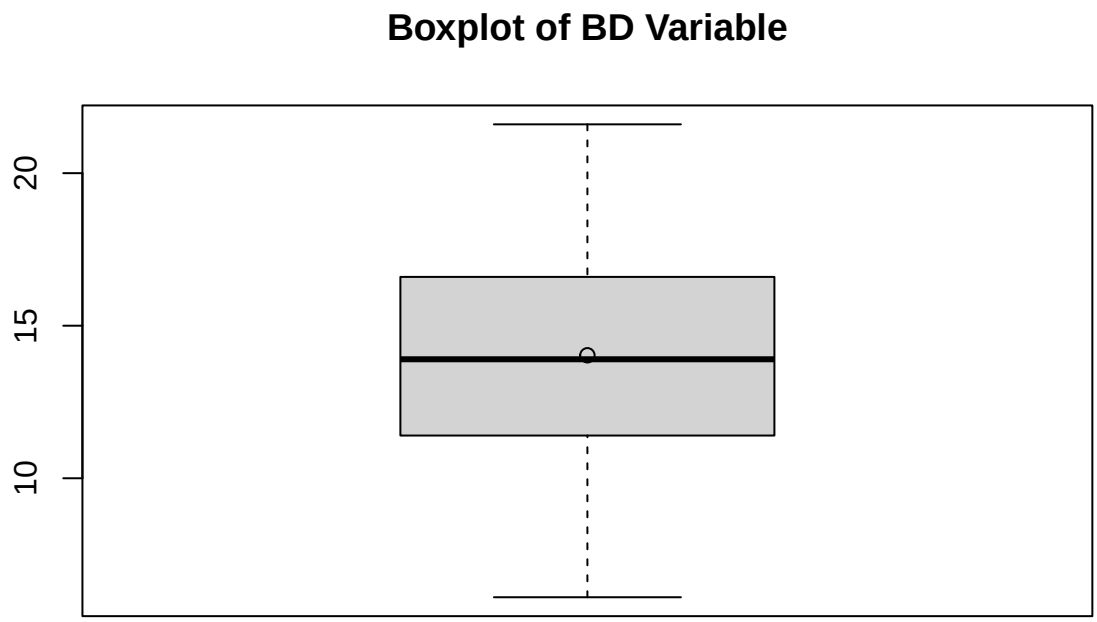
Based on the results, the mean and median values are approximately equal, indicating an almost symmetrical distribution without skewness. The similarity between the mean and median further supports the conclusion that the distribution is nearly equal. Additionally, the skewness value, as shown in the subsequent command, is close to zero, further confirming the lack of skewness in the distribution.

```
# install.packages("moments")
# library(moments)
skewness(crabs$BD)
```

```
## [1] 0.01937031
```

#Boxplot

```
boxplot(crabs$BD, main = "Boxplot of BD Variable")
points(mean(crabs$BD))
```

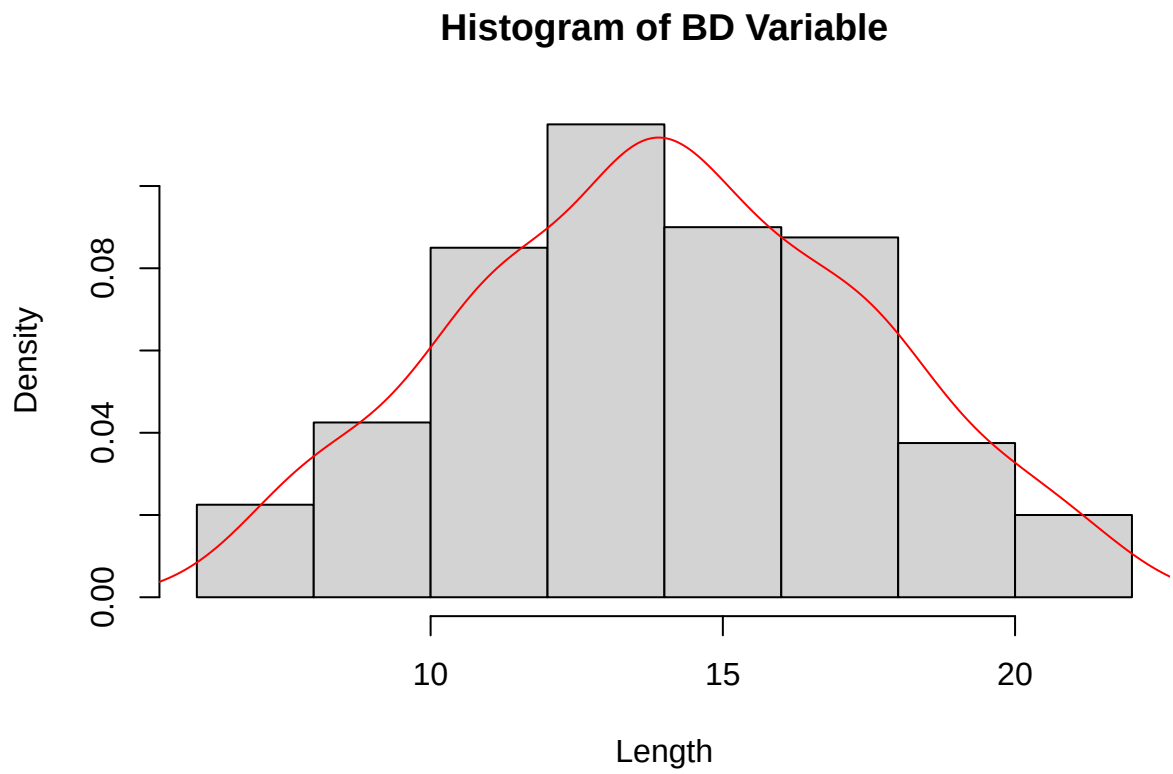


We can see that in this variable there is no outlier.

Now, we can plot the histogram and look for the model that fits the distribution better.

#Histogram

```
hist(crabs$BD,xlab = "Length" ,freq=FALSE, main = "Histogram of BD Variable")
lines(density(crabs$BD), col="red")
```



As the dataset is referring to the crabs dataset, from above plot, we can realize that the distribution of data above mean is equal to below mean so we can assume that the distribution of crabs both sides of the 14.03 are equal. If we look at the histogram of data, we can see that the diagram has two peaks. Also, we can see that the right tail is equal to the left tail and it was obvious for us as we saw already that the skewness is zero.

Data fitting for BD

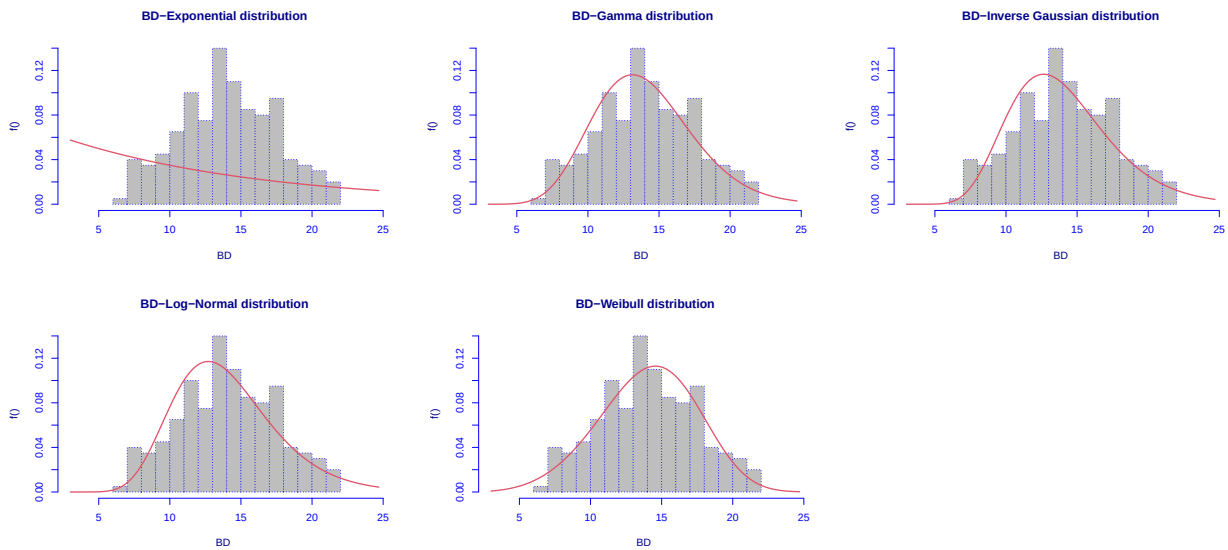
In the following codes, we will try to establish the model to fit the BD data, by looking at three different criterions: the Log-likelihood value (which has to be maximized), the Akaike Information Criterion (AIC) and the Bayesian Information criterion (BIC) (which have to be minimized).

```
round(2*nrow(crabs)**(1/3))
```

```
## [1] 12
```

```
BD.fit.EXP <- histDist(crabs$BD, family=EXP, xlab= "BD", nbins =12
                        ,main="BD-Exponential distribution")
BD.fit.GA <- histDist(crabs$BD, family=GA, xlab= "BD ", nbins =12,
                               main="BD-Gamma distribution")
BD.fit.IG <- histDist(crabs$BD, family=IG, xlab= " BD ", nbins =12,
                               main="BD-Inverse Gaussian distribution")
BD.fit.LOGNO <- histDist(crabs$BD, family=LOGNO, xlab= "BD ", nbins =12,
                                  main="BD-Log-Normal distribution")
```

```
BD.fit.WEI <- histDist(crabs$BD, family=WEI, xlab= "BD ", nbins =12,
                                 main="BD-Weibull distribution")
```



A summary table of the Bic, the Aic and Log-likelihood values for each model is shown.

```
BD.fitted<-matrix(c(BD.fit.EXP$df.fit, logLik(BD.fit.EXP),
                              AIC(BD.fit.EXP), BD.fit.EXP$sbc,
BD.fit.GA$df.fit, logLik(BD.fit.GA), AIC(BD.fit.GA),
                              BD.fit.GA$sbc,
BD.fit.LOGNO$df.fit, logLik(BD.fit.LOGNO),
                              AIC(BD.fit.LOGNO), BD.fit.LOGNO$sbc,
BD.fit.IG$df.fit, logLik(BD.fit.IG), AIC(BD.fit.IG),
                             BD.fit.IG$sbc,
BD.fit.WEI$df.fit, logLik(BD.fit.WEI),
                             AIC(BD.fit.WEI), BD.fit.WEI$sbc), ncol=4, byrow=TRUE)
colnames(BD.fitted)<-c("df","LogLik", "AIC", "BIC")
rownames(BD.fitted)<-c("EXP", "GA", "LOGNO", "IG", "WEI")
BD.fitted<-as.table(BD.fitted)
BD.fitted
```

```
##              df    LogLik       AIC       BIC
## EXP      1.0000 -728.2467 1458.4934 1461.7917
## GA       2.0000 -531.7527 1067.5053 1074.1020
## LOGNO    2.0000 -535.6565 1075.3131 1081.9097
## IG       2.0000 -535.7790 1075.5580 1082.1546
## WEI      2.0000 -529.4582 1062.9164 1069.5130
```

According to the Maximum Likelihood method, the BIC criterion and the AIC criterion, the best model between the fitted ones is the Weibull distribution (two parameters' distributions). The model has the maximum value of Log-likelihood (-529.4582) and the minimum value both AIC (1062.9164) and BIC (1069.5130).

Likelihood-ratio test

The Likelihood-ratio test goodness of fit performed between the Exponential model (under the null hypothesis) and the Weibull model (under the alternative hypothesis).

```
#install.packages("lmtest")
#library(lmtest)
lrtest(BD.fit.EXP, BD.fit.WEI)
```

```
## Likelihood ratio test
##
## Model 1: gamlssML(formula = crabs$BD, family = "EXP")
## Model 2: gamlssML(formula = crabs$BD, family = "WEI")
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1   1 -728.25
## 2   2 -529.46  1 397.58  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The null hypothesis would be rejected at nearly every significance level. Thus, we know that we should definitely use the Weibull model as it increases the accuracy of our model by a substantial amount.

```
LR.test(BD.fit.EXP, BD.fit.WEI)
```

```
##  Likelihood Ratio Test for nested GAMLSS models.
##  (No check whether the models are nested is performed).
##
##        Null model: deviance= 1456.493 with  1 deg. of freedom
##  Altenative model: deviance= 1058.916 with  2 deg. of freedom
##
##  LRT = 397.5771 with 1 deg. of freedom and p-value= 0
```

We are comparing the Exponential distribution, under the null hypothesis, with the Weibull one, under the alternative hypothesis. Since the p-value is equal to 0, which is less than the significance level of 0.05, we can reject the null hypothesis, according to which the right model for our data, between the two aforementioned, was the Exponential and say that, instead, the right model is the Weibull one, which has, in fact, AIC and BIC values lower than the ones of the Exponential distribution. The p-values is 0: the null model, i.e. the Exponential distribution is rejected.

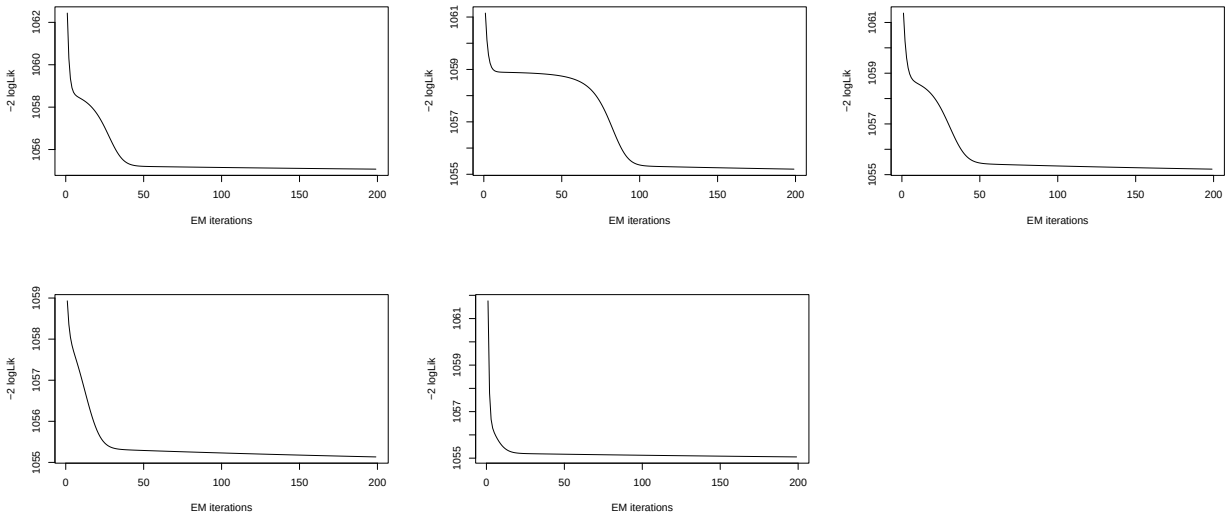
Mixture of distributions

To find the best mixture of two Weibull distributions, the algorithm is repeated five times. This iterative process helps identify the optimal combination of parameters for the mixture model.

```
# install.packages("gamlss")
# install.packages("gamlss.mx")
library(gamlss.mx)
BD.mix.WEI <- gamlssMXfits(n = 5, crabs$BD~1, family = WEI,
                                    K = 2, data = crabs)
```

```
## model= 1
```

```
## model= 2

## model= 3

## model= 4

## model= 5
```




```
BD.mix.WEI$aic
```

```
## [1] 1065.053
```

```
BD.mix.WEI$sbc
```

```
## [1] 1081.545
```

```
logLik(BD.mix.WEI)
```

```
## 'log Lik.' -527.5265 (df=5)
```

```
BD.mix.WEI$prob
```

```
## [1] 0.639933 0.360067
```

```
fitted(BD.mix.WEI, "mu")[1]
```

```
## [1] 15.08488
```

```r
fitted(BD.mix.WEI, "sigma")[2]
```

```
## [1] 15.08488
```

We can see that with the mixture of two Weibull distributions the AIC and BIC values have increased. The log-likelihood value of the mixture is -527.5516, greater than the log-likelihood value computed on a single Weibull distribution (-529.4582).

```r
mu.hat1 <- exp(BD.mix.WEI[["models"]][[1]][["mu.coefficients"]])
sigma.hat1 <- exp(BD.mix.WEI[["models"]][[1]][["sigma.coefficients"]])

mu.hat2 <- exp(BD.mix.WEI[["models"]][[2]][["mu.coefficients"]])
sigma.hat2 <- exp(BD.mix.WEI[["models"]][[2]][["sigma.coefficients"]])

hist(crabs$BD, breaks = 50,freq = FALSE, xlab = "BD",
     main="BD-Mixture of two Weibull distributions")

lines(seq(min(crabs$BD),max(crabs$BD),BD=BD(crabs$BD)),
      BD.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$BD),
max(crabs$BD),BD=BD(crabs$BD)), mu = mu.hat1, sigma = sigma.hat1),
lty=2,lwd=3,col=2)
```

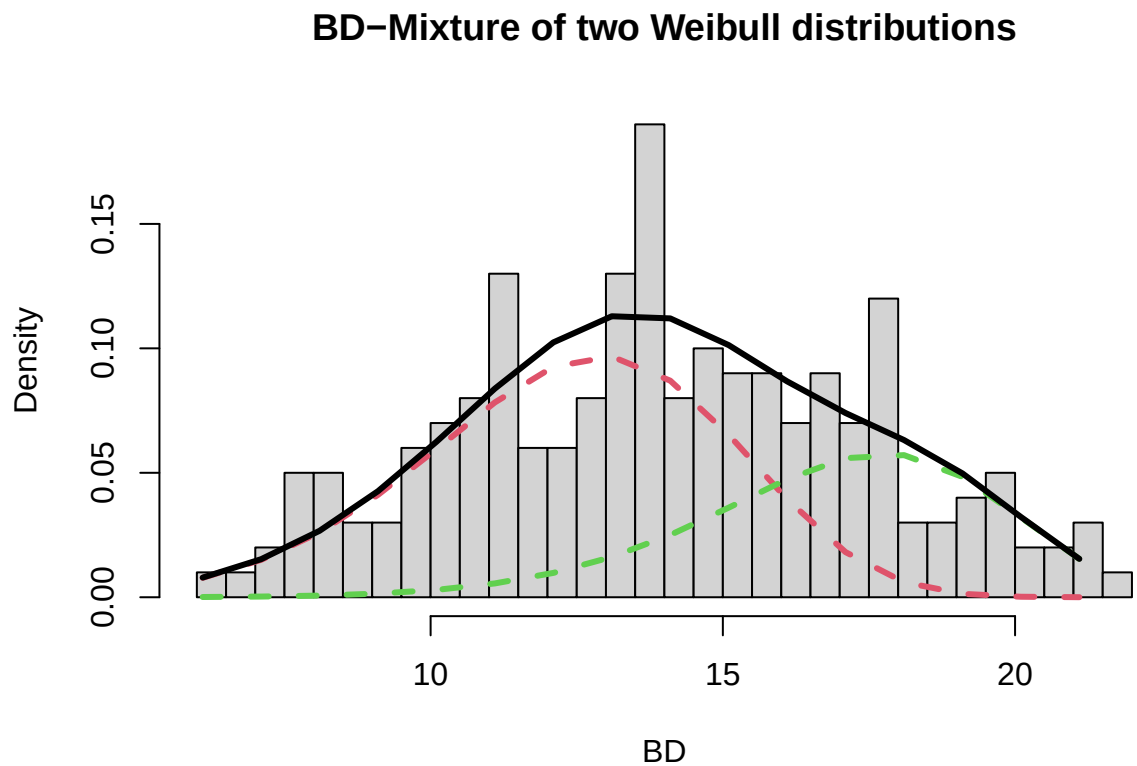
```
## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded

## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded
```

```r
lines(seq(min(crabs$BD),max(crabs$BD),BD=BD(crabs$BD)),
 BD.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$BD),max(crabs$BD)
,BD=BD(crabs$BD)), mu = mu.hat2, sigma = sigma.hat2),lty=2,lwd=3,col=3)
```

```
## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded

## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded
```

```r
lines(seq(min(crabs$BD),max(crabs$BD),BD=BD(crabs$BD)),
 BD.mix.WEI[["prob"]][1]*dWEI(seq(min(crabs$BD),max(crabs$BD),
 BD=BD(crabs$BD)), mu = mu.hat1, sigma = sigma.hat1) +
 BD.mix.WEI[["prob"]][2]*dWEI(seq(min(crabs$BD),
 max(crabs$BD),BD=BD(crabs$BD)), mu = mu.hat2, sigma = sigma.hat2),
 lty = 1, lwd = 3, col = 1)
```

```
## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded

## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded

## Warning: In seq.default(min(crabs$BD), max(crabs$BD), BD = BD(crabs$BD)) :
##  extra argument 'BD' will be disregarded
```

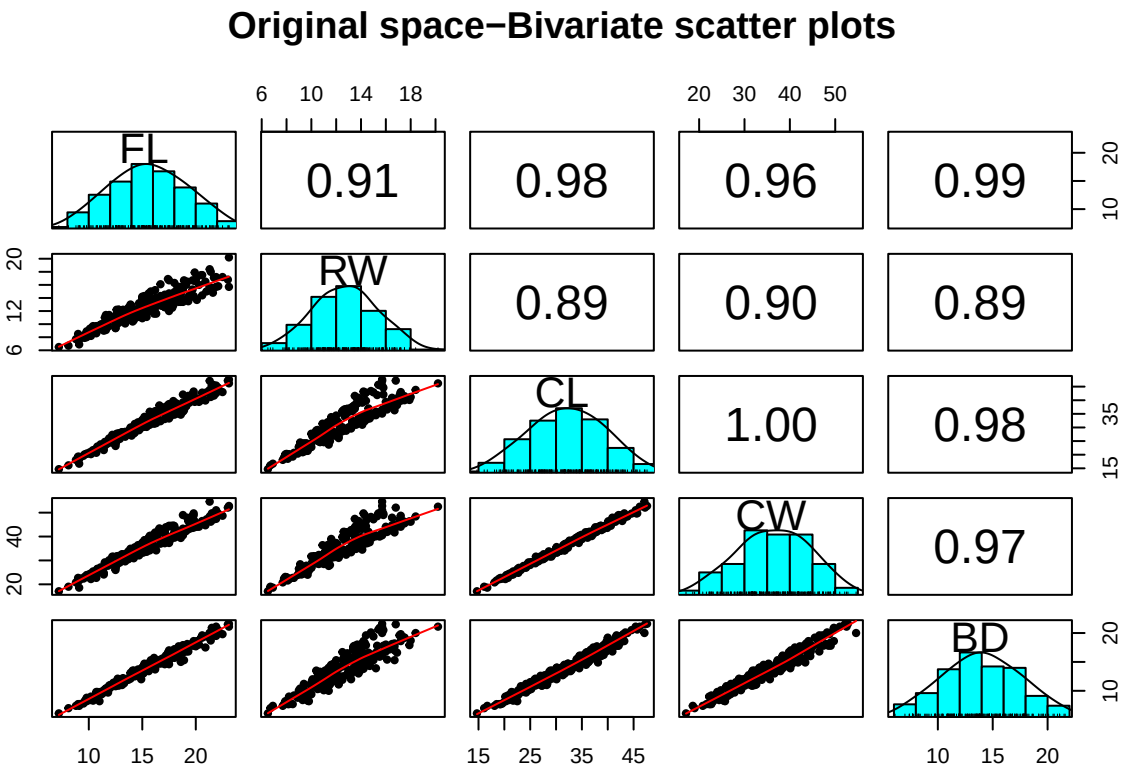


Since we have selected K=2, we can see in the plot 2 peaks, each corresponding to a distribution. The dotted red line is relative to the first distribution, the dotted green one to the second distribution, while the black line is relative to the mixture, i.e. the overall model for all data. In this mixture (solid black line), the first Weibull distribution (dotted red line) accounts for 65% the second Weibull distribution (dotted green line) accounts for 35%.

(2) Principal Component Analysis

Before proceeding with the PCA, it is necessary to evaluate whether there is a correlation between the numerical variables. For this aim first we need a sub-dataset with all continuous values.

```
crabs_p<-crabs[,-(1:3)]
pairs.panels(crabs_p, main= "Original space-Bivariate scatter plots", ellipses = FALSE)
```



This preliminary analysis of the data in the original space aims to determine the suitability of running Principal Component Analysis (PCA) and Cluster Analysis (CA) on the dataset. The upper triangle of the correlation matrix provides insights into the correlations between variables, which help assess the potential usefulness of PCA. The lower triangle of the matrix displays scatterplots of the data, while the main diagonal shows the non-parametric density of the variables, both of which help evaluate the potential usefulness of CA.

Upon examining the plot, it is observed that all variables exhibit correlations with each other. For example, CL and CW have the highest correlation of 1, while BD and RW have a correlation of 0.89. Since all correlations are positive and in the same direction, increasing one variable leads to an increase in the other variable and vice versa. This suggests that conducting a PCA on this dataset could be useful, as it may enable the creation of a linear combination of variables expressed through a single variable.

Regarding the diagonal panels, they help determine whether individual variables are useful for clustering. If the density line on the diagonal is bimodal, it suggests that the respective variable could be valuable for clustering. In this case, when considering each variable separately, they do not appear useful for identifying clusters. However, when examining pairwise combinations of variables in the scatterplots in the lower triangle, clusters become evident. The data points tend to group along the diagonal rather than being scattered throughout space, indicating the presence of clusters.

Overall, based on this preliminary analysis, conducting PCA and Cluster Analysis on the dataset seems promising. PCA can capture underlying relationships among variables and potentially reduce dimensionality, while Cluster Analysis can help identify distinct groups or clusters within the data.

Prepare the data

In order to evaluate the difference between the variables, the mean and the variance are computed for each variable.

```
apply(crabs_p, 2, mean)
```

```
##      FL      RW      CL      CW      BD
## 15.5830 12.7385 32.1055 36.4145 14.0305
```

```
apply(crabs_p, 2, var)
```

```
##        FL        RW        CL        CW        BD
## 12.217297  6.622078 50.679919 61.967678 11.729065
```

Due to the substantial differences among the variables, it is advisable to standardize them to achieve homogeneity. Standardizing the variables involves transforming them to have a mean of zero and a variance of one. This process ensures that the variables are on a comparable scale, facilitating consistent analysis and interpretation.

```
scaled_crabs<-apply(crabs_p, 2, scale)
head(scaled_crabs, 5)
```

```
##             FL        RW        CL        CW        BD
## [1,] -2.140860 -2.346561 -2.248285 -2.212220 -2.052837
## [2,] -1.940592 -1.957961 -1.967346 -1.983561 -1.936041
## [3,] -1.826153 -1.919101 -1.840923 -1.780307 -1.848444
## [4,] -1.711715 -1.880241 -1.686407 -1.691384 -1.702449
## [5,] -1.654496 -1.841381 -1.658313 -1.704087 -1.702449
```

Computing PCs (Eigen decomposition)

In order to find the Principal Components, the Eigen decomposition is applied to the covariance matrix of the standardized data.

```
crabs.cov <- cov(scaled_crabs)
crabs.eigen <- eigen(crabs.cov)
crabs.eigen$value
```

```
## [1] 4.788834784 0.151685207 0.046632974 0.011135357 0.001711678
```

The eigenvalues of the principal components represent the amount of variance explained by each component. They indicate the relative importance or contribution of each principal component in capturing the variability within the dataset.

The eigenvectors matrix displays the loadings for each principal component. These loadings represent the weights or coefficients assigned to the original variables in each principal component. The eigenvectors of the first three principal components are typically provided, showing the relationships between the original variables and these specific components.

```r
phi <- crabs.eigen$vectors[,1:3]
phi <- -phi
row.names(phi) <- c("FL", "RW", "CL","CW","BD")
colnames(phi) <- c("PC1", "PC2", "PC3")
phi
```

```
##          PC1        PC2         PC3
## FL 0.4520437  0.1375813 -0.53076841
## RW 0.4280774 -0.8981307  0.01197915
## CL 0.4531910  0.2682381  0.30968155
## CW 0.4511127  0.1805959  0.65256956
## BD 0.4511336  0.2643219 -0.44316103
```

Upon examining the loading vectors, we observe that the first loading vector (PC1) assigns a higher weight to the variable CL (0.4532) compared to RW (0.4281). This indicates that CL has a stronger influence on the first principal component. The second loading vector (PC2) assigns a significant weight to the variable RW (-0.8981) and a lesser weight to FL (0.1376). This suggests that the second principal component is primarily driven by the RW variable.

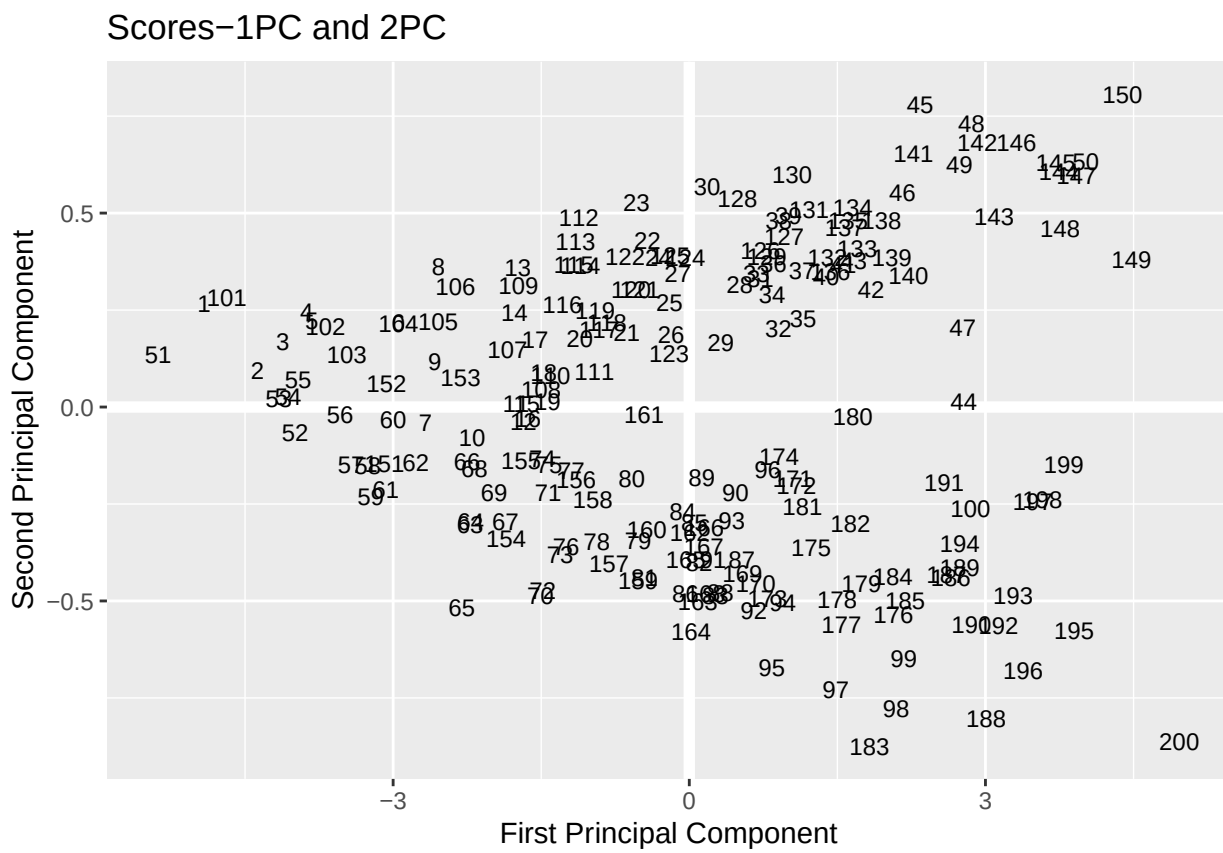
principal component scores

For each observation in the dataset, the scores are computed with respect the PC1 and PC2.

```r
PC1 <- scaled_crabs %*% phi[,1]
PC2 <- scaled_crabs %*% phi[,2]
PC <- data.frame(ID = row.names(crabs), PC1, PC2)
head(PC)
```

```
##   ID       PC1        PC2
## 1  1 -4.915239 0.26777335
## 2  2 -4.375197 0.09383811
## 3  3 -4.118329 0.16845321
## 4  4 -3.873960 0.24539253
## 5  5 -3.824458 0.22360515
## 6  6 -2.945563 0.21946999
```

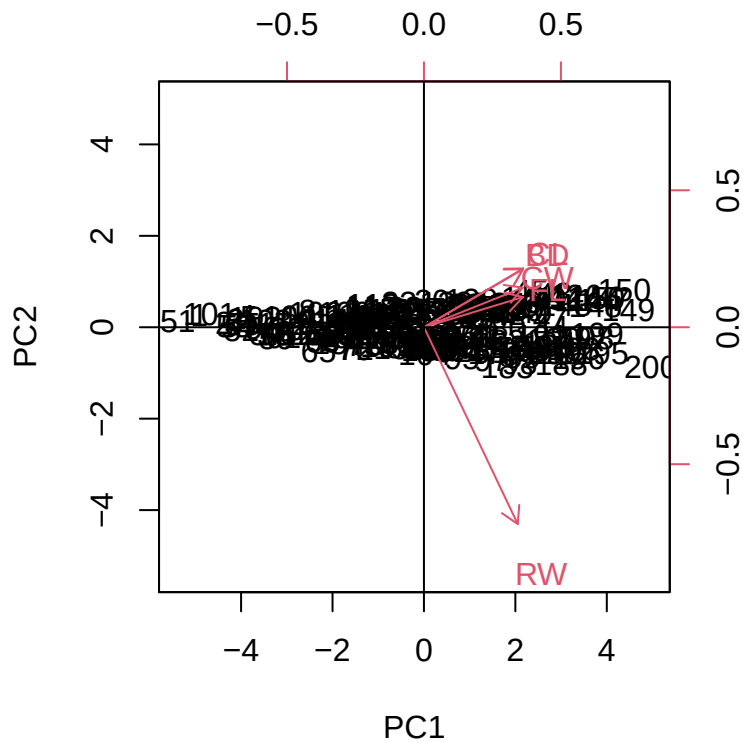
```r
ggplot(PC, aes(PC1, PC2)) +
 modelr::geom_ref_line(h = 0) +
 modelr::geom_ref_line(v = 0) +
 geom_text(aes(label = ID), size = 3) +
 xlab("First Principal Component") +
 ylab("Second Principal Component") +
 ggtitle("Scores-1PC and 2PC")
```



Biplot

In the space defined by the first two principal components, it is feasible to visually represent the scores and the original variables using arrows.

```
set.seed(123)
crabs_pc <- prcomp(crabs_p, scale. = TRUE)
biplot(crabs_pc, scale=0)
abline(h=0)
abline(v=0)
```



The angle between the arrows provides information about the correlation between the two variables.Longer arrows indicate variables that have a larger influence on the corresponding principal components.

```
cor(crabs_p)
```

```
##           FL        RW        CL        CW        BD
## FL 1.0000000 0.9069876 0.9788418 0.9649558 0.9876272
## RW 0.9069876 1.0000000 0.8927430 0.9004021 0.8892054
## CL 0.9788418 0.8927430 1.0000000 0.9950225 0.9832038
## CW 0.9649558 0.9004021 0.9950225 1.0000000 0.9678117
## BD 0.9876272 0.8892054 0.9832038 0.9678117 1.0000000
```

Selecting the Number of Principal Components

To select the number of principal components, three heuristic methods are proposed.

Cumulative proportion of Variance Explained (CPVE)

To implement this approach, we retain the first q principal components that collectively account for at least 80% of the total variance.

```r
PVE <- crabs.eigen$values/sum(crabs.eigen$values)
round(PVE, 3)
```

```
## [1] 0.958 0.030 0.009 0.002 0.000
```

- The first PC explains 95.8% of the variability; • The second PC explains 3.0% of the variability;

```r
cumsum(PVE)
```

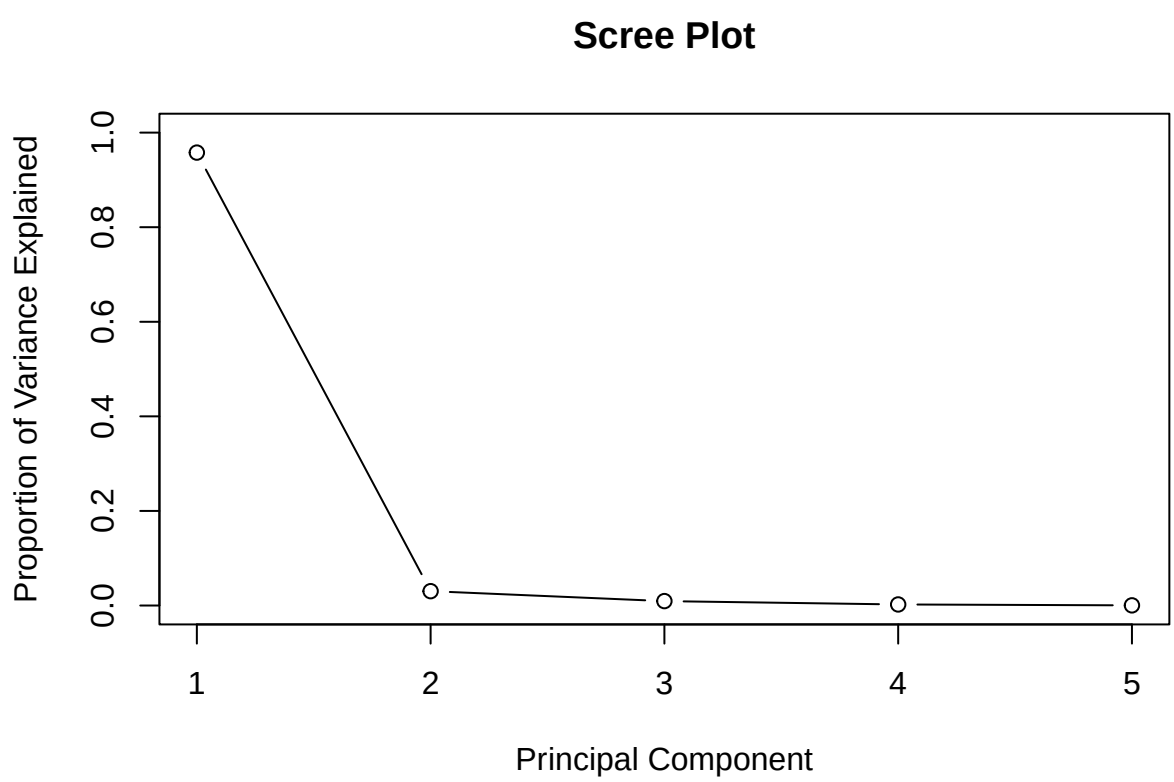
```
## [1] 0.9577670 0.9881040 0.9974306 0.9996577 1.0000000
```

According to this method, the first principal component is retained because it explain the 95.8% of the total variance.

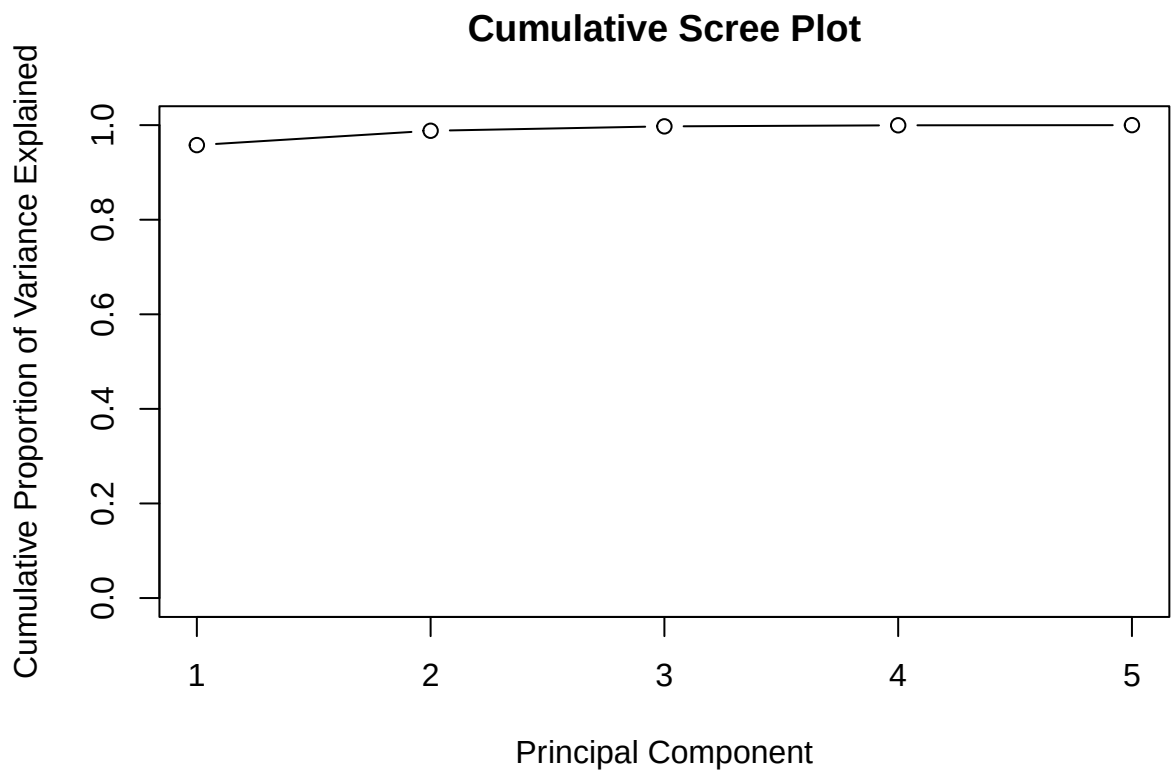
Scree plot

The scree plot suggests selecting the value of q that corresponds to the point on the curve where it becomes flat. This is often used as an indication of the optimal number of principal components to retain.

```r
plot(PVE, xlab="Principal Component", ylab="Proportion of Variance Explained",
     main="Scree Plot", ylim=c(0,1), type='b')
```



```r
plot(cumsum(PVE), xlab="Principal Component", main="Cumulative Scree Plot",
     ylab="Cumulative Proportion of Variance Explained", ylim=c(0,1),type='b')
```



In the "Proportion of Variance Explained" plot, it can be difficult to identify a clear elbow point. However, based on this method, it would be reasonable to retain the first three principal components, as they capture a substantial amount of the variance in the dataset.

Kaiser's rule

According to the Kaiser's rule, for standardized data, the principal components with the eigenvalue greater than 1 are taken.

```r
crabs.eigen$values
```

```
## [1] 4.788834784 0.151685207 0.046632974 0.011135357 0.001711678
```

The Kaiser's rule suggests keeping the first principal components.

PCA result

Different strategies can lead to different results when determining the number of principal components to retain. The cumulative proportion of variance explained (PVE) rule suggests retaining the number of

principal components needed to reach a certain threshold (e.g., 80% or 90%) of cumulative PVE. Kaiser's rule suggests retaining principal components with eigenvalues greater than 1. The Scree plot suggests selecting the number of principal components corresponding to the point where the plot starts to level off or show a significant decrease in eigenvalues. In our case, the cumulative PVE rule suggests retaining the first principal component, Kaiser's rule suggests retaining the first principal component, and the Scree plot suggests retaining the first three principal components. Ultimately, the choice of how many principal components to retain depends on the specific context and goals of the analysis.

Multivariate Analysis:

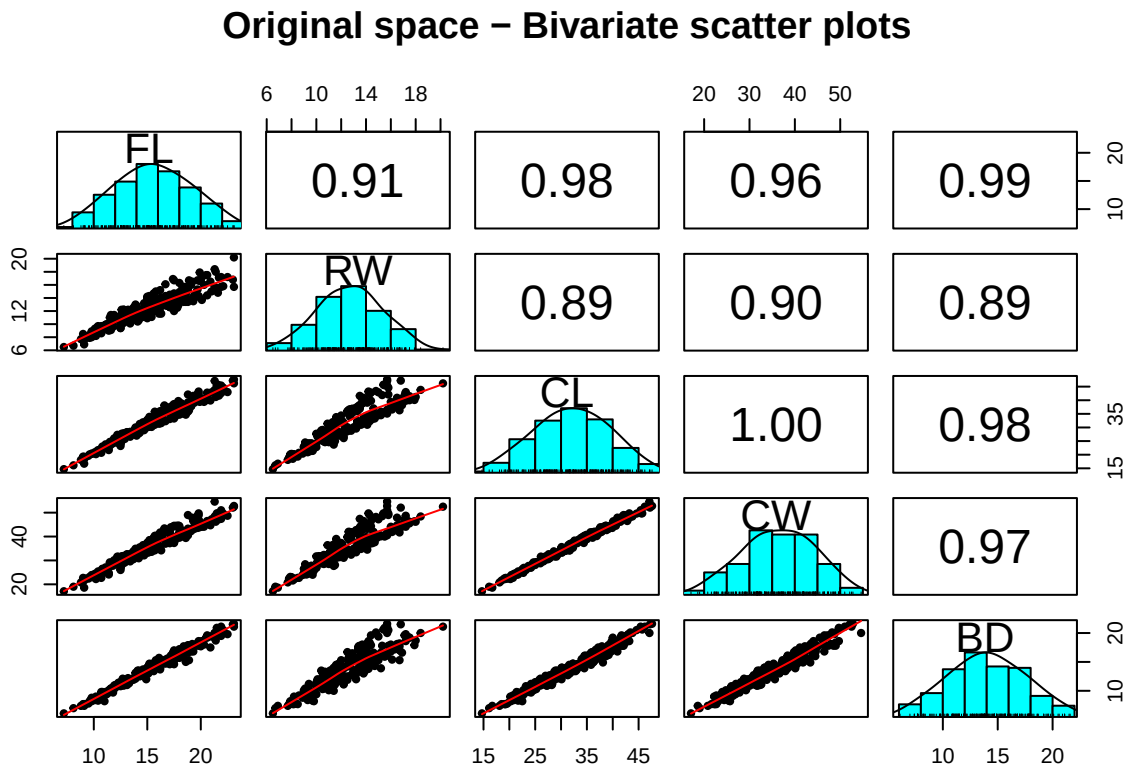
Cluster analysis

Cluster Analysis (CA) or clustering is a significant statistical method used to uncover information in data. Its objective is to identify patterns or groups (clusters) of similar units within a dataset. Before conducting cluster analysis, it is necessary to evaluate whether the dataset contains meaningful clusters. Therefore, the purpose of clustering analysis in the context of the crabs dataset is to identify patterns of similar units within the data.

Assessing clustering tendency

Before applying any clustering method, it's important to evaluate if dataset contains meaningful clusters or not.

```
crabs_w<-crabs[,-(1:3)]
pairs.panels(crabs_w, main= "Original space - Bivariate scatter plots",ellipses = FALSE)
```



Despite the presence of multimodal variables in the dataset, the two-dimensional space does not show significant separation of the data into distinct groups. However, there is a possibility of finding clusters when considering the dataset in a three-dimensional space.

Hopkins statistic

The Hopkins statistic (H) is a measure that ranges from 0 to 1 and indicates the clustering tendency of the data. A value of H close to 0 suggests the presence of clustered data, while a value around 0.5 indicates uniformly distributed data without meaningful clusters.

```
df_scaled_crabs <- crabs[, -(1:3)]
df_scaled_crabs <-crabs.scaled<- scale(df_scaled_crabs)
hopkins(df_scaled_crabs, n = nrow(df_scaled_crabs)-1)
```

```
## Warning: Package ‘clustertend‘ is deprecated. Use package ‘hopkins‘ instead.
```

```
## $H
## [1] 0.08872073
```

It can be seen that the crabs data set is clusterable because the H value (0.0887) is close enough to 0.

Computing Euclidean distance:

```
dist.eucl <- dist(crabs.scaled, method = "euclidean")
eucl <- round(as.matrix(dist.eucl)[1:5,1:5],2)
rownames(eucl) <- c("FL", "RW", "CL","CW","BD")
colnames(eucl) <- c("FL", "RW", "CL","CW","BD")
eucl
```

```
##      FL   RW   CL   CW   BD
## FL 0.00 0.58 0.82 1.05 1.10
## RW 0.58 0.00 0.28 0.53 0.57
## CL 0.82 0.28 0.00 0.26 0.31
## CW 1.05 0.53 0.26 0.00 0.08
## BD 1.10 0.57 0.31 0.08 0.00
```

Computing Manhattan distance:

```
dist.man <- dist(crabs.scaled, method = "manhattan")
manhattan <- round(as.matrix(dist.man)[1:5,1:5],2)
rownames(manhattan) <- c("FL", "RW", "CL","CW","BD")
colnames(manhattan) <- c("FL", "RW", "CL","CW","BD")
manhattan
```

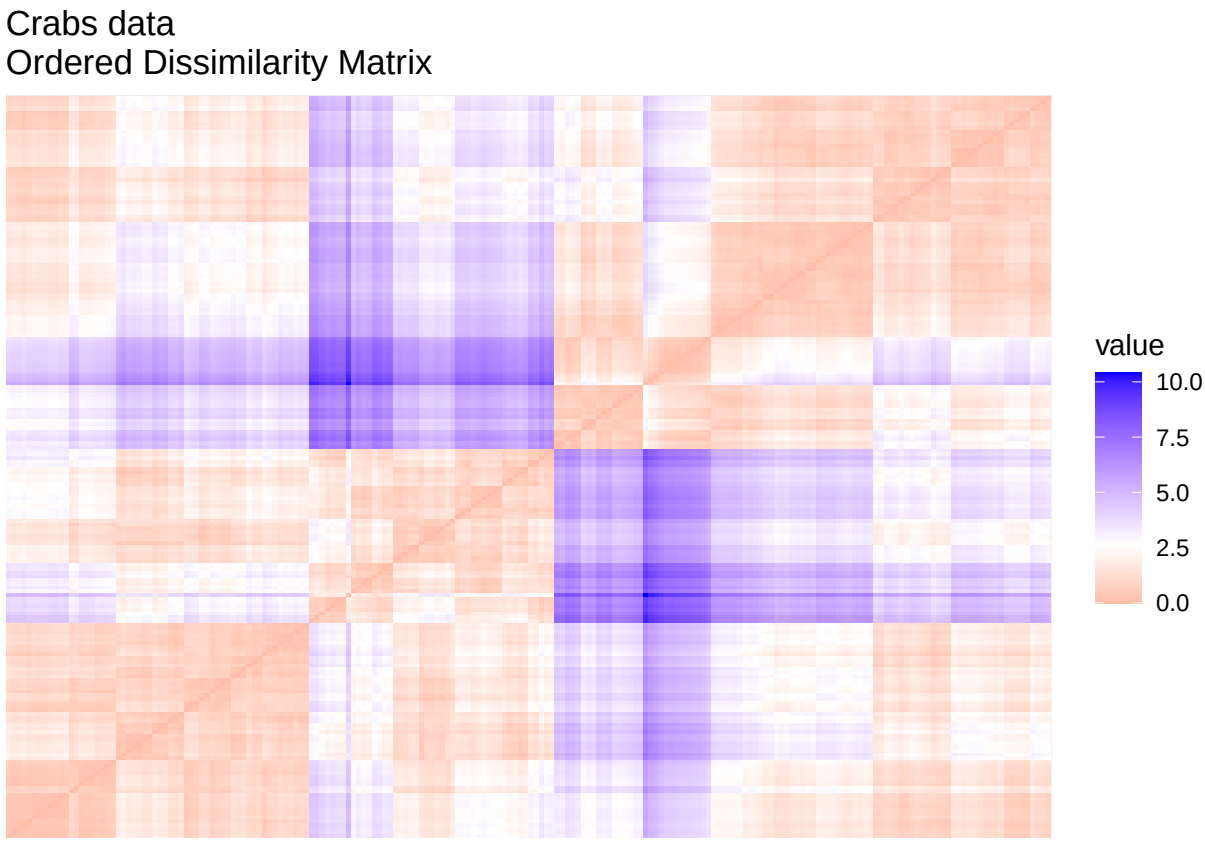
```
##      FL   RW   CL   CW   BD
## FL 0.00 1.22 1.79 2.33 2.44
## RW 1.22 0.00 0.57 1.11 1.22
## CL 1.79 0.57 0.00 0.54 0.65
## CW 2.33 1.11 0.54 0.00 0.14
## BD 2.44 1.22 0.65 0.14 0.00
```

In the above symmetric matrix, each value represents the distance between units. The values on the diagonal represent the distance between units and themselves (which is zero).

Visual Assessment of cluster Tendency (VAT) algorithm

Visual Assessment of cluster tendency is a technique used to determine whether a given dataset exhibits any inherent clustering structure. It involves visually inspecting the data to identify patterns or groupings that may suggest the presence of clusters.

```
fviz_dist(dist(df_scaled_crabs), show_labels = FALSE)+
  labs(title = "Crabs data\nOrdered Dissimilarity Matrix",
                     gradient = list(low = "red", mid = "white", high =  "blue"))
```



The color level in the visual representation is determined by the dissimilarity between observations. A higher similarity (lower dissimilarity) is indicated by red, while blue represents lower similarity. According to the visual method, which employs the Euclidean distance as the measure of dissimilarity, there is no clear clustering structure apparent in the data. Despite the uncertainty regarding the presence of clustering, the analysis will proceed.

Optimal number of clusters

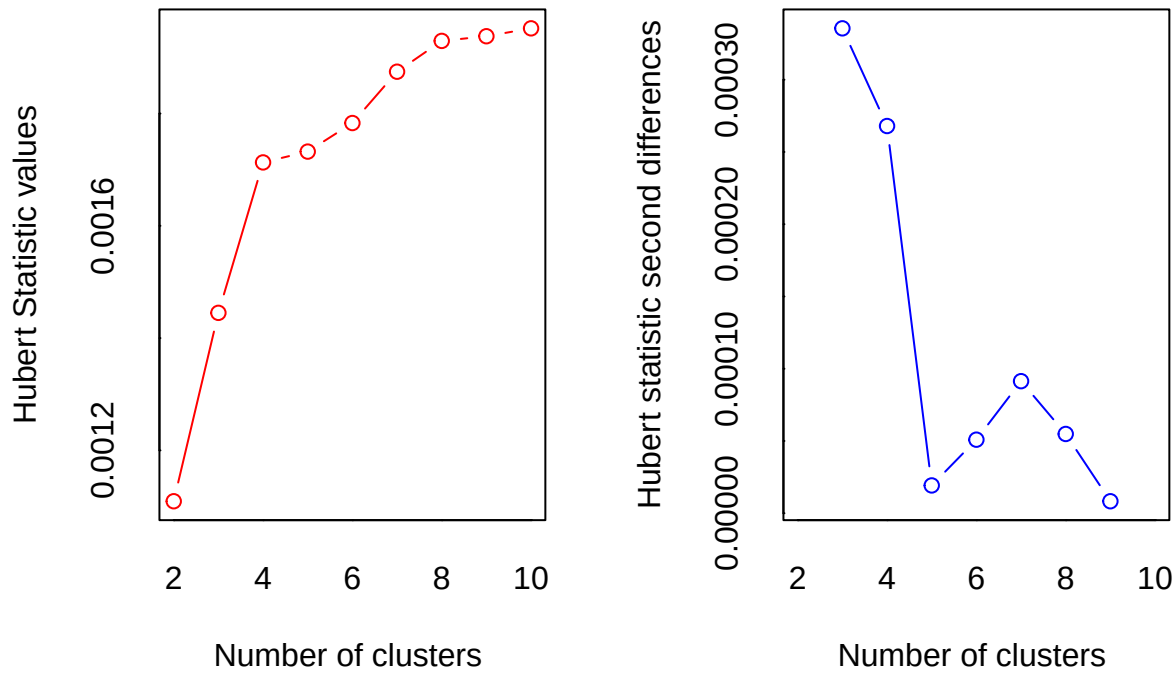
Using both Euclidean and Manhattan distance, according to different clustering methods, the optimal number of clusters will be computed.

Hard clustering approach

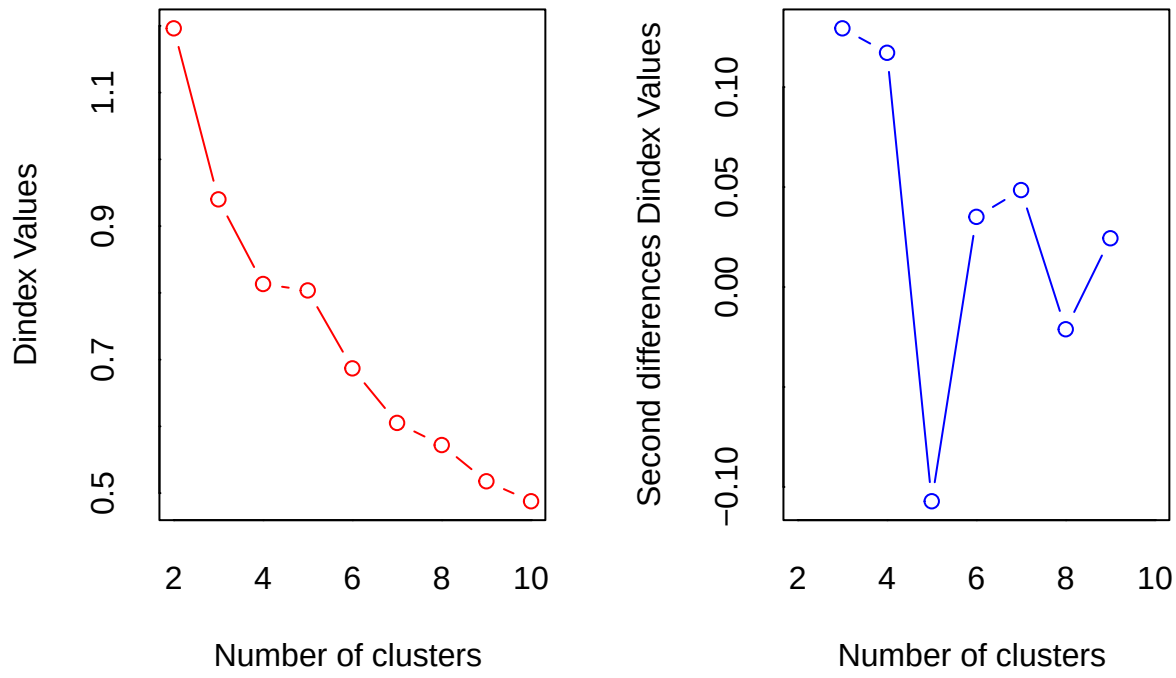
Hierarchical method

Average linkage method and Euclidean distance

```
nb <- NbClust(crabs.scaled, distance = "euclidean", min.nc = 2, max.nc = 10,
method = "average")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                   In the plot of Hubert index, we seek a significant knee that corresponds to a
##                   significant increase of the value of the measure i.e the significant peak in Hubert
##                   index second differences plot.
##
```




```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                 second differences plot) that corresponds to a significant increase of the value of
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 4 proposed 2 as the best number of clusters
## * 5 proposed 3 as the best number of clusters
## * 7 proposed 4 as the best number of clusters
## * 2 proposed 5 as the best number of clusters
## * 1 proposed 7 as the best number of clusters
## * 1 proposed 8 as the best number of clusters
## * 1 proposed 9 as the best number of clusters
## * 2 proposed 10 as the best number of clusters
##
##                    ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  4
##
##
## *******************************************************************
```

According to the result obtained from the "NbClust" analysis, the optimal number of clusters, determined using the hierarchical clustering method with average linkage and Euclidean distance, is 4.

```r
hc <- hclust(dist.eucl, method = "average")
group <- cutree(hc, k=4)
table(group)
```

```
## group
##  1  2  3  4
## 13 62 95 30
```

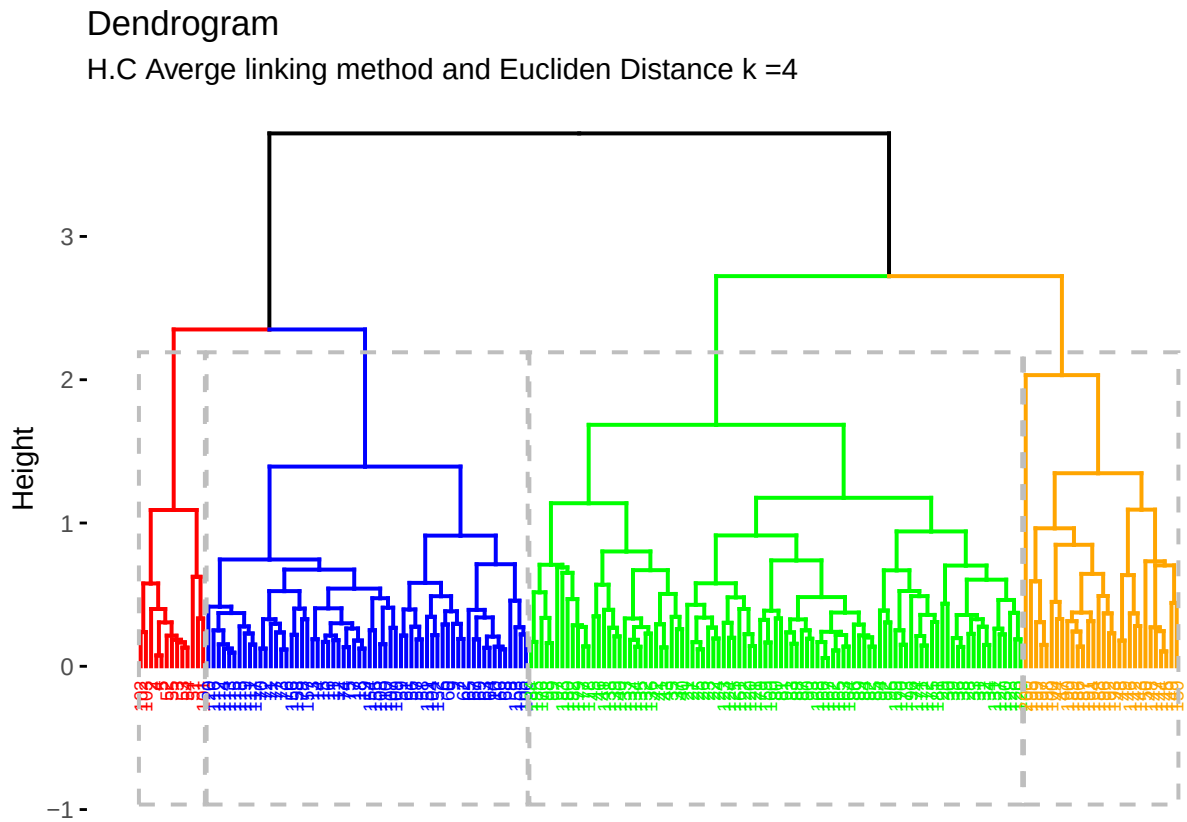
```r
group
```

```
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   1   1   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   3   3   3   4   3   3   4   4   4   4   1   1   1   1   1   2   2   2   2   2
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   3   3
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   4
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   3
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
## 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160
##   3   4   4   4   4   4   4   4   4   4   2   2   2   2   2   2   2   2   3   3
## 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
##   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3   3
## 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
##   3   3   3   3   3   4   4   4   4   4   4   4   4   4   4   4   4   4   4   4
```

In fact, cluster 3 contains more units than other clusters.

```r
fviz_dend(hc, k = 4, cex = 0.5, k_colors = c(
"red","blue","green","orange"), color_labels_by_k = TRUE, rect = TRUE)+
  labs(title = "Dendrogram", subtitle = "H.C Averge linking method and Eucliden Distance k =4")
```

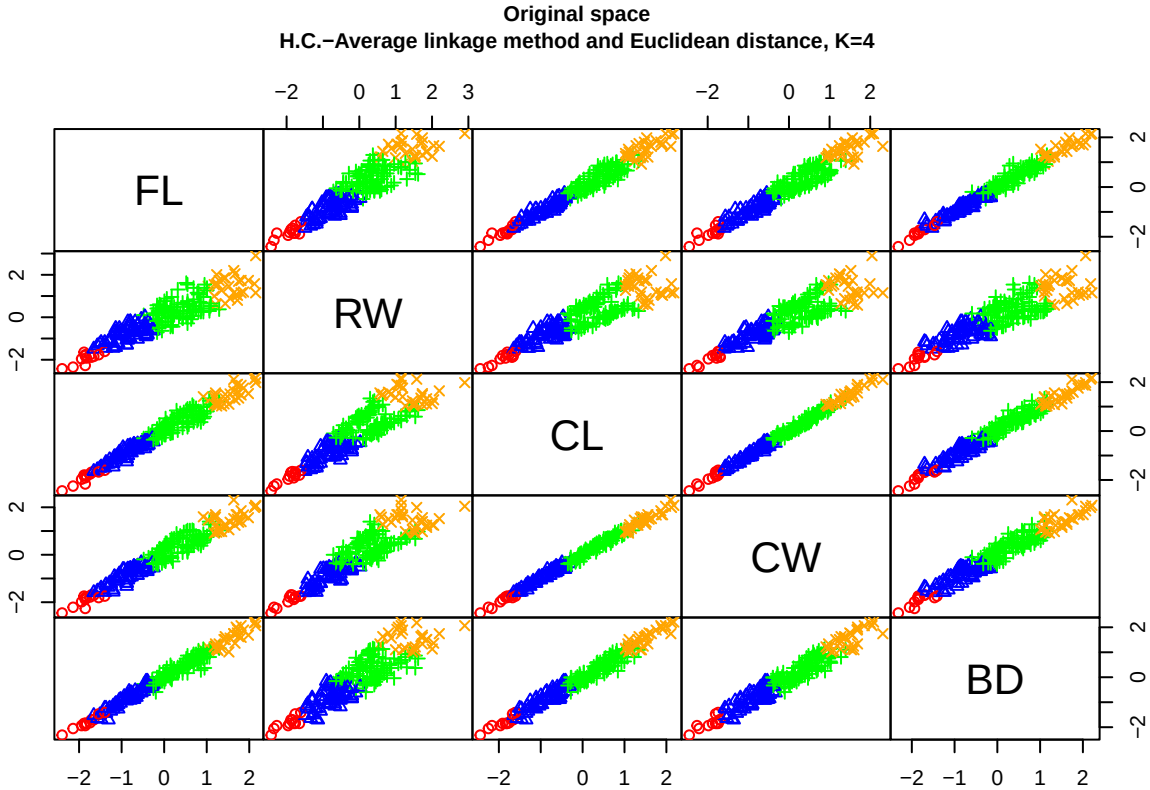
```
## Warning: The ‘<scale>‘ argument of ‘guides()‘ cannot be ‘FALSE‘. Use "none" instead as
## of ggplot2 3.3.4.
## i The deprecated feature was likely used in the factoextra package.
##   Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```




```
cor(dist.eucl, cophenetic(hc))
```

```
## [1] 0.6714525
```

```
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 0.7,
      main="Original space\nH.C.-Average linkage method and Euclidean distance, K=4" ,col=c("red","blue
```



Cluster Validation Statistics:

The term cluster validation is used to design the procedure of evaluating the goodness of clustering algorithm results. This is important to avoid finding patterns in a random data, as well as, in the situation where you want to compare two clustering algorithms. it can be classified into three categories: internal, external and relative cluster validation.

Internal cluster validation,:

This category of cluster validation methods uses the internal information of the clustering process to evaluate the goodness of a clustering structure without reference to external information. It can be also used for estimating the number of clusters and the appropriate clustering algorithm without any external data.

Silhouette Width

Silhouette width is a measure of how well-defined or compact clusters are in a clustering analysis. It quantifies the quality of clustering by evaluating the separation between clusters and the cohesion within each cluster.

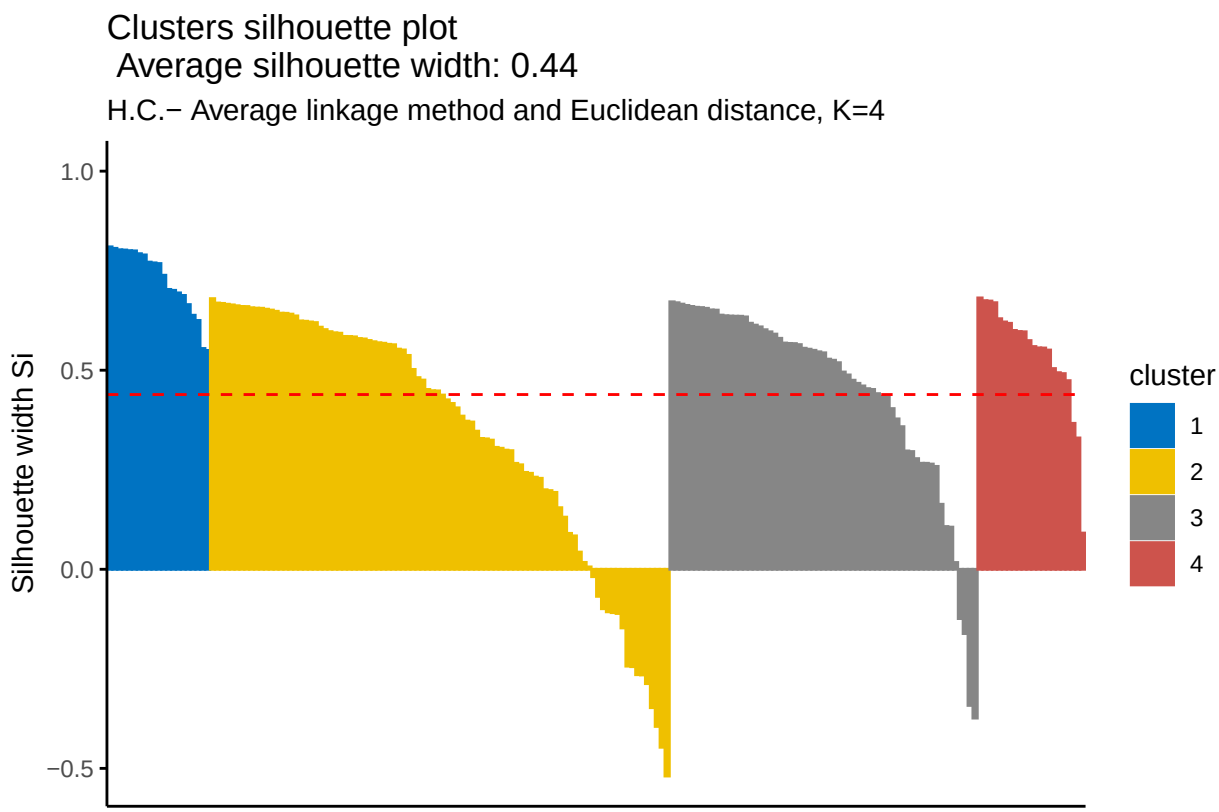
```
hclust<- eclust(crabs_p,k=4 ,"hclust",hc_method = "average",nboot = 50, hc_metric = "euclidean")
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.4388968
```

The average silhouette width is calculated to be 0.44.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+
labs(subtitle = "H.C.- Average linkage method and Euclidean distance, K=4", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   21          0.73
## 2       2   94          0.34
## 3       3   63          0.45
## 4       4   22          0.54
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.7270267 0.3439401 0.4487772 0.5412945
```

Upon examining the cluster widths, we observe that cluster 1 has the highest width, followed by cluster 4, cluster 3, and finally cluster 2. This is due to the number of units contained within each cluster. The presence of negative values in certain parts suggests that those units may exhibit some degree of overlap or similarity with neighboring clusters. Units with a negative silhouette value indicate that they are potentially assigned to the wrong cluster. To address this, it is necessary to identify their neighboring clusters, which may better reflect their characteristics or similarities.

```r
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##     cluster neighbor   sil_width
## 91        2        3 -0.01854053
## 88        2        3 -0.06791875
## 65        2        1 -0.09906052
## 106       2        1 -0.10677257
## 153       2        1 -0.10890642
## 9         2        1 -0.11098821
## 170       2        3 -0.14722086
## 173       2        3 -0.24320055
## 93        2        3 -0.24454599
## 90        2        3 -0.26469465
## 7         2        1 -0.26537942
## 87        2        3 -0.28715088
## 105       2        1 -0.34749878
## 62        2        1 -0.39467775
## 6         2        1 -0.44695119
## 60        2        1 -0.51924647
## 192       3        4 -0.12372423
## 188       3        4 -0.16137567
## 196       3        4 -0.34190913
## 193       3        4 -0.37366702
```

Dunn index:

The Dunn index is an internal clustering validation measure. If the data contain compact and well-separated clusters, the diameter of the clusters is expected to be small and the distance between the clusters is expected to be large. Thus, the Dunn index D should be maximized.

```r
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.0675329
```

According to the Dunn index, the units are not clustered well enough because it is near to 0.

External cluster validation,:

External validation methods consists in comparing the results of a cluster analysis to an externally known result, such as externally provided class labels. It measures the extent to which cluster labels match externally supplied class labels. Since we know the "true" cluster number in advance, this approach is mainly used for selecting the right clustering algorithm for a specific data set.

1.Confusion matrix:

According to the Confusion matrix the number of clusters is more than nominal values. the nominal variable can take 2 possible values.

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2  3  4
##   F 12 51 31  6
##   M  9 43 32 16
```

2.Corrected Rand index:

The Corrected Rand Index is a measure of the similarity between two data clusterings, taking into account both the pairwise agreements and disagreements between the clusters. It adjusts for chance agreement and provides a value between -1 and 1. An ARI value close to 1 indicates a high agreement between the true and predicted clusterings, while a value close to 0 indicates a random or chance agreement. A negative value suggests that the clustering results are worse than random.

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] 0.002113447
```

According to the Correct Rand Index, there is no agreement between the numerical value and the cluster solution.

3.Meila's IV index:

The Meila variation is based on information theory and evaluates the similarity between two clusterings by measuring the mutual information between the true and predicted cluster assignments.The low value indicates agreement between partitions.

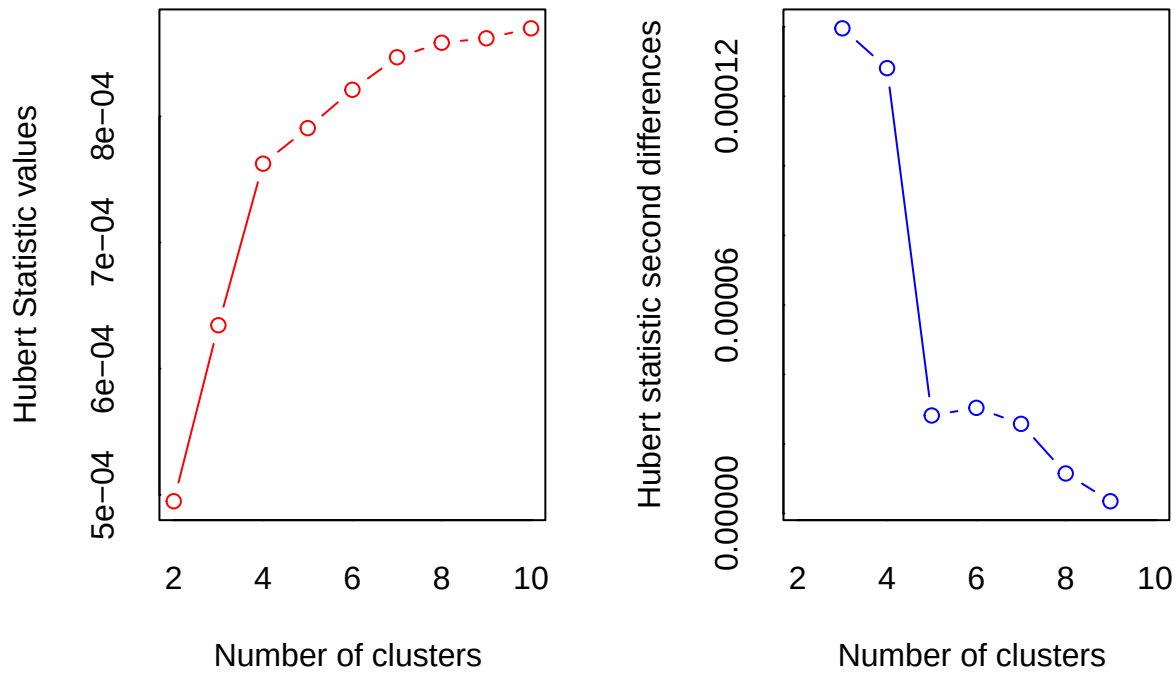
```
stats$vi
```

```
## [1] 1.862118
```

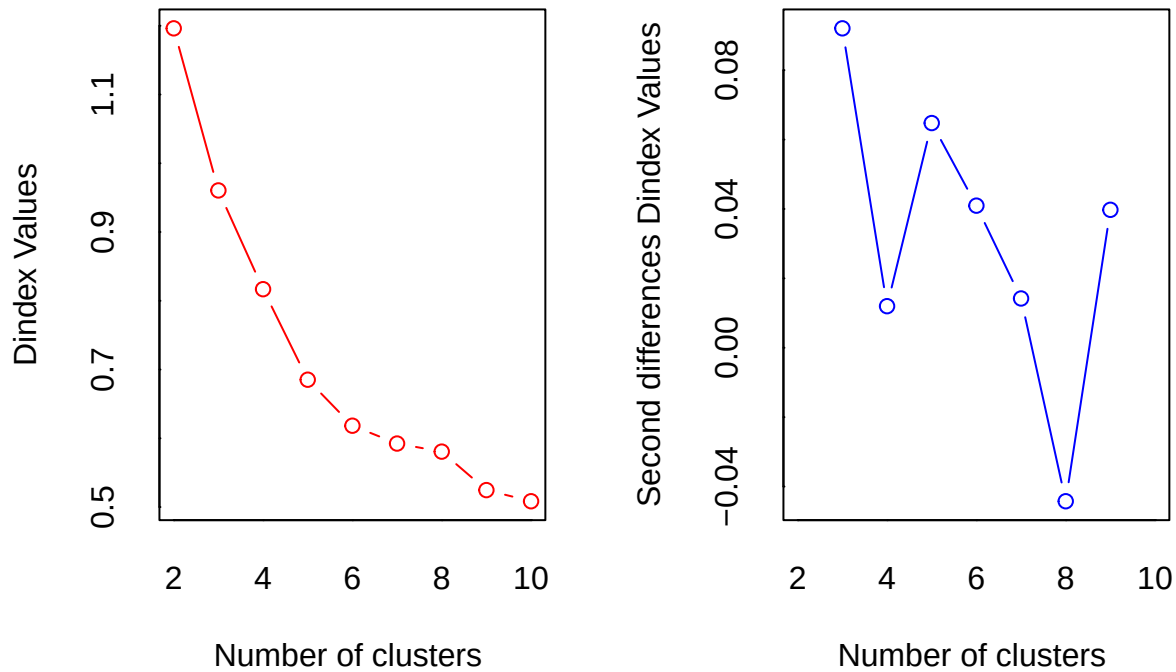
Disagreement between the numerical value and the cluster solution is 1.86.

Complete linkage method and Manhattan distance:

```
nb <- NbClust(df_scaled_crabs, distance = "manhattan", min.nc = 2, max.nc = 10,
method = "complete")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                  In the plot of Hubert index, we seek a significant knee that corresponds to a
##                  significant increase of the value of the measure i.e the significant peak in Hubert
##                  index second differences plot.
##
```



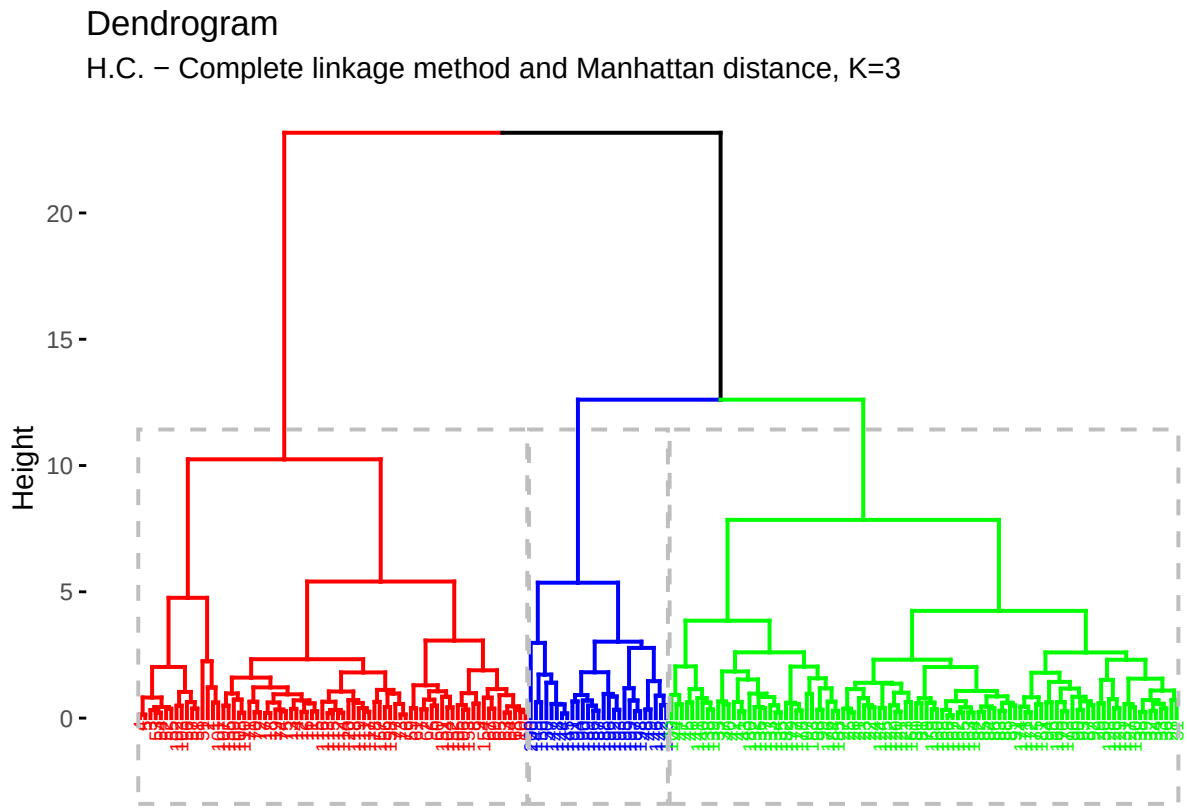

```
## *** : The D index is a graphical method of determining the number of clusters.
##                  In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                  second differences plot) that corresponds to a significant increase of the value of
##                  the measure.
##
## *******************************************************************
## * Among all indices:
## * 4 proposed 2 as the best number of clusters
## * 6 proposed 3 as the best number of clusters
## * 2 proposed 4 as the best number of clusters
## * 2 proposed 5 as the best number of clusters
## * 2 proposed 6 as the best number of clusters
## * 1 proposed 7 as the best number of clusters
## * 4 proposed 9 as the best number of clusters
## * 2 proposed 10 as the best number of clusters
##
##                     ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  3
##
##
## *******************************************************************
```

```r
dist.man <- dist(df_scaled_crabs, method = "manhattan")
hc <- hclust(dist.man, method = "complete")
```

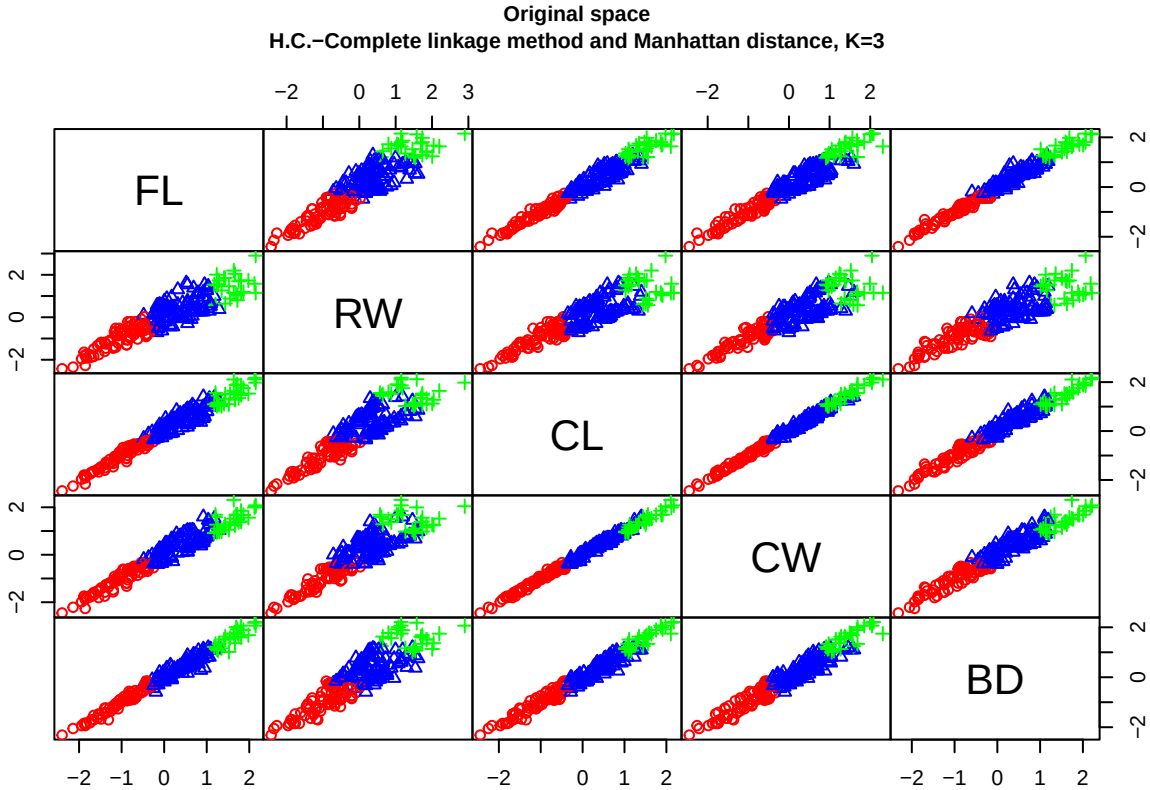
```r
group <- cutree(hc, k=3)
table(group)
```

```
## group
##  1  2  3
## 75 98 27
```

```r
fviz_dend(hc, k = 3, cex = 0.5, k_colors = c("red", "blue", "green"),
r_labels_by_k = TRUE, rect = TRUE)+labs(title = "Dendrogram",
subtitle = "H.C. - Complete linkage method and Manhattan distance, K=3",cex.subtitle= 0.5)
```

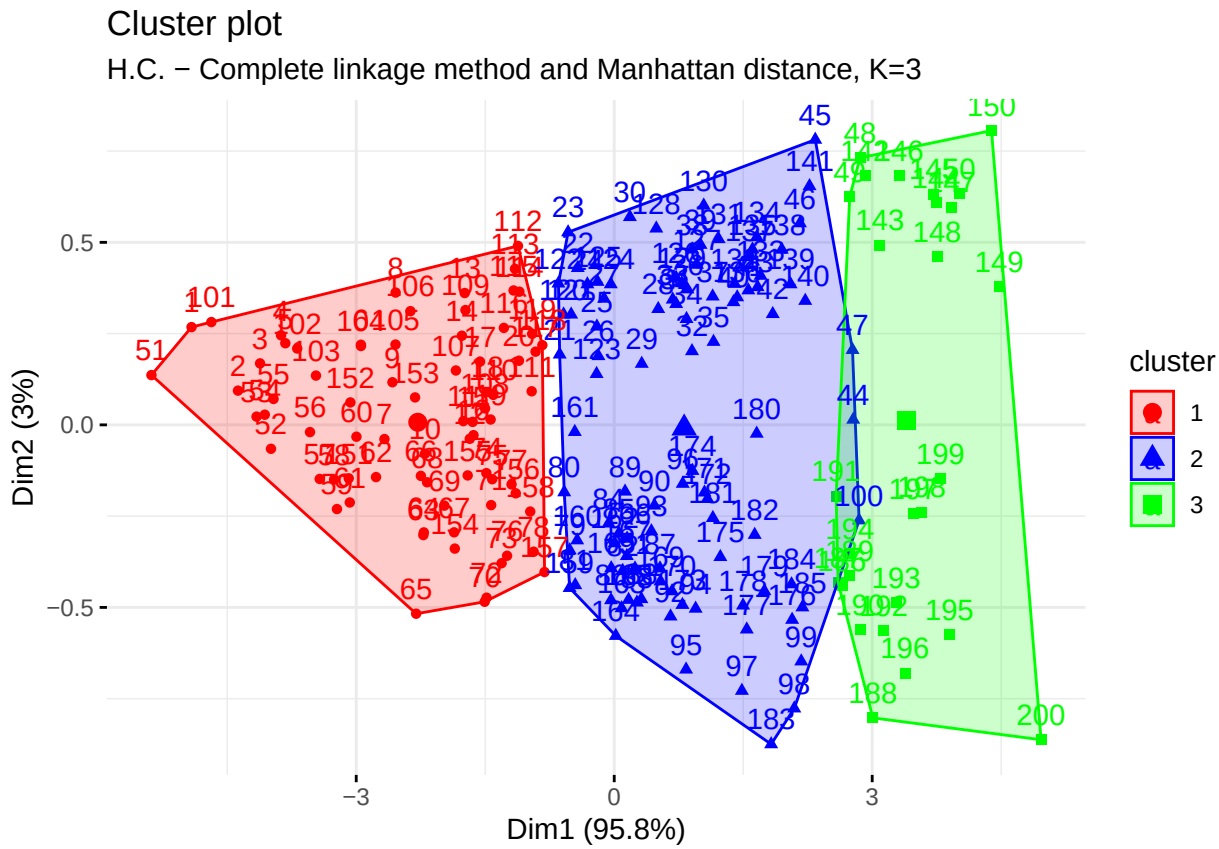



```r
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 0.7,
      main="Original space\nH.C.-Complete linkage method and Manhattan distance, K=3"
      ,col=c("red", "blue", "green")[group])
```



To have a simple bi-dimensional graphical representation, we can plot the obtained results in the space spanned by the first 2 PCs.

```
fviz_cluster(list(data = df_scaled_crabs, cluster = group), palette = c("red", "blue", "green"),
             show.clust.aver = FALSE, ggtheme = theme_minimal())+labs(
subtitle = "H.C. - Complete linkage method and Manhattan distance, K=3", cex.sub= 0.5)
```



To assess the quality of clustering algorithm results, it is important to analyze both internal and external validation measures. These measures provide insights into different aspects of the clustering performance.

Internal cluster validation:

1.Silhouette width:

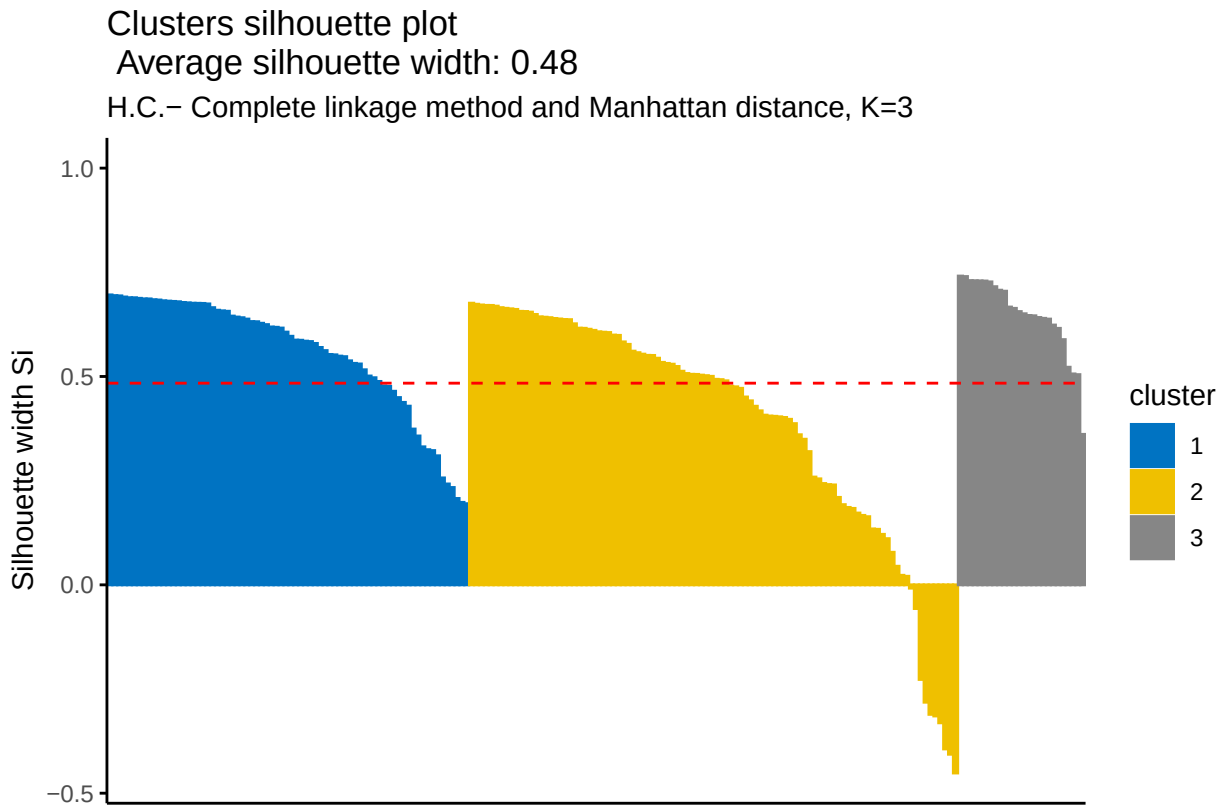
```
hclust<- eclust(crabs_p,k=3 ,"hclust",hc_method = "complete",nboot = 50, hc_metric = "manhattan")
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.4839342
```

The average silhouette width is calculated to be 0.48.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+
labs(subtitle = "H.C.- Complete linkage method and Manhattan distance, K=3", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   74          0.56
## 2       2  100          0.39
## 3       3   26          0.65
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.5551541 0.3888078 0.6471024
```

Upon examining the cluster widths, we observe that cluster 3 has the highest width, followed by cluster 1, and finally cluster 2. This is due to the number of units contained within each cluster. The presence of negative values in certain parts suggests that those units may exhibit some degree of overlap or similarity with neighboring clusters. Units with a negative silhouette value indicate that they are potentially assigned to the wrong cluster. To address this, it is necessary to identify their neighboring clusters, which may better reflect their characteristics or similarities.

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##     cluster neighbor    sil_width
## 139       2        3 -0.007983442
## 118       2        1 -0.056888474
## 140       2        3 -0.227067625
## 187       2        3 -0.281714068
## 186       2        3 -0.310809129
## 191       2        3 -0.314761313
## 141       2        3 -0.331140157
## 190       2        3 -0.393812501
## 194       2        3 -0.406555957
## 189       2        3 -0.451560728
```

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.05415108
```

According to the Dunn index, the units are not clustered well enough because the value is near 0.

External validation measures:

1.Confusion matrix:

According to the Confusion matrix, the number of clusters is equal to 3.

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2  3
##   F 36 54 10
##   M 38 46 16
```

2.Corrected Rand Index:

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.0007967338
```

According to the Correct Rand Index, there is no agreement between the numerical value and the cluster solution.

3.3.Meila's VI Index:

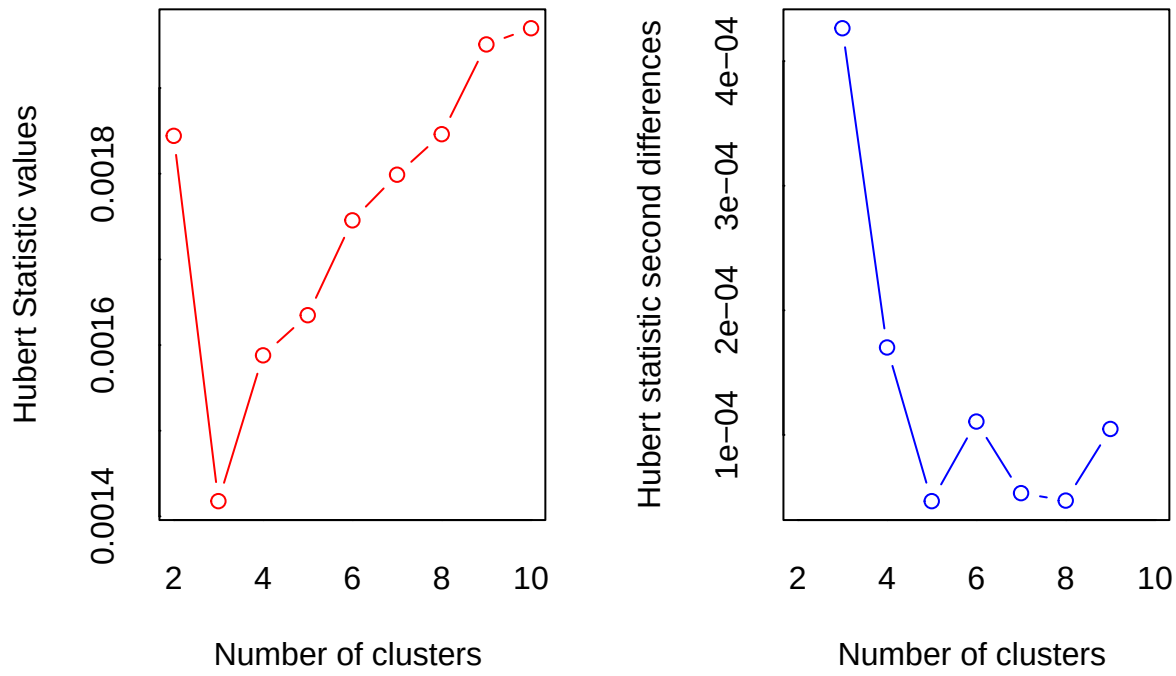
```
stats$vi
```

```
## [1] 1.662363
```

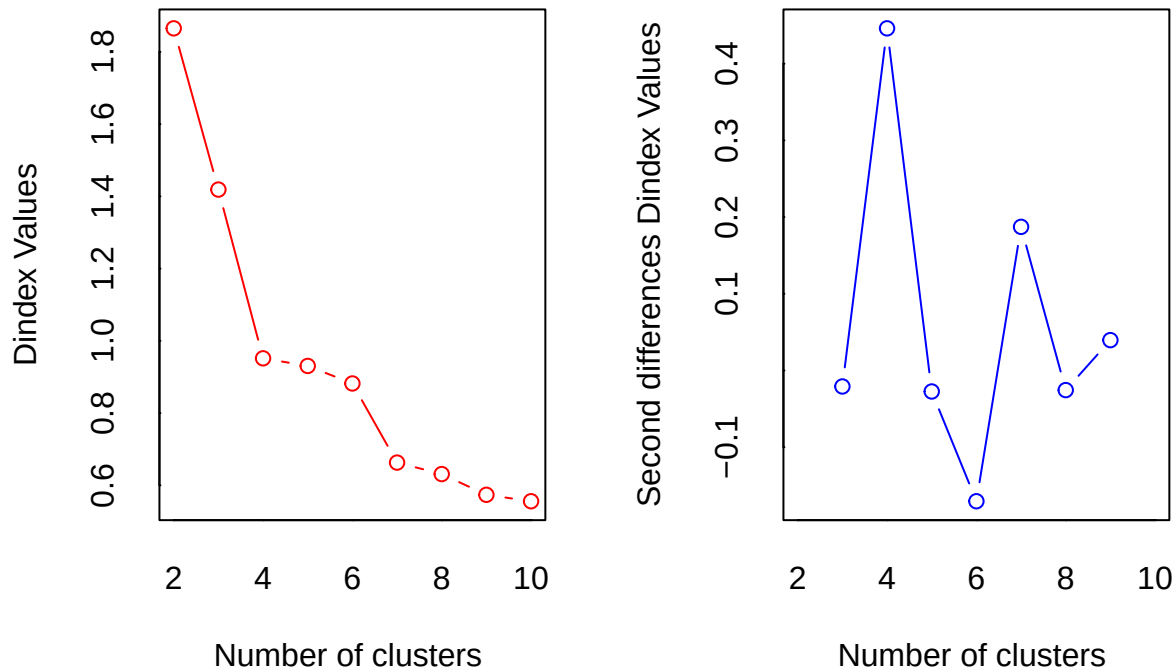
Disagreement between the numerical value and the cluster solution is 1.66.

Centroid linkage method and Euclidean distance:

```r
nb <- NbClust(df_scaled_crabs, distance = "euclidean", min.nc = 2, max.nc = 10,
method = "centroid")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                 In the plot of Hubert index, we seek a significant knee that corresponds to a
##                 significant increase of the value of the measure i.e the significant peak in Hubert
##                 index second differences plot.
##
```



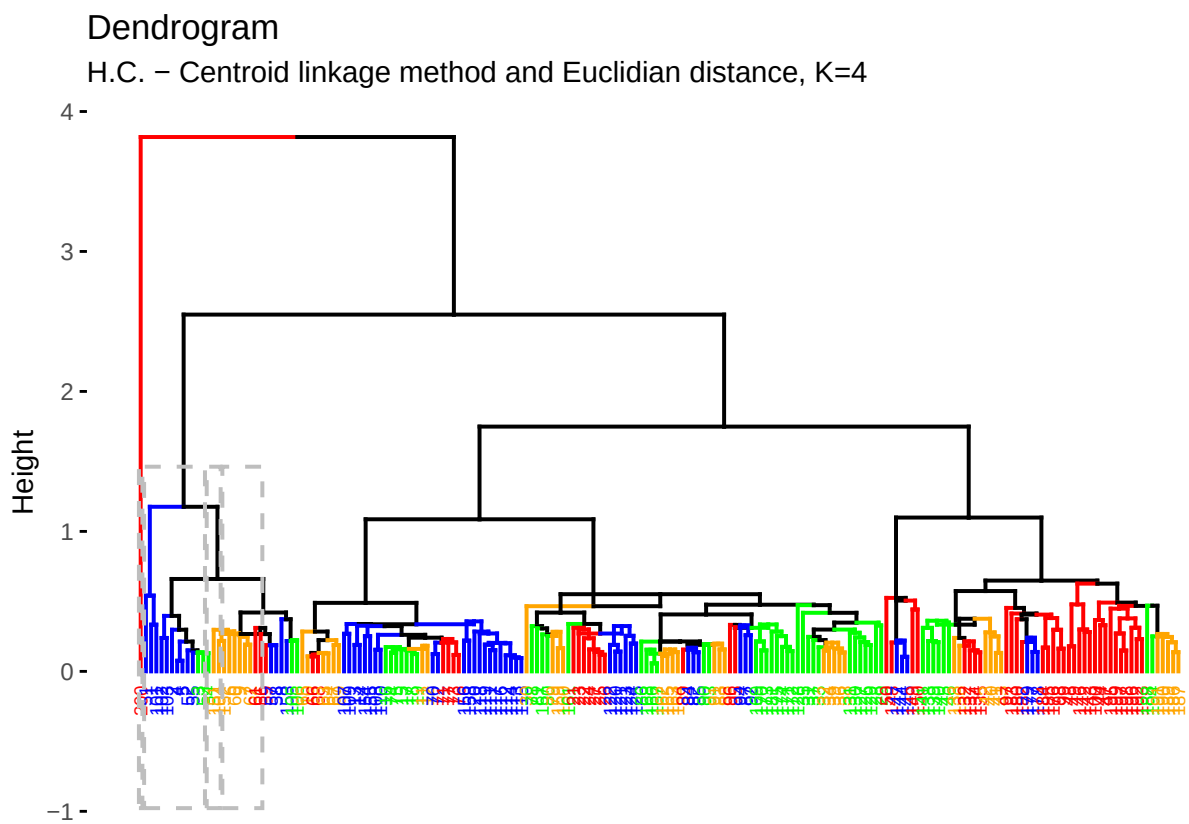

```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                 second differences plot) that corresponds to a significant increase of the value of
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 6 proposed 2 as the best number of clusters
## * 3 proposed 3 as the best number of clusters
## * 9 proposed 4 as the best number of clusters
## * 1 proposed 5 as the best number of clusters
## * 2 proposed 7 as the best number of clusters
## * 2 proposed 9 as the best number of clusters
## * 1 proposed 10 as the best number of clusters
##
##                    ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  4
##
##
## *******************************************************************
```

```r
dist.eucl <- dist(df_scaled_crabs, method = "euclidian")
hc <- hclust(dist.eucl, method = "centroid")
group <- cutree(hc, k=4)
table(group)
```

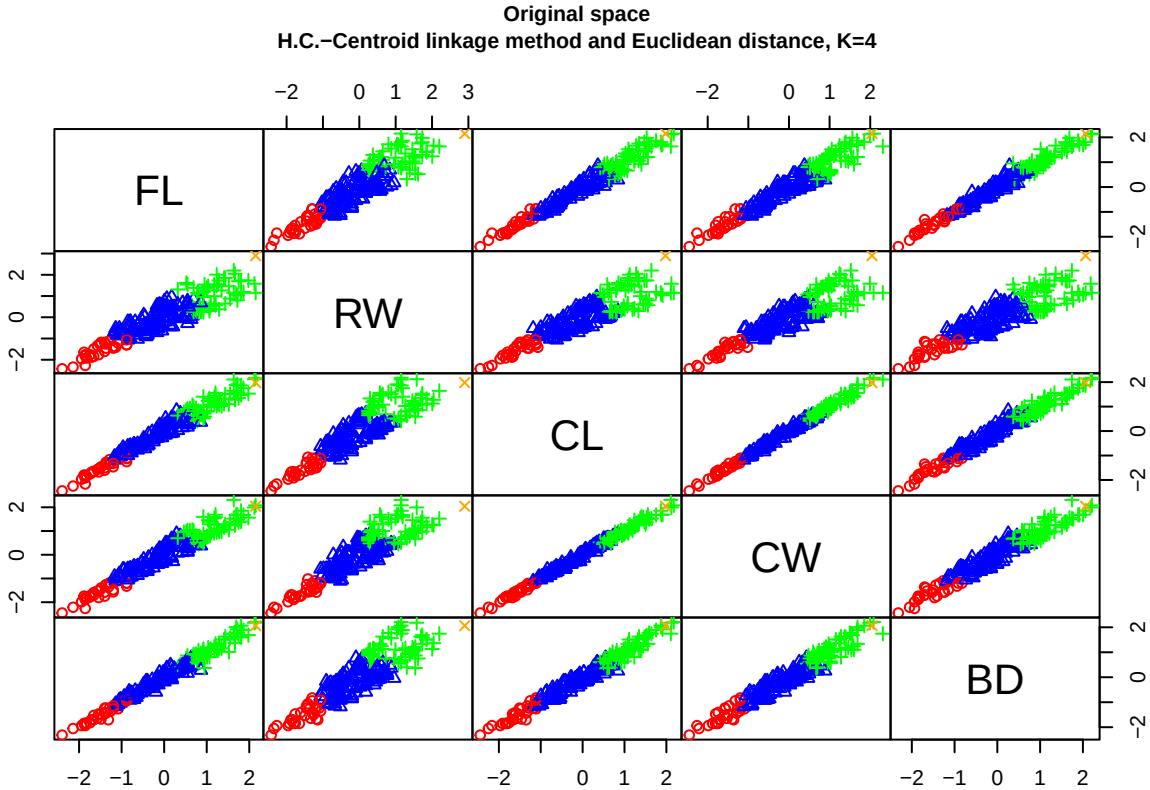
```
## group
##   1   2   3   4
##  30 112  57   1
```

```
fviz_dend(hc, k = 4, cex = 0.5, k_colors = c("red", "blue", "green", "orange"),
r_labels_by_k = TRUE, rect = TRUE)+labs(title = "Dendrogram",
subtitle = "H.C. - Centroid linkage method and Euclidian distance, K=4",cex.subtitle= 0.5)
```

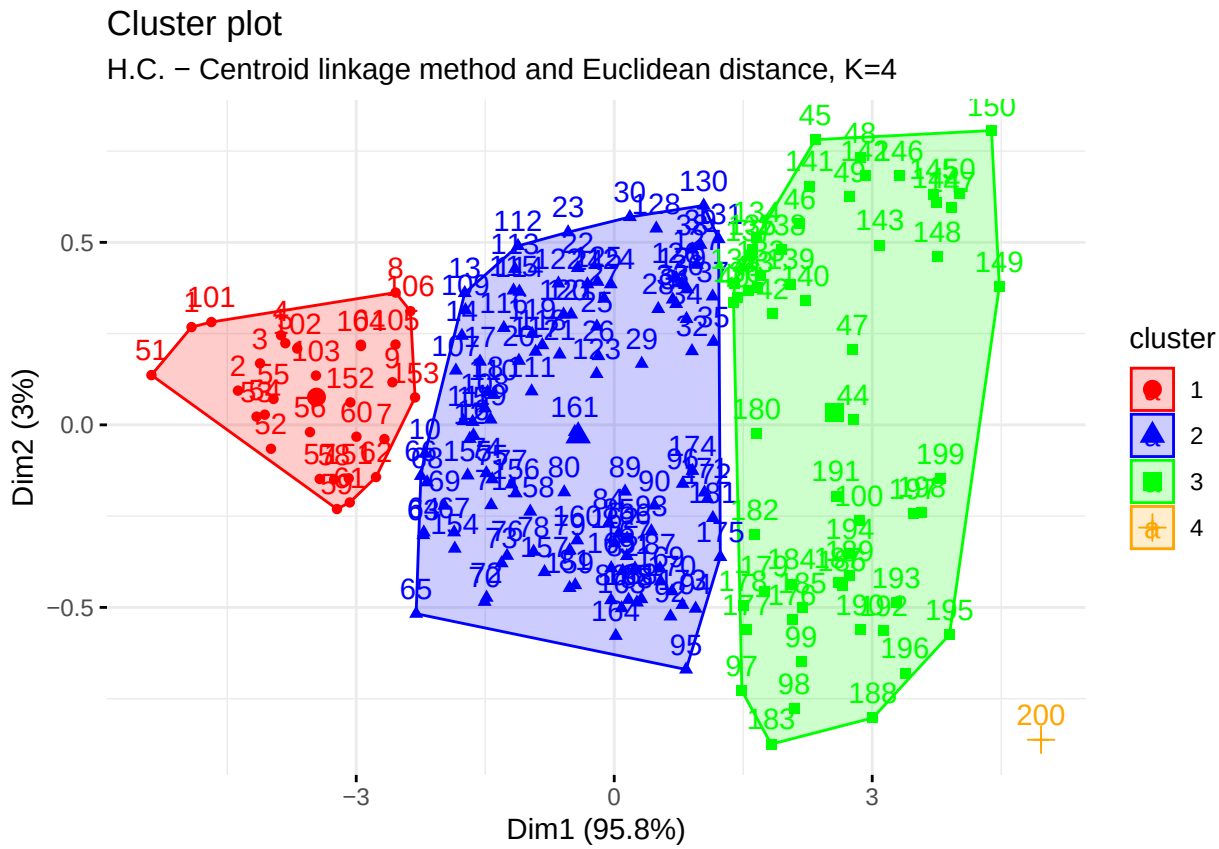
```
## Warning in get_col(col, k): Length of color vector was shorter than the number
## of clusters - color vector was recycled
```




```
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 0.7,
      main="Original space\nH.C.-Centroid linkage method and Euclidean distance, K=4" ,col=c("red", "blu
```




```
fviz_cluster(list(data = df_scaled_crabs, cluster = group), palette = c("red", "blue", "green", "orange
              show.clust.aver = FALSE, ggtheme = theme_minimal())+labs(
subtitle = "H.C. - Centroid linkage method and Euclidean distance, K=4", cex.sub= 0.5)
```



To assess the quality of clustering algorithm results, it is important to analyze both internal and external validation measures. These measures provide insights into different aspects of the clustering performance.

Internal validation measures:

1.Silhouette width:

```
hclust<- eclust(crabs_p,k=4 ,"hclust",hc_method = "centroid", nboot = 50, hc_metric = "euclidean")
```

```
## Warning in get_col(col, k): Length of color vector was shorter than the number
## of clusters - color vector was recycled
```

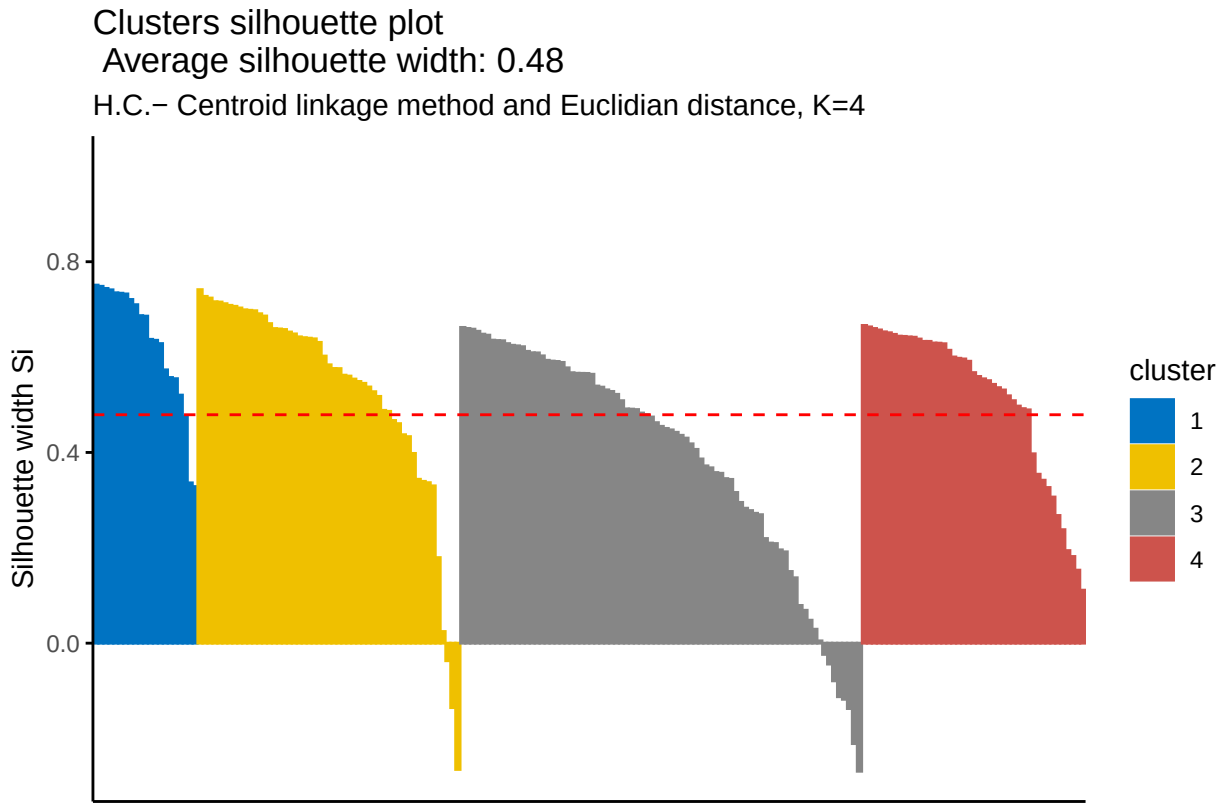
```
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.4790865
```

The average silhouette width is calculated to be 0.48.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+labs(
subtitle = "H.C.- Centroid linkage method and Euclidian distance, K=4",
cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   21          0.63
## 2       2   53          0.53
## 3       3   81          0.39
## 4       4   45          0.52
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.6302960 0.5291565 0.3870943 0.5151367
```

Upon examining the cluster widths, we observe that cluster 1 has the highest width, followed by cluster 2, cluster 4 and finally cluster 3. This is due to the number of units contained within each cluster. The presence of negative values in certain parts suggests that those units may exhibit some degree of overlap or similarity with neighboring clusters. Units with a negative silhouette value indicate that they are potentially assigned to the wrong cluster. To address this, it is necessary to identify their neighboring clusters, which may better reflect their characteristics or similarities.

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##     cluster neighbor   sil_width
## 62        2        1 -0.03678196
## 6         2        1 -0.13513048
```

```
## 60        2        1 -0.26481342
## 134       3        4 -0.02357823
## 40        3        4 -0.04387760
## 160       3        2 -0.07904873
## 161       3        2 -0.11229588
## 122       3        2 -0.11772316
## 120       3        2 -0.13715669
## 159       3        2 -0.21062920
## 118       3        2 -0.26864002
```

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.06277516
```

According to the Dunn index, the units are not clustered well enough because the value is near 0.

External validation measures:

1.Confusion matrix:

According to the Confusion matrix, the number of clusters is equal to 4.

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2  3  4
##   F 12 24 43 21
##   M  9 29 38 24
```

2.Corrected Rand Index:

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.003702583
```

According to the Correct Rand Index, there is no agreement between the numerical value and the cluster solution.

3.Meila's VI Index:

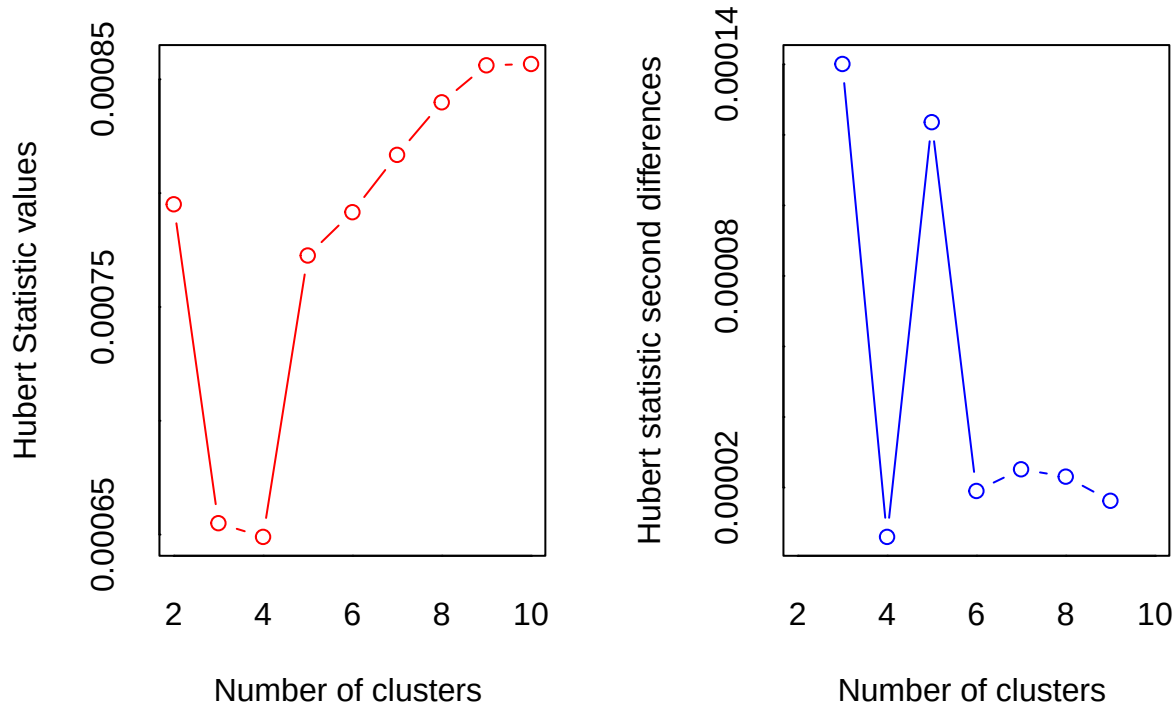
```
stats$vi
```

```
## [1] 1.976354
```

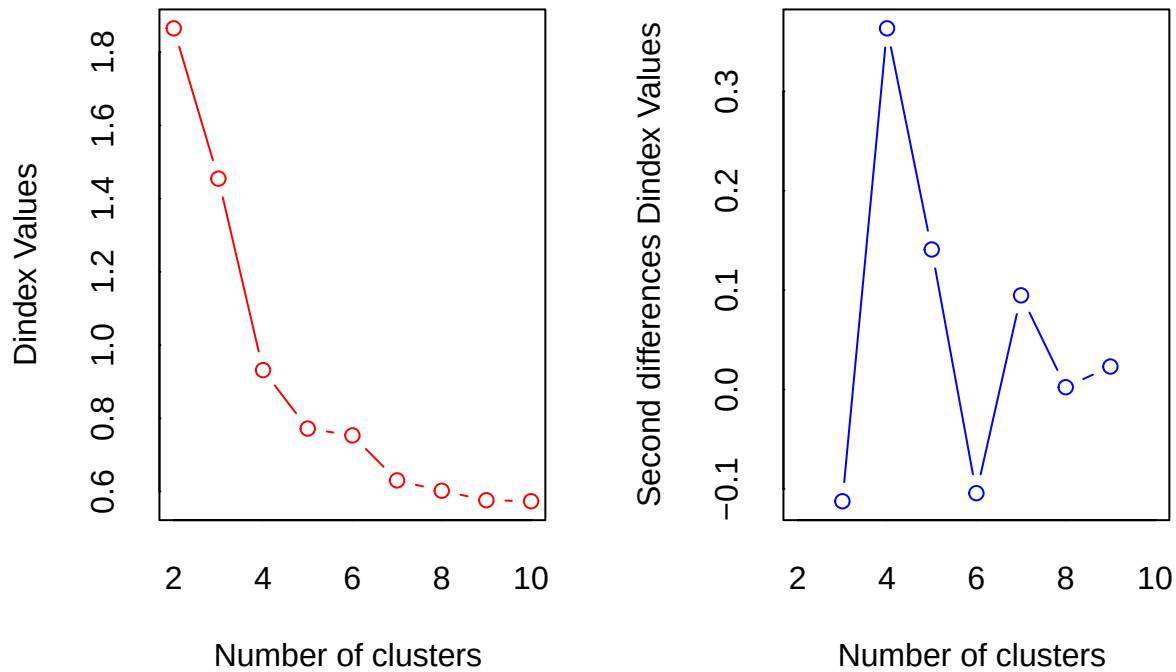
Disagreement between the numerical value and the cluster solution is 1.98.

Centroid linkage method and Manhattan distance:

```
nb <- NbClust(df_scaled_crabs, distance = "manhattan", min.nc = 2, max.nc = 10,
method = "centroid")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                 In the plot of Hubert index, we seek a significant knee that corresponds to a
##                 significant increase of the value of the measure i.e the significant peak in Hubert
##                 index second differences plot.
##
```



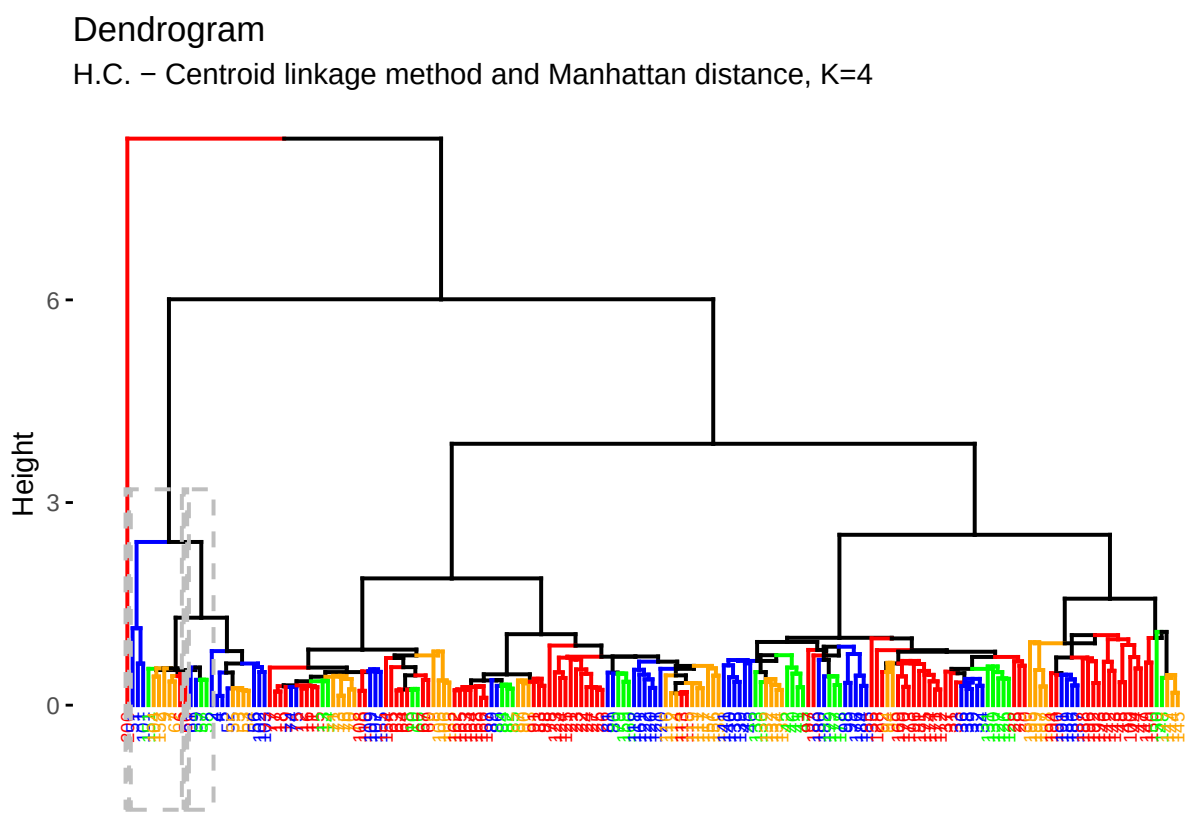

```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                 second differences plot) that corresponds to a significant increase of the value of
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 4 proposed 2 as the best number of clusters
## * 1 proposed 3 as the best number of clusters
## * 9 proposed 4 as the best number of clusters
## * 4 proposed 5 as the best number of clusters
## * 4 proposed 7 as the best number of clusters
## * 1 proposed 10 as the best number of clusters
##
##                    ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  4
##
##
## *******************************************************************
```

```r
dist.man <- dist(df_scaled_crabs, method = "manhattan")
hc <- hclust(dist.man, method = "centroid")
group <- cutree(hc, k=4)
table(group)
```

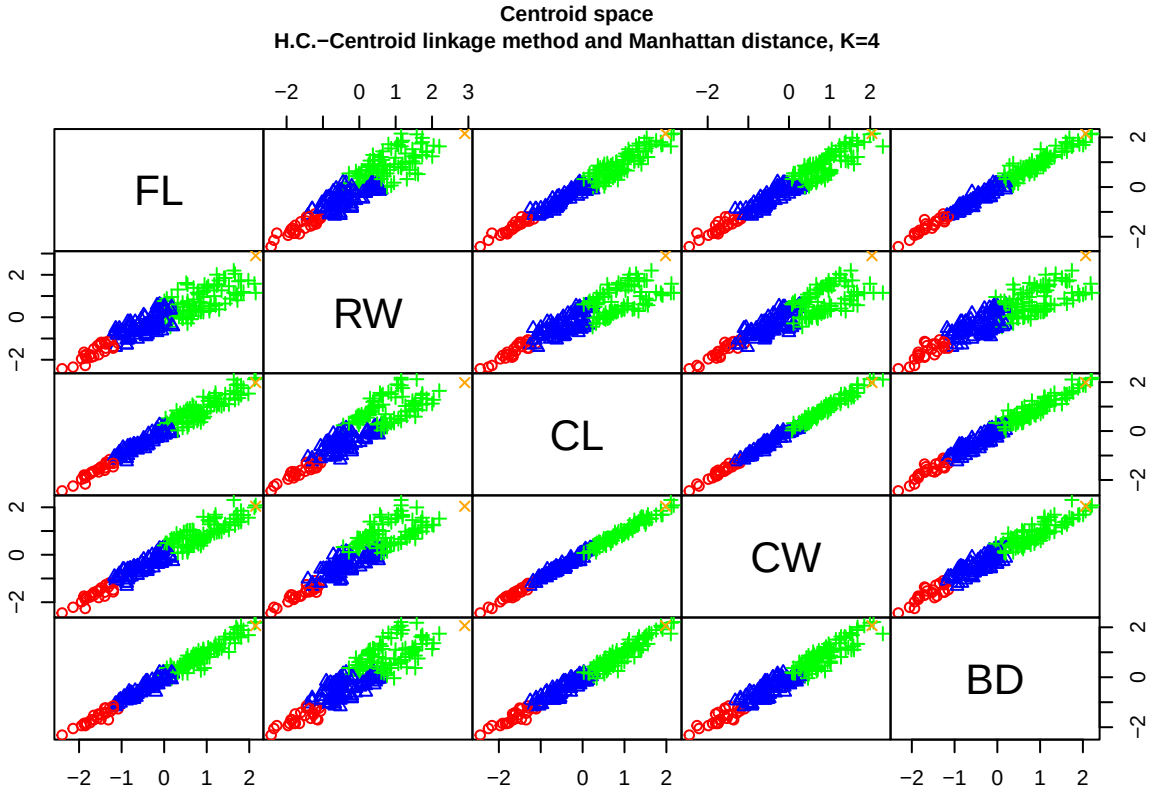
```
## group
##  1  2  3  4
## 26 86 87  1
```

```
fviz_dend(hc, k = 4, cex = 0.5, k_colors = c("red", "blue", "green", "orange"),
color_labels_by_k = TRUE, rect = TRUE)+labs(title = "Dendrogram",
subtitle = "H.C. - Centroid linkage method and Manhattan distance, K=4", cex.subtitle= 0.5)
```

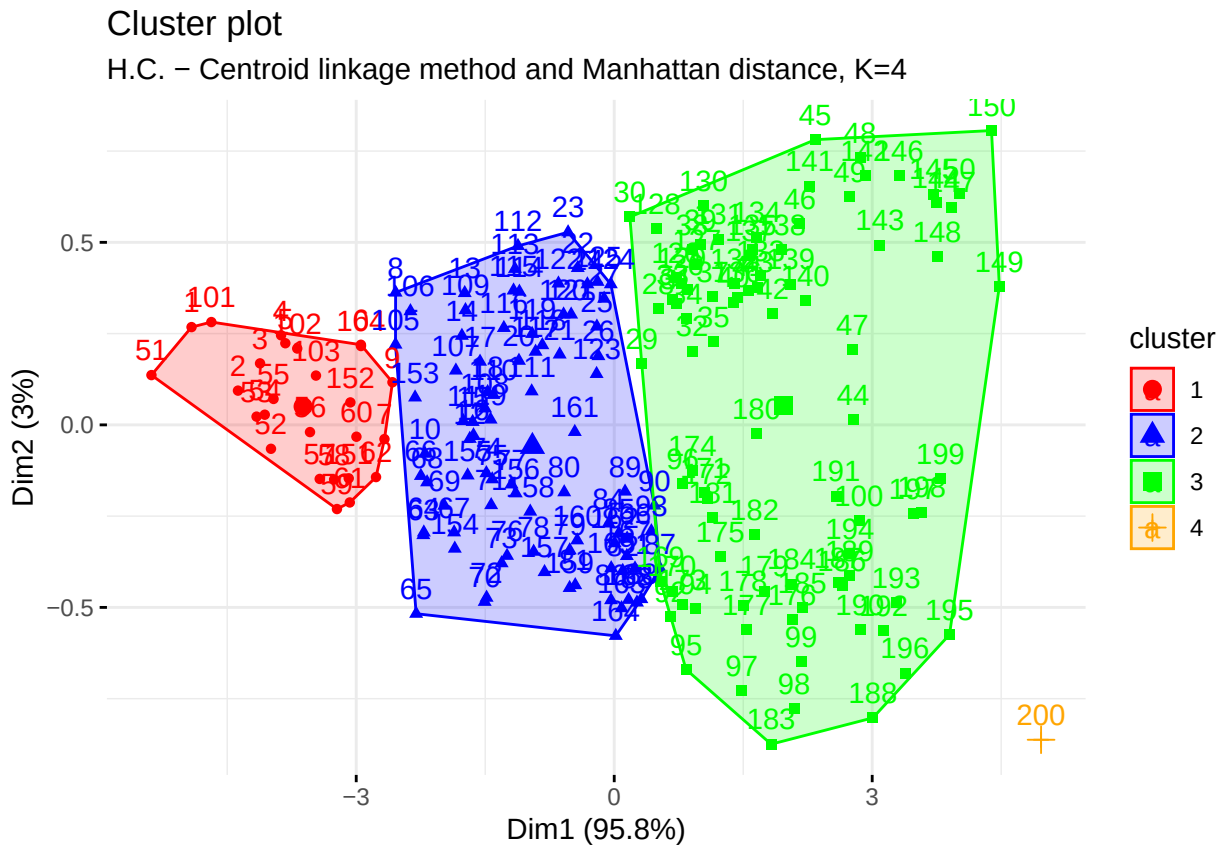
```
## Warning in get_col(col, k): Length of color vector was shorter than the number
## of clusters - color vector was recycled
```




```
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 0.7,
      main="Centroid space\nH.C.-Centroid linkage method and Manhattan distance, K=4" ,col=c("red", "blu
```




```
fviz_cluster(list(data = df_scaled_crabs, cluster = group), palette = c("red", "blue", "green", "orange
              show.clust.aver = TRUE, ggtheme = theme_minimal())+labs(
subtitle = "H.C. - Centroid linkage method and Manhattan distance, K=4", cex.sub= 0.5)
```




```
hclust<- eclust(crabs_p,k=4 ,"hclust",hc_method = "centroid", nboot = 50, hc_metric = "manhattan")
```

```
## Warning in get_col(col, k): Length of color vector was shorter than the number
## of clusters - color vector was recycled
```

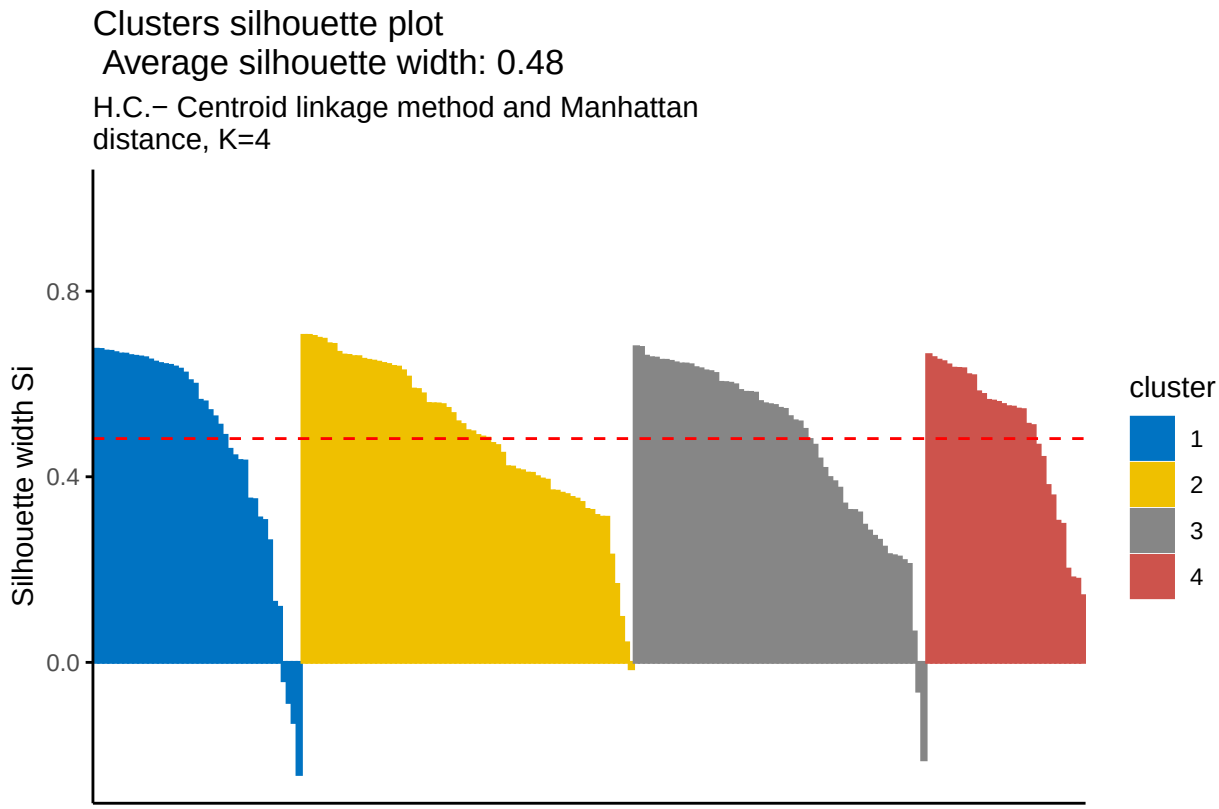
```
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.4823068
```

The average silhouette width is calculated to be 0.48.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+labs(
subtitle = "H.C.- Centroid linkage method and Manhattan
distance, K=4", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   42          0.48
## 2       2   67          0.49
## 3       3   59          0.47
## 4       4   32          0.50
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.4750157 0.4902642 0.4692859 0.4992232
```

Upon examining the cluster widths, we observe that cluster 4 has the highest width, followed by cluster 2, cluster 1 and finally cluster 3. This is due to the number of units contained within each cluster. The presence of negative values in certain parts suggests that those units may exhibit some degree of overlap or similarity with neighboring clusters. Units with a negative silhouette value indicate that they are potentially assigned to the wrong cluster. To address this, it is necessary to identify their neighboring clusters, which may better reflect their characteristics or similarities.

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##     cluster neighbor   sil_width
## 107       1        2 -0.03952450
## 67        1        2 -0.08633766
## 155       1        2 -0.12949777
## 109       1        2 -0.24154559
## 88        2        3 -0.01373733
## 140       3        4 -0.06207549
## 141       3        4 -0.20996483
```

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.06339909
```

External validation measures:

1.Confusion matrix:

According to the Confusion matrix, the number of clusters is equal to nominal values. The clusters found are 4.

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2  3  4
##   F 24 33 27 16
##   M 18 34 32 16
```

2.Correct Rand Index:

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.004278673
```

Meila's VI Index:

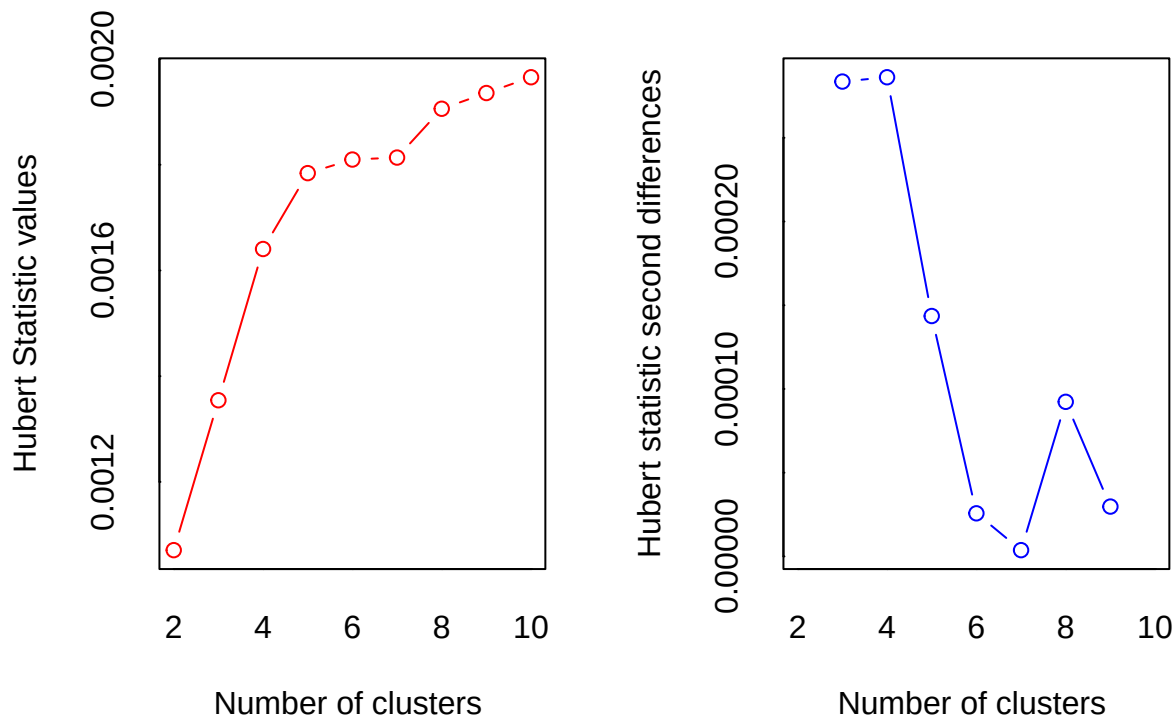
```
stats$vi
```

```
## [1] 2.034094
```

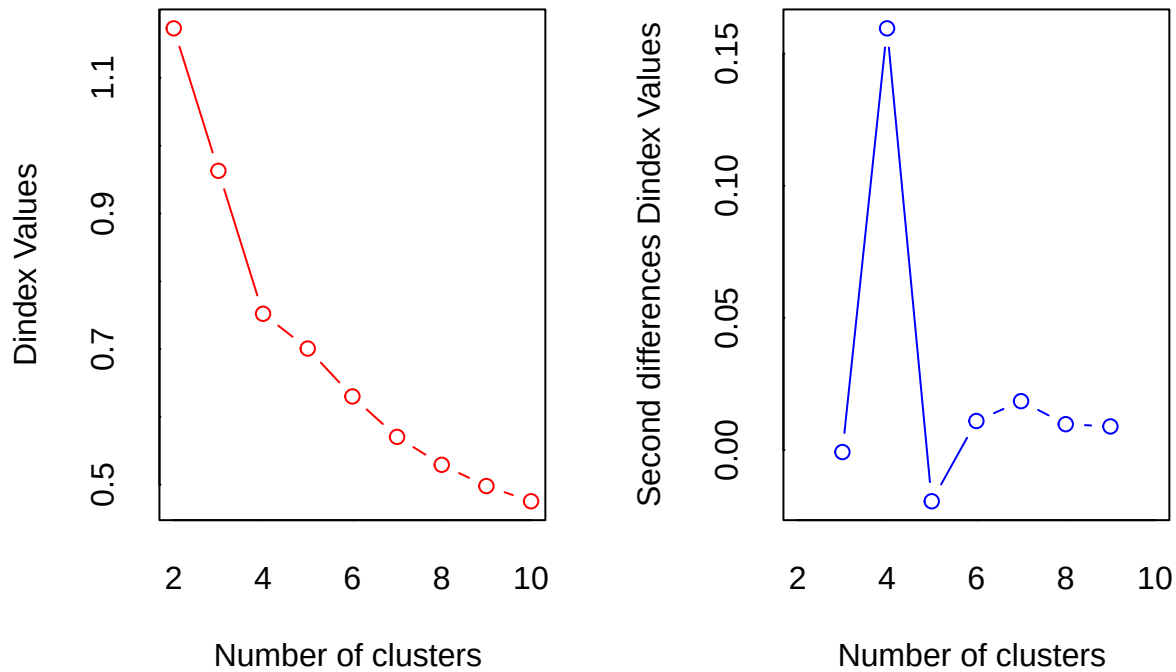
Disagreement between the numerical value and the cluster solution is 1.98.

Ward's (minimum deviance) method:

```r
nb <- NbClust(df_scaled_crabs, distance = "euclidean", min.nc = 2, max.nc = 10,
method = "ward.D2")
```




```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                 In the plot of Hubert index, we seek a significant knee that corresponds to a
##                 significant increase of the value of the measure i.e the significant peak in Hubert
##                 index second differences plot.
##
```




```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                 second differences plot) that corresponds to a significant increase of the value of
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 7 proposed 2 as the best number of clusters
## * 3 proposed 3 as the best number of clusters
## * 7 proposed 4 as the best number of clusters
## * 2 proposed 7 as the best number of clusters
## * 2 proposed 9 as the best number of clusters
## * 3 proposed 10 as the best number of clusters
##
##                    ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  2
##
##
## *******************************************************************
```

```r
hc <- hclust(dist.eucl, method = "ward.D2")
group <- cutree(hc, k=2)
table(group)
```

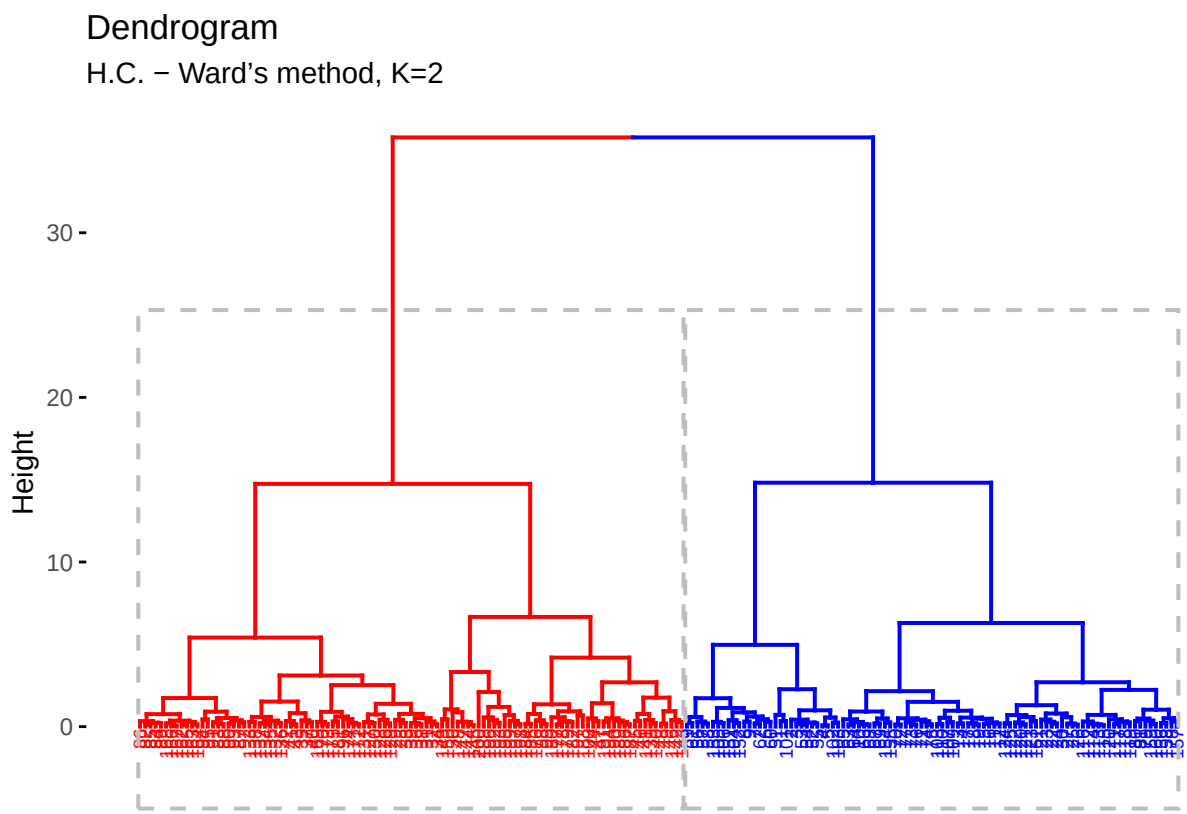
```
## group
```

```
##   1   2
##  95 105
```

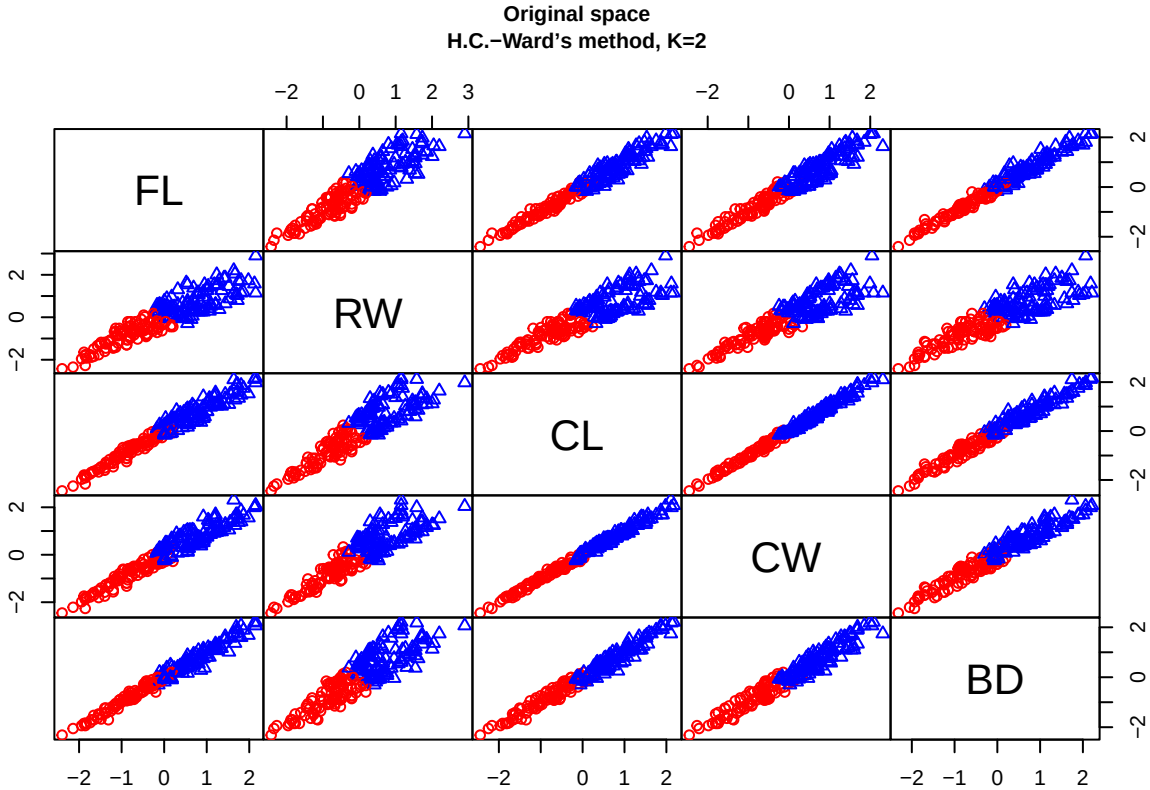
```
group
```

```
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   1   1   1   1   1   1   1   2   2   1   2   2   2   2   2   2   2   2   2   2
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   1   1   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
## 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
##   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
```

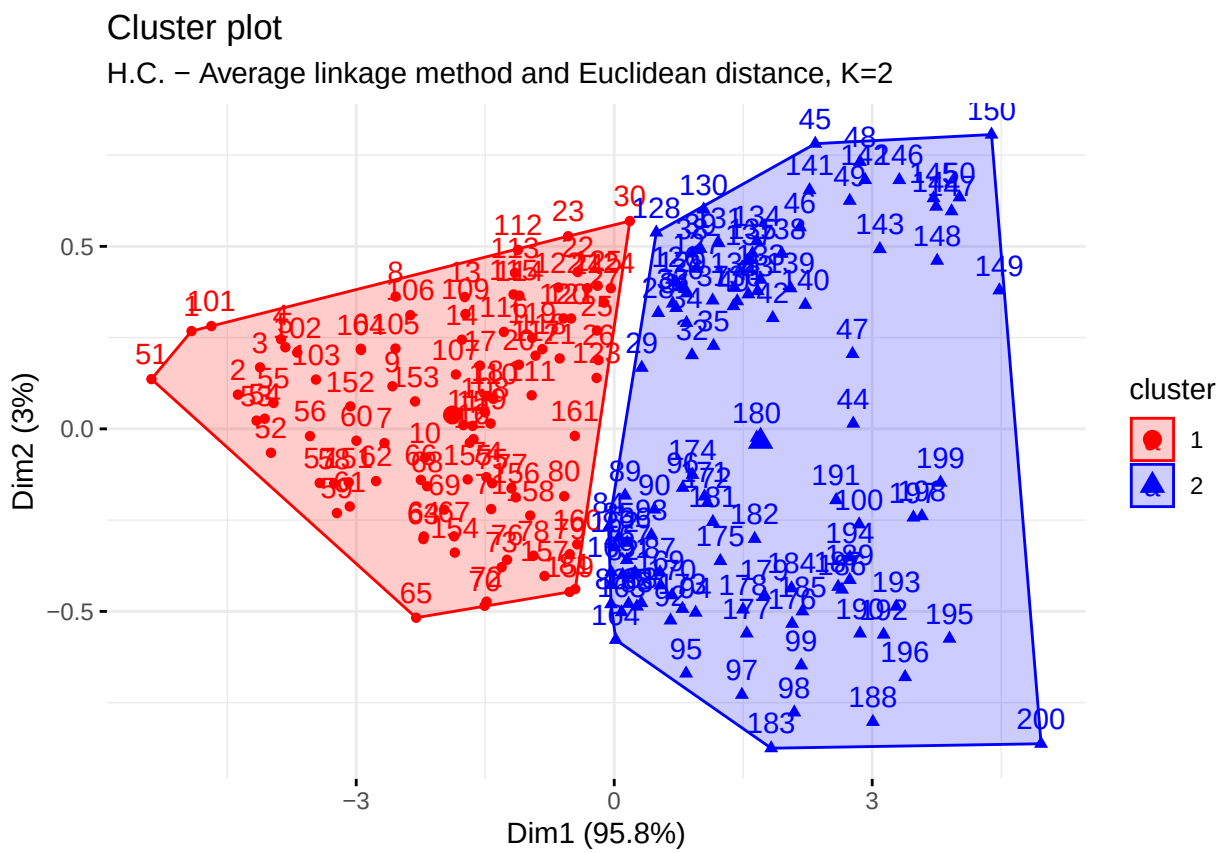
```
fviz_dend(hc, k = 2, cex = 0.5, k_colors = c("red", "blue"),
color_labels_by_k = TRUE, rect = TRUE)+labs(title = "Dendrogram",
subtitle = "H.C. - Ward's method, K=2", cex.subtitle= 0.5)
```




```
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 0.7,
      main="Original space\nH.C.-Ward's method, K=2" ,col=c("red", "blue")[group])
```




```
fviz_cluster(list(data = df_scaled_crabs, cluster = group),
             palette = c("red", "blue"),
             show.clust.aver = FALSE, ggtheme = theme_minimal())+
labs(subtitle = "H.C. - Average linkage method and Euclidean distance, K=2", cex.sub= 0.5)
```



Internal validation measures:

Silhouette width:

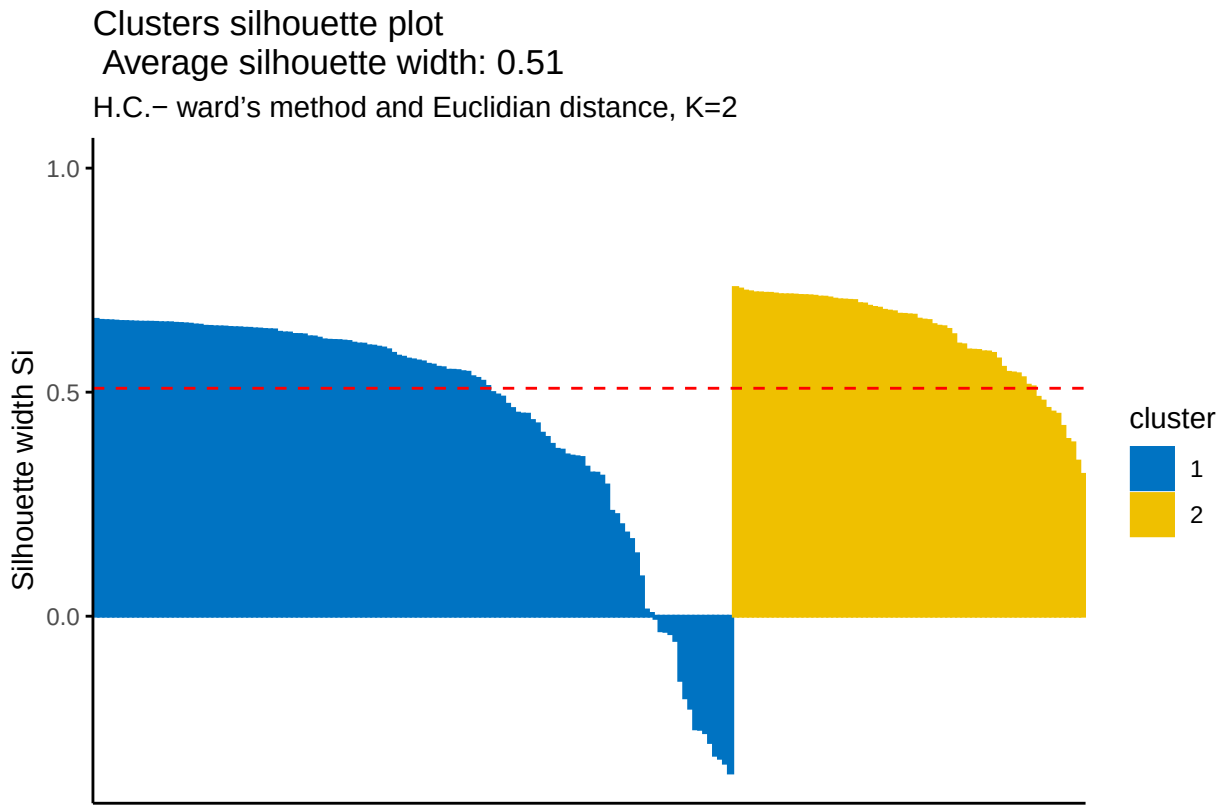
```
hclust<- eclust(crabs_p,k=2 ,"hclust",hc_method = "ward.D2",nboot = 50)
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.5086809
```

The average silhouette width is calculated to be 0.51.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+labs(
subtitle = "H.C.- ward's method and Euclidian distance, K=2", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1  129          0.44
## 2       2   71          0.63
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.4430005 0.6280156
```

Upon examining the cluster widths, we observe that cluster 2 has the higher width than cluster 2.

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##     cluster neighbor   sil_width
## 90        1        2 -0.00485337
## 128       1        2 -0.03248965
## 87        1        2 -0.03377069
## 30        1        2 -0.03893387
## 29        1        2 -0.05406875
## 92        1        2 -0.14313886
## 174       1        2 -0.18213118
## 129       1        2 -0.20530795
## 96        1        2 -0.25137890
## 126       1        2 -0.25241965
## 95        1        2 -0.25989880
## 172       1        2 -0.28182945
## 94        1        2 -0.31034550
```

```
## 171       1        2 -0.31701756
## 181       1        2 -0.32802525
## 175       1        2 -0.34996165
```

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.03823786
```

According to the Dunn index, the units are not clustered well enough.

External validation measures:

Confusion matrix:

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2
##   F 72 28
##   M 57 43
```

Correct Rand Index:

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] 0.01798024
```

According to the Correct Rand Index, there is no agreement between the numerical value and the cluster solution.

Meila's VI Index:

```
stats$vi
```

```
## [1] 1.318928
```

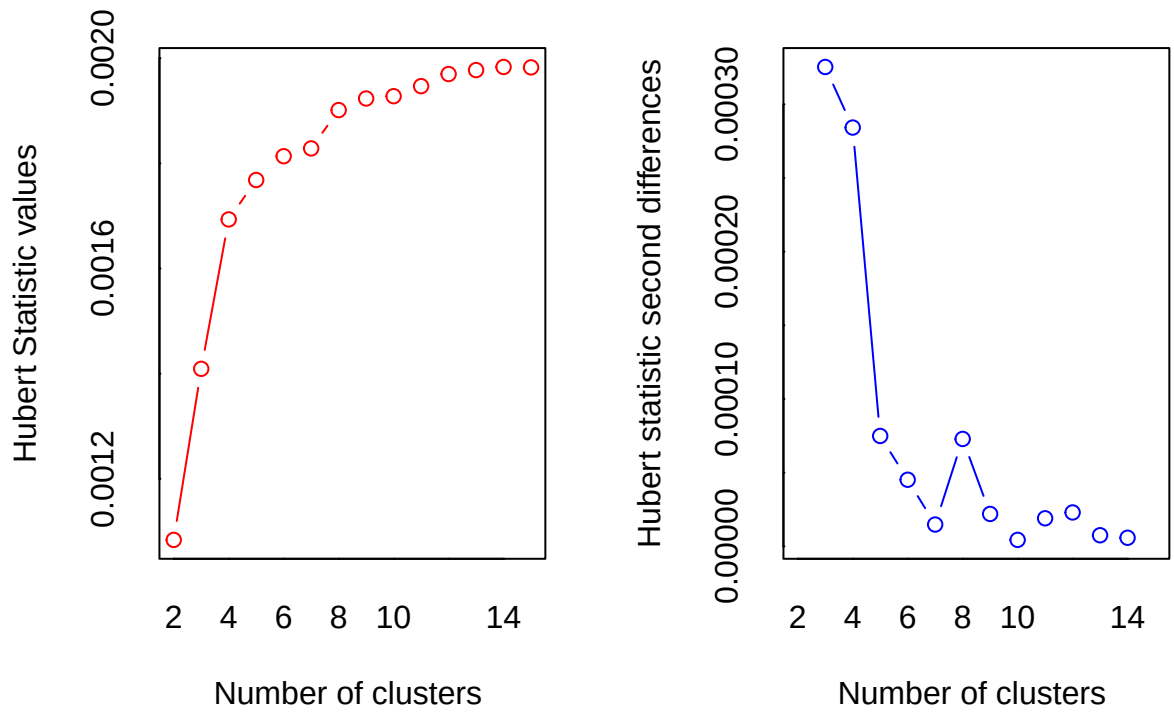
Disagreement between the numerical value and the cluster solution is 1.32.

Partitional method:

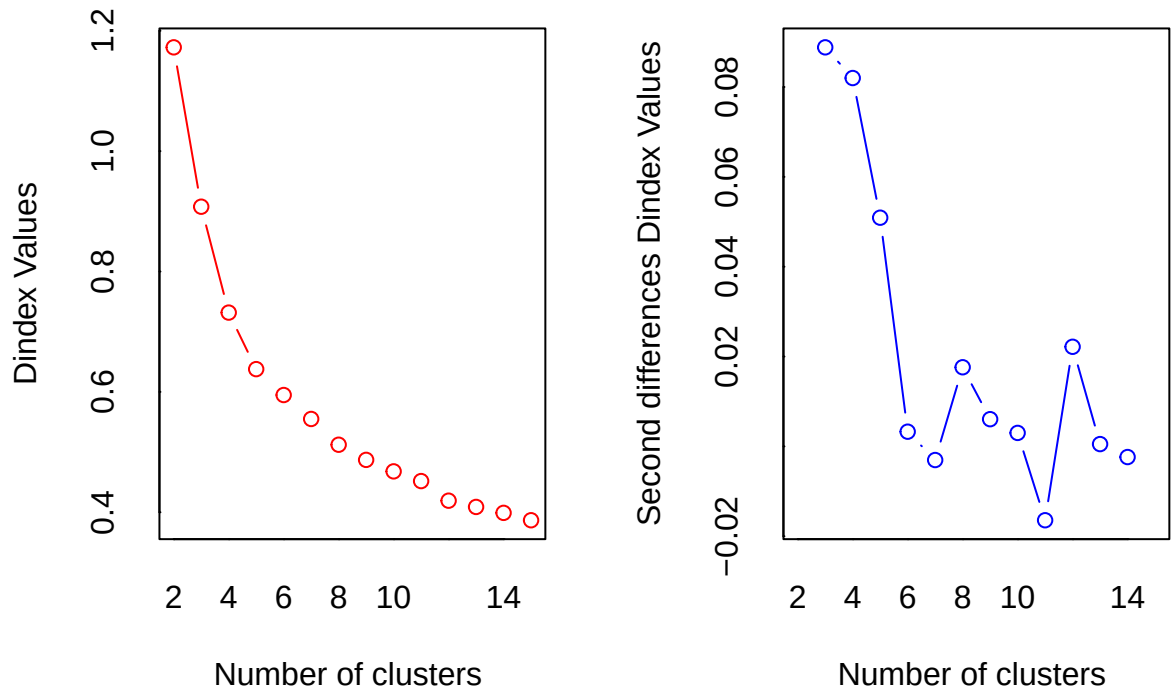
K-means:

In K-means method, each cluster is represented by the mean of the data points belonging to the cluster.

```
nb <- NbClust(df_scaled_crabs, min.nc=2, max.nc=15, method="kmeans")
```

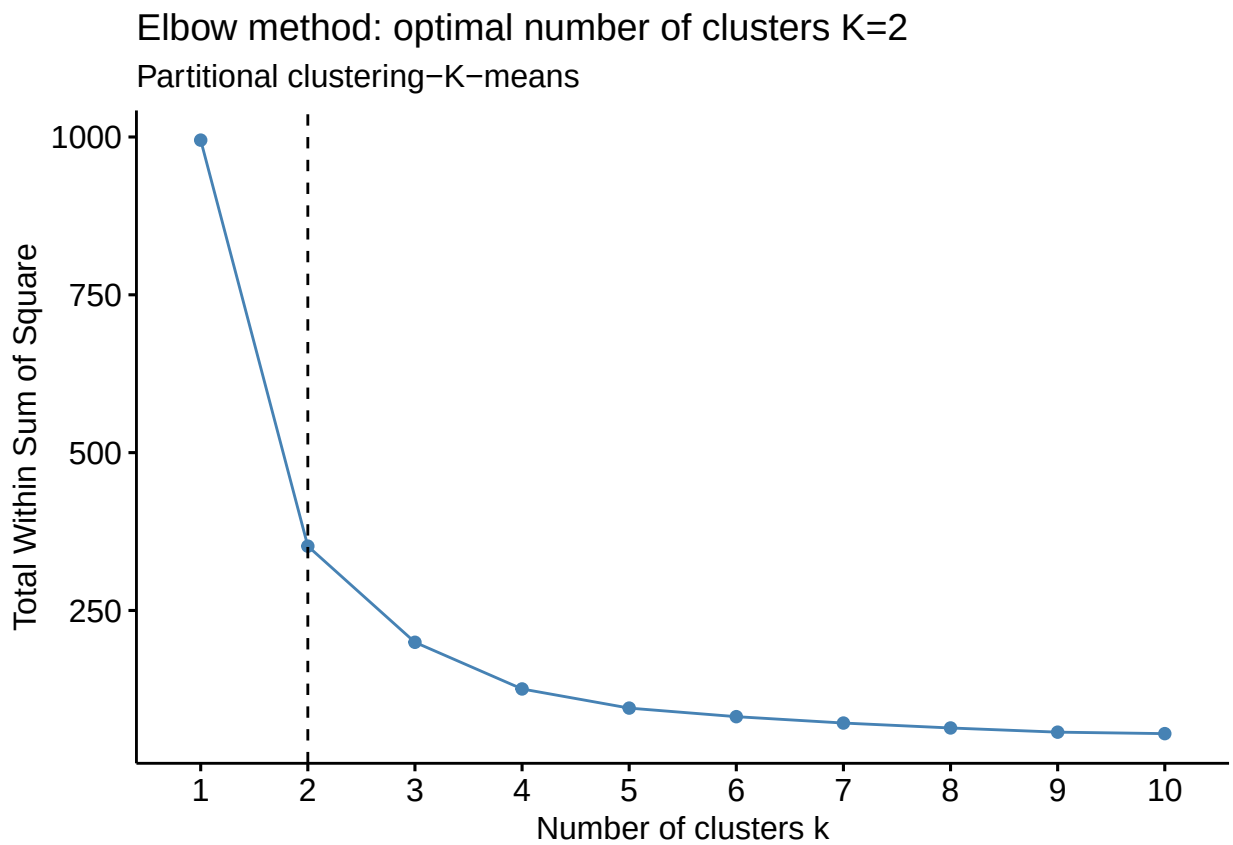



```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##                In the plot of Hubert index, we seek a significant knee that corresponds to a
##                significant increase of the value of the measure i.e the significant peak in Hubert
##                index second differences plot.
##
```




```
## *** : The D index is a graphical method of determining the number of clusters.
##                 In the plot of D index, we seek a significant knee (the significant peak in Dindex
##                 second differences plot) that corresponds to a significant increase of the value of
##                 the measure.
##
## *******************************************************************
## * Among all indices:
## * 8 proposed 2 as the best number of clusters
## * 4 proposed 3 as the best number of clusters
## * 3 proposed 4 as the best number of clusters
## * 1 proposed 5 as the best number of clusters
## * 1 proposed 6 as the best number of clusters
## * 1 proposed 7 as the best number of clusters
## * 2 proposed 12 as the best number of clusters
## * 1 proposed 13 as the best number of clusters
## * 2 proposed 15 as the best number of clusters
##
##                    ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  2
##
##
## *******************************************************************
```

```r
fviz_nbclust(df_scaled_crabs, kmeans, method = "wss") +
geom_vline(xintercept = 2, linetype = 2)+
labs(title= "Elbow method: optimal number of clusters K=2", subtitle = "Partitional clustering-K-means")
```




```r
set.seed(123)
(km.res<- kmeans(df_scaled_crabs, 2, nstart = 25))
```

```
## K-means clustering with 2 clusters of sizes 92, 108
##
## Cluster means:
##           FL         RW         CL         CW         BD
## 1 -0.8708402 -0.8538303 -0.8767221 -0.8819614 -0.8607554
## 2  0.7418268  0.7273370  0.7468373  0.7513005  0.7332361
##
## Clustering vector:
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   1   1   1   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
```

```
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   1   1   1   2   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
## 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
##   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
##
## Within cluster sum of squares by cluster:
## [1] 152.4655 199.4228
##  (between_SS / total_SS =  64.6 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

It's possible to compute the mean of each variable by cluster, using the original data, via the aggregate() function:

```r
aggregate(crabs_p, by=list(cluster=km.res$cluster), mean)
```

```
##   cluster       FL       RW       CL       CW       BD
## 1       1 12.53913 10.54130 25.86413 29.47174 11.08261
## 2       2 18.17593 14.61019 37.42222 42.32870 16.54167
```

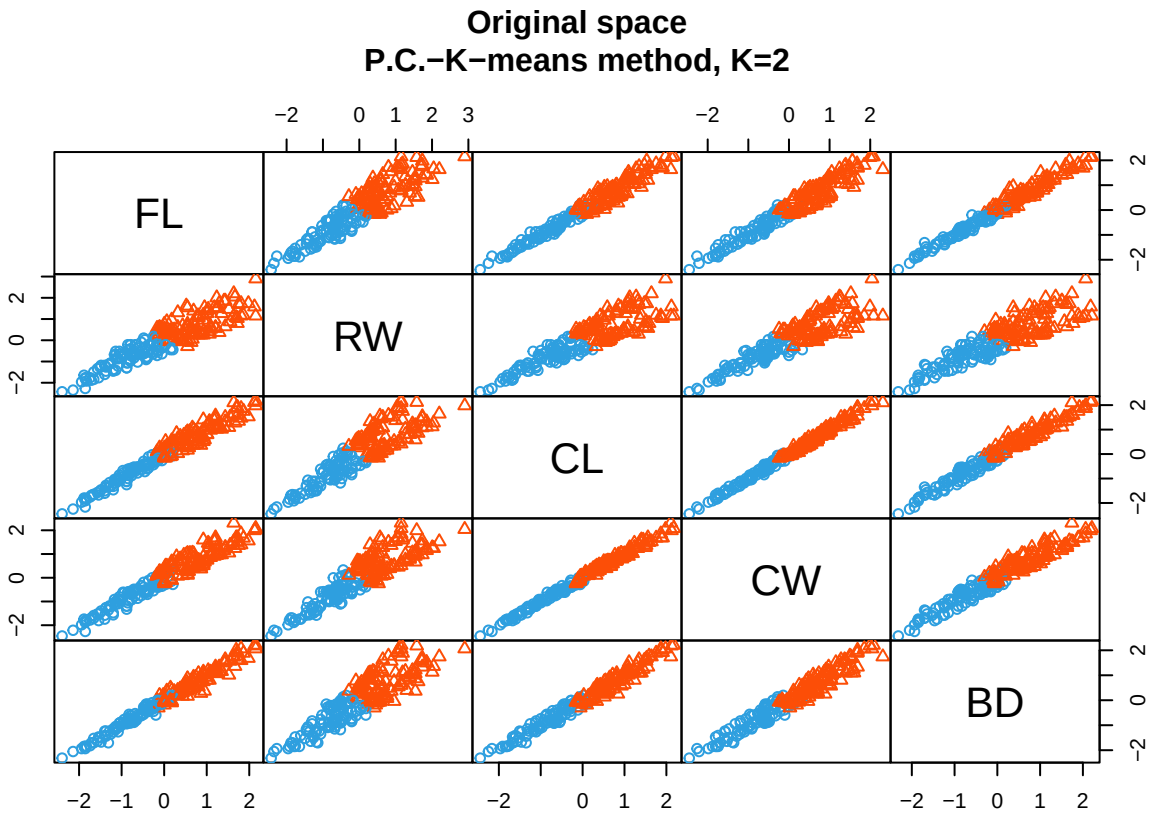
This matrix is useful to characterize the clusters.As an example, the first cluster likely contains units with small values for all the variables.

If we want to add the cluster memberships to the original data, use this:

```r
dd <- cbind(df_scaled_crabs, cluster = km.res$cluster)
head(dd)
```

```
##          FL        RW        CL        CW        BD cluster
## 1 -2.140860 -2.346561 -2.248285 -2.212220 -2.052837       1
## 2 -1.940592 -1.957961 -1.967346 -1.983561 -1.936041       1
## 3 -1.826153 -1.919101 -1.840923 -1.780307 -1.848444       1
## 4 -1.711715 -1.880241 -1.686407 -1.691384 -1.702449       1
## 5 -1.654496 -1.841381 -1.658313 -1.704087 -1.702449       1
## 6 -1.368399 -1.452781 -1.279045 -1.259471 -1.235265       1
```

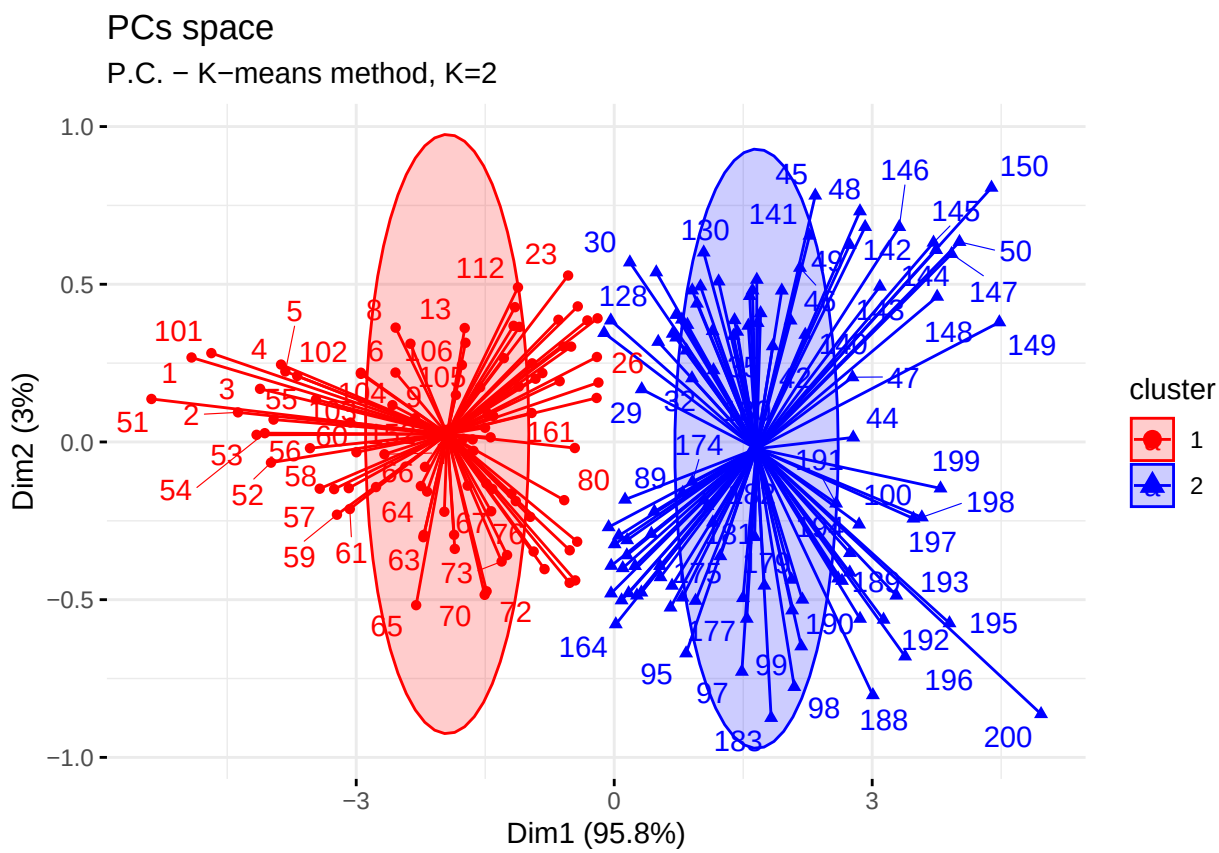
```r
pairs(df_scaled_crabs, gap=0, pch=group, cex.main= 1,
      main="Original space\nP.C.-K-means method, K=2" ,col=c("#2E9FDF", "#FC4E07")[group])
```



Visualizing cluster results can be useful to assess the choice of the number of clusters as well as to compare two different cluster analyses.

```
fviz_cluster(km.res, data = df_scaled_crabs, palette = c("red", "blue"),
ellipse.type = "euclid", star.plot = TRUE, repel = TRUE,
main= "PCs space", ggtheme = theme_minimal())+
labs(subtitle = "P.C. - K-means method, K=2")
```

```
## Warning: ggrepel: 105 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



Internal validation measures:

1.Silhouette width:

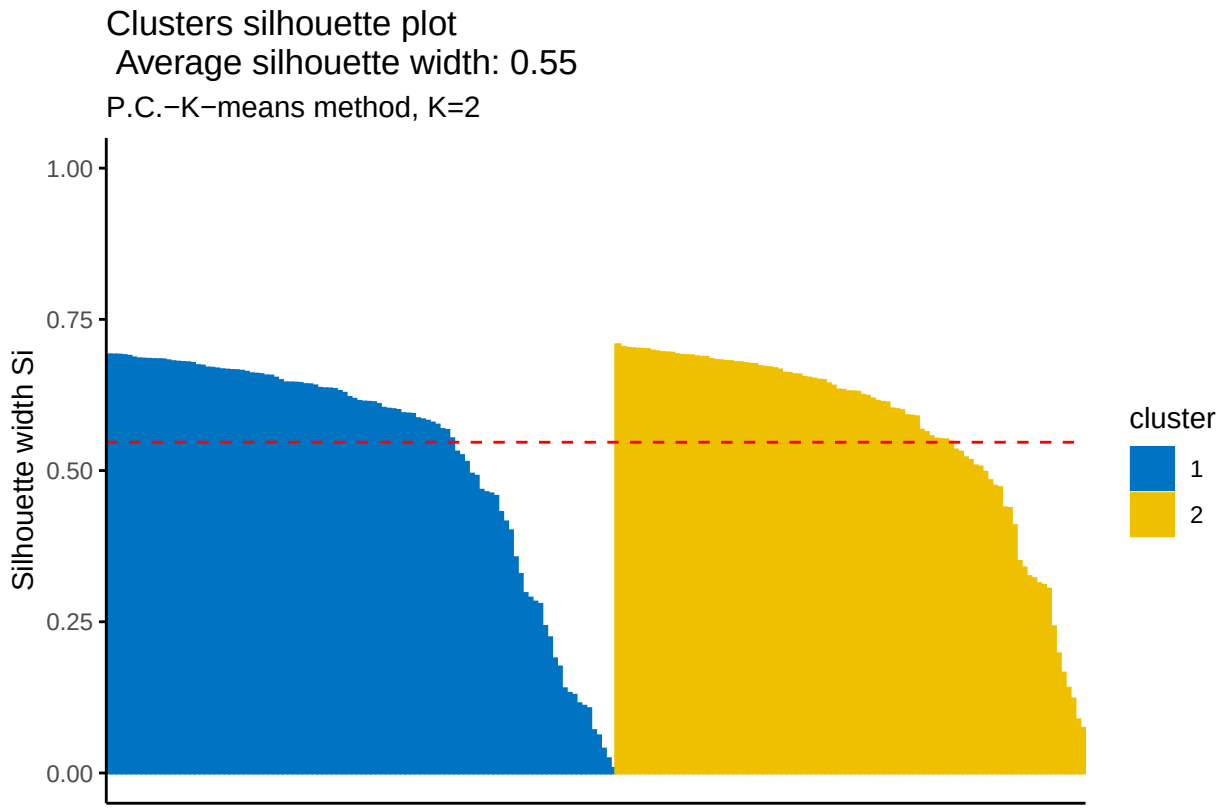
```
hclust<- eclust(crabs_p,k=2 ,"kmeans", nstart=25, graph = FALSE)
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.546818
```

The Silhouette width value is 0.55.

```
fviz_silhouette(hclust, palette = "jco", ggtheme = theme_classic())+
labs(subtitle = "P.C.-K-means method, K=2")
```

```
##   cluster size ave.sil.width
## 1       1  104          0.53
## 2       2   96          0.57
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.5277193 0.5675083
```

Upon examining the cluster widths, we observe that cluster 2 has higher width than cluster 1. And also we can see that there is no Unit with a negative silhouette value indicate that they are assigned to the right cluster.

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.02593545
```

According to the Dunn index, the units are not clustered well enough.

External validation measures:

Confusion matrix:

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2
##   F 53 47
##   M 51 49
```

2.Correct Rand Index:

```
Status <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.004640367
```

According to the Correct Rand Index, there is no agreement between the numerical value and the cluster solution.

Meila's VI Index:

```
stats$vi
```

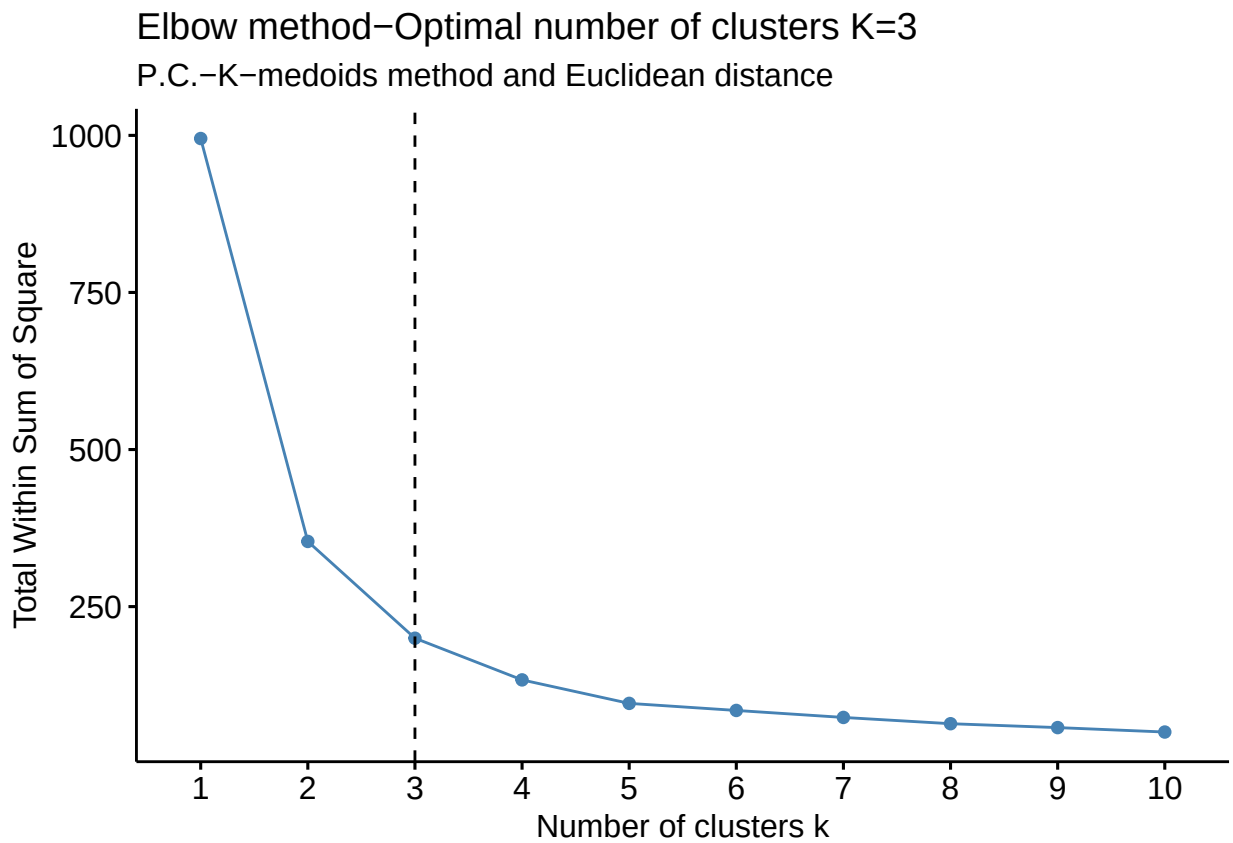
```
## [1] 1.385093
```

Disagreement between the numerical value and the cluster solution is 1.38.

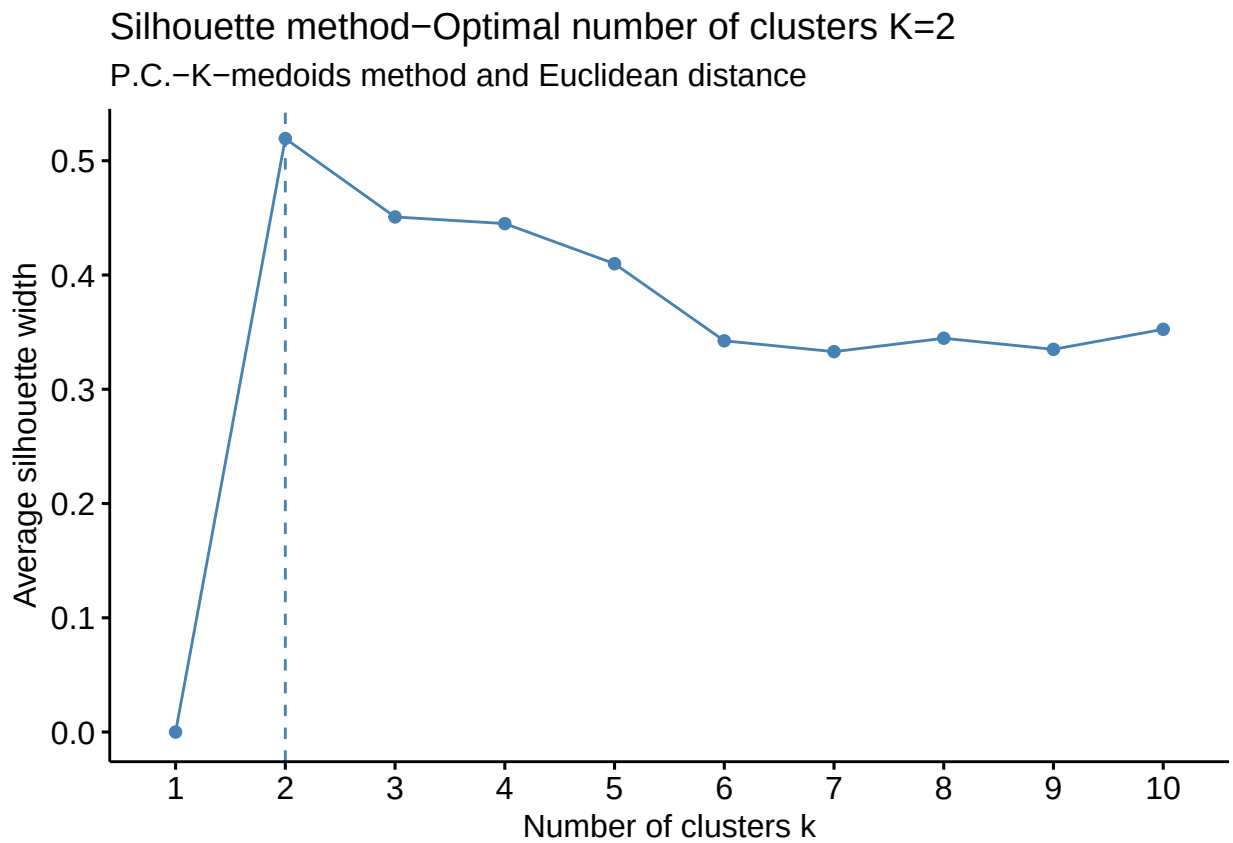
Partitioning around medoids (PAM) method and Euclidean distance:

K-medoids, also known as partitioning around medoids (PAM), where each cluster is represented by one of the points in the cluster.

```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "wss") +
geom_vline(xintercept = 3, linetype = 2)+
labs(title = "Elbow method-Optimal number of clusters K=3",
subtitle="P.C.-K-medoids method and Euclidean distance", cex.sub= 0.5)
```

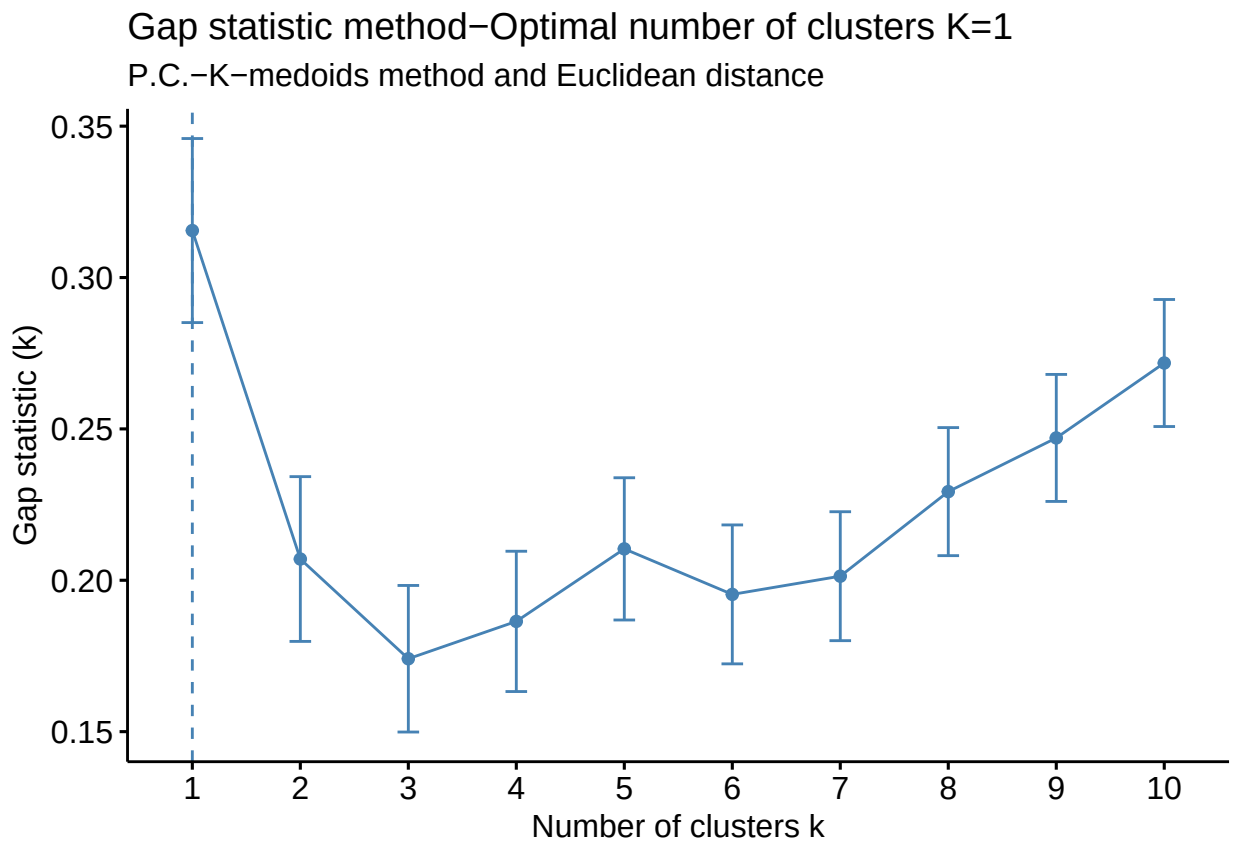



```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "silhouette")+
labs(title = "Silhouette method-Optimal number of clusters K=2",
subtitle="P.C.-K-medoids method and Euclidean distance", cex.sub= 0.5)
```



The maximum in the plot indicates the appropriate number of clusters. From the plot, the suggested number of clusters is K = 2.

```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "gap_stat", nboot = 500)+
labs(title = "Gap statistic method-Optimal number of clusters K=1",
subtitle="P.C.-K-medoids method and Euclidean distance", cex.sub= 0.5)
```



To determine the optimal number of clusters, three indices were employed. However, the results from these indices did not provide a unanimous decision. The elbow method did not yield a clear outcome, while the Total Within Sum of Square indicated 2 clusters as the suggested number. The Silhouette method also suggested 2 clusters. On the other hand, the Gap statistics recommended 1 cluster, indicating a lack of clear clustering structure in the data. Considering these results, it was decided to proceed with identifying 2 clusters.

```
(pam.res <- pam(df_scaled_crabs, 2, metric = "euclidean"))
```

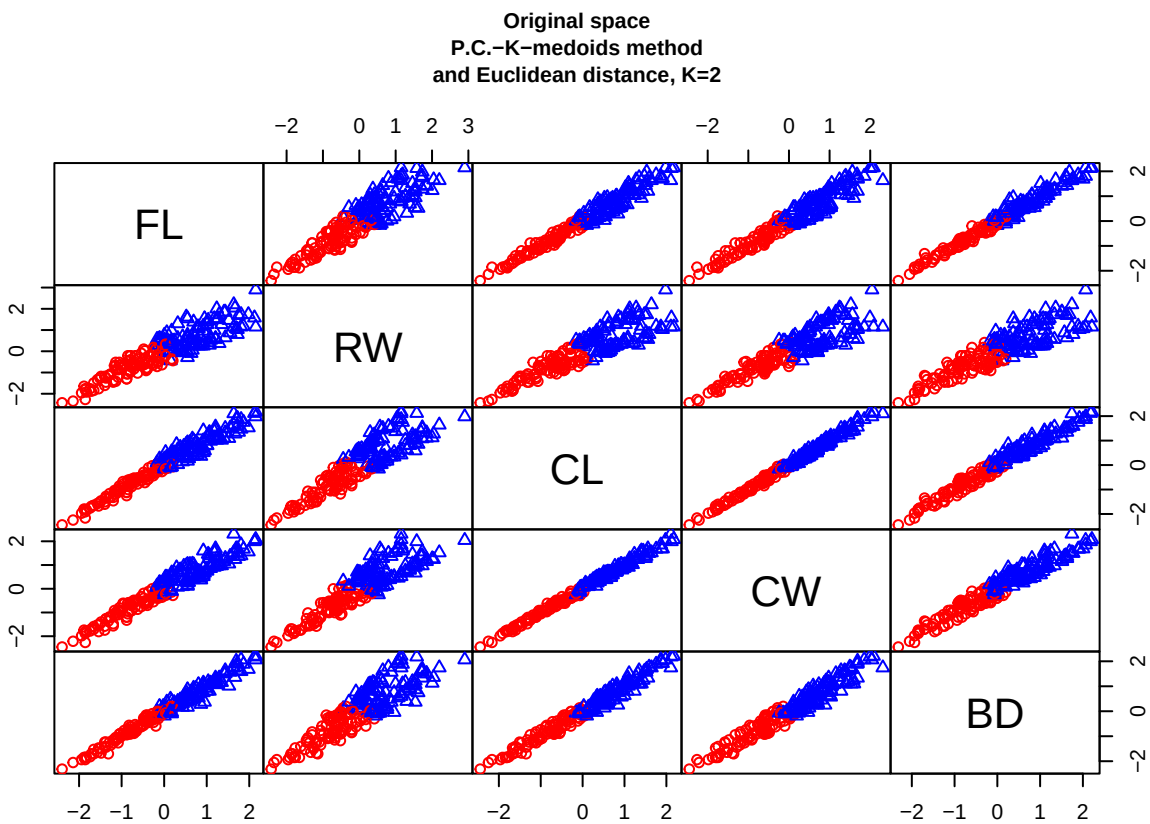
```
## Medoids:
##      ID         FL         RW         CL         CW         BD
## 16   16 -0.7675967 -0.6755812 -0.7452609 -0.7005248 -0.7680803
## 180 180  0.8345433  0.7233790  0.6875280  0.7095442  0.7502689
## Clustering vector:
##   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36  37  38  39  40
##   1   1   1   1   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2
##  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
##  61  62  63  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79  80
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
##  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96  97  98  99 100
##   1   2   2   1   2   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
##   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1   1
```

```
## 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140
##   1   1   1   1   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160
##   2   2   2   2   2   2   2   2   2   2   1   1   1   1   1   1   1   1   1   1
## 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
##   1   1   2   2   1   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
##   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2   2
## Objective function:
##    build     swap
## 1.360124 1.161307
##
## Available components:
##  [1] "medoids"    "id.med"     "clustering" "objective"  "isolation"
##  [6] "clusinfo"   "silinfo"    "diss"       "call"       "data"
```

```
pam.res$clusinfo
```

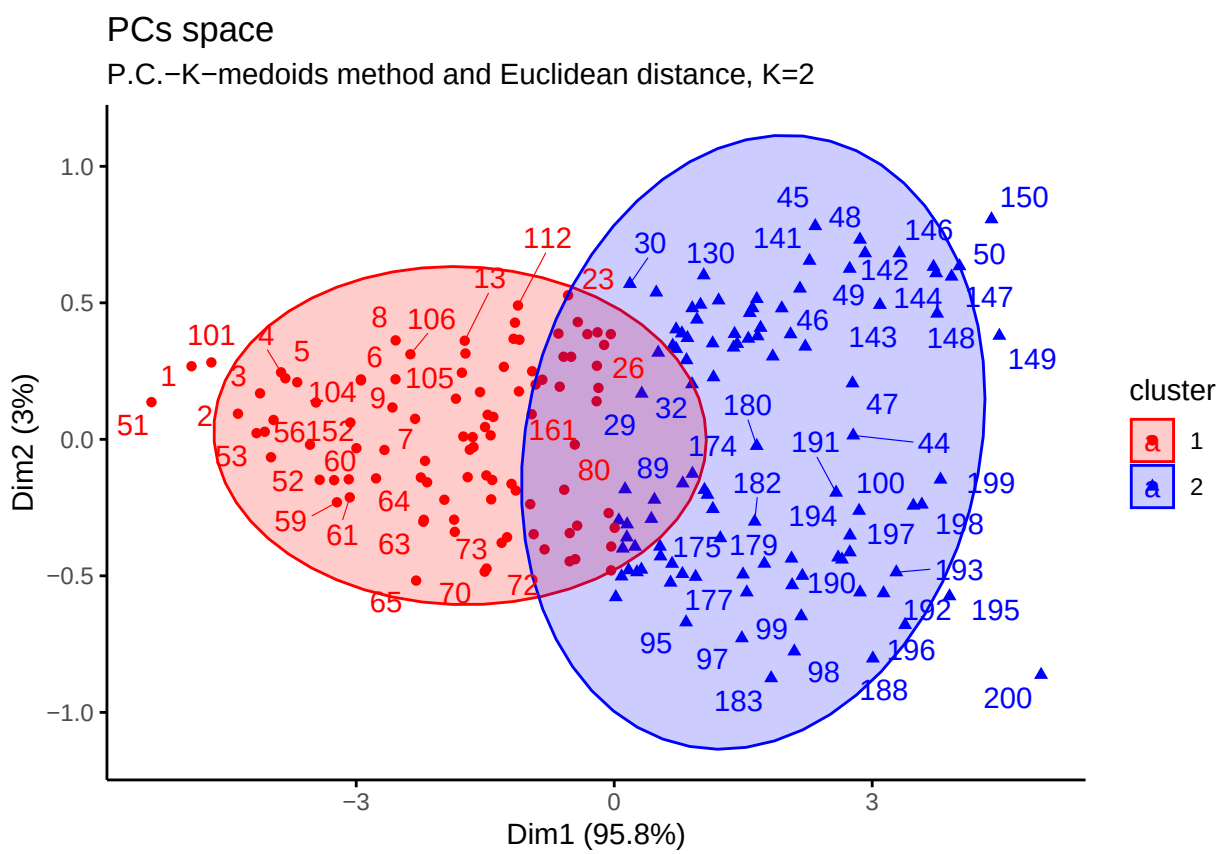
```
##      size max_diss  av_diss diameter separation
## [1,]   98 3.751577 1.123292 5.406655  0.1477102
## [2,]  102 3.412238 1.197831 4.998164  0.1477102
```

```
cc <- pam.res$cluster
pairs(df_scaled_crabs, gap=0, pch=cc, main="Original space\nP.C.-K-medoids method
and Euclidean distance, K=2", cex.main= 0.7,
col=c("red", "blue")[cc])
```



```r
fviz_cluster(pam.res, palette = c("red","blue"), ellipse.type = "t",
repel = TRUE, main= "PCs space", ggtheme = theme_classic())+labs(
subtitle = "P.C.-K-medoids method and Euclidean distance, K=2", cex.sub= 0.5)
```

```
## Warning: ggrepel: 122 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



Internal validation measures:

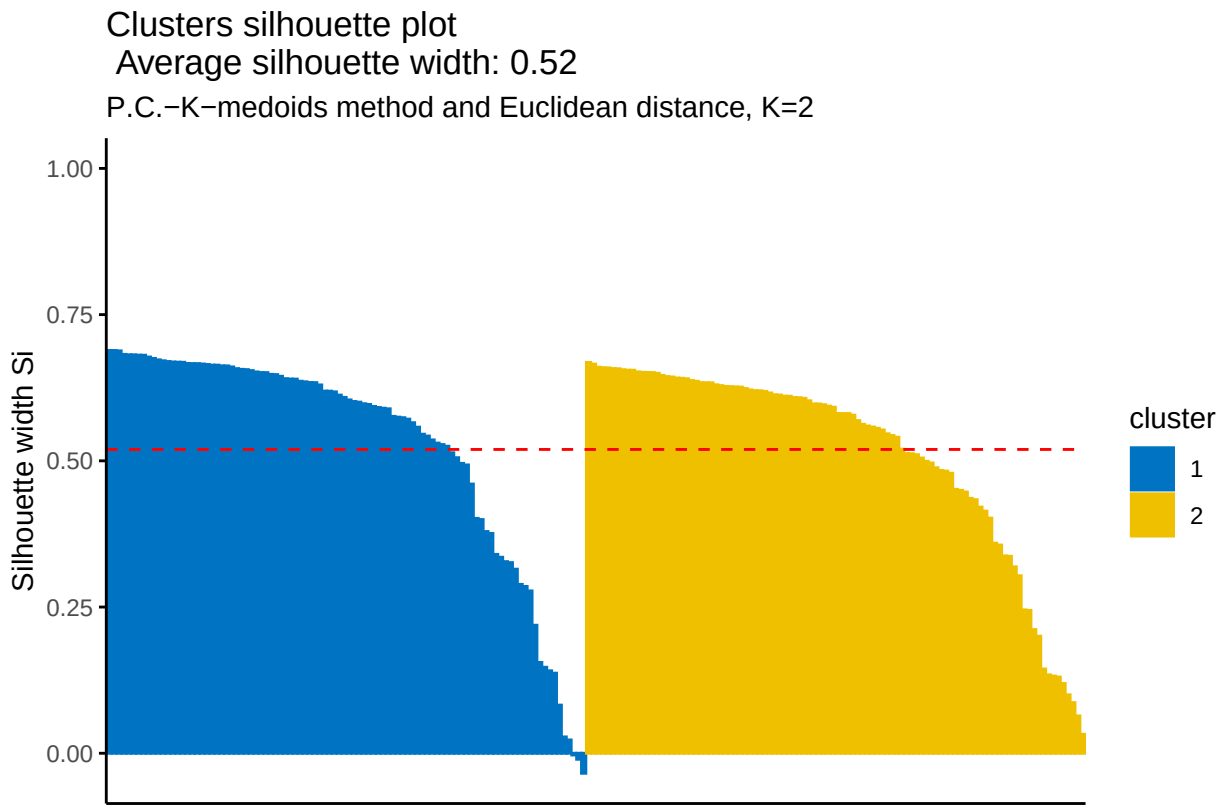
Silhouette width:

```r
hclust<- eclust(crabs_p,k=2 ,"pam", graph = FALSE, hc_metric = "euclidean")
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.5451375
```

```r
fviz_silhouette(pam.res, palette = "jco",
ggtheme = theme_classic())+labs(
subtitle = "P.C.-K-medoids method and Euclidean distance, K=2", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   98          0.53
## 2       2  102          0.51
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.5341951 0.5563010
```

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##    cluster neighbor    sil_width
## 82       2        1 -0.005313539
```

Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.02128465
```

According to the Dunn index, the units are not clustered well enough.

External validation measures:

#Confusion matrix:

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2
##   F 51 49
##   M 50 50
```

Correct Rand Index:

```
Status <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.004949492
```

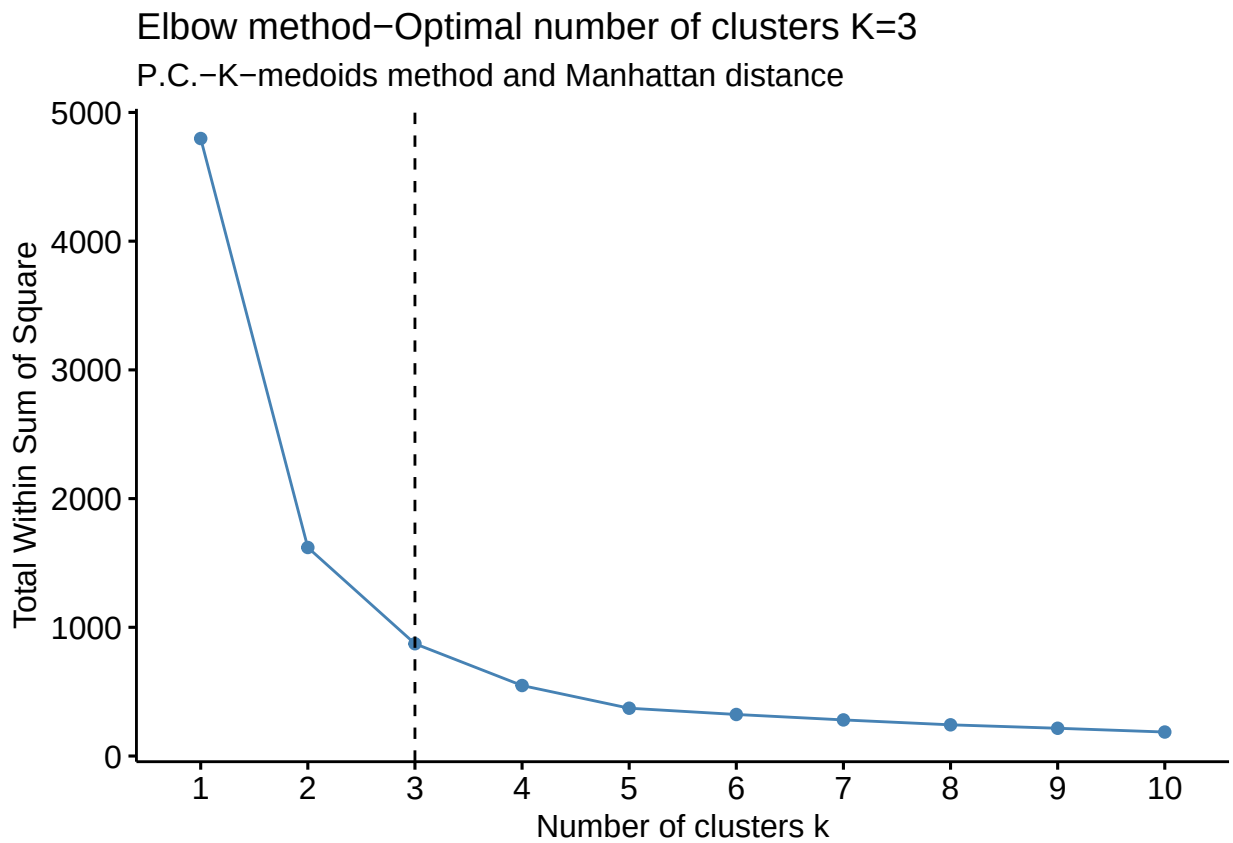
Meila's VI Index:

```
stats$vi
```

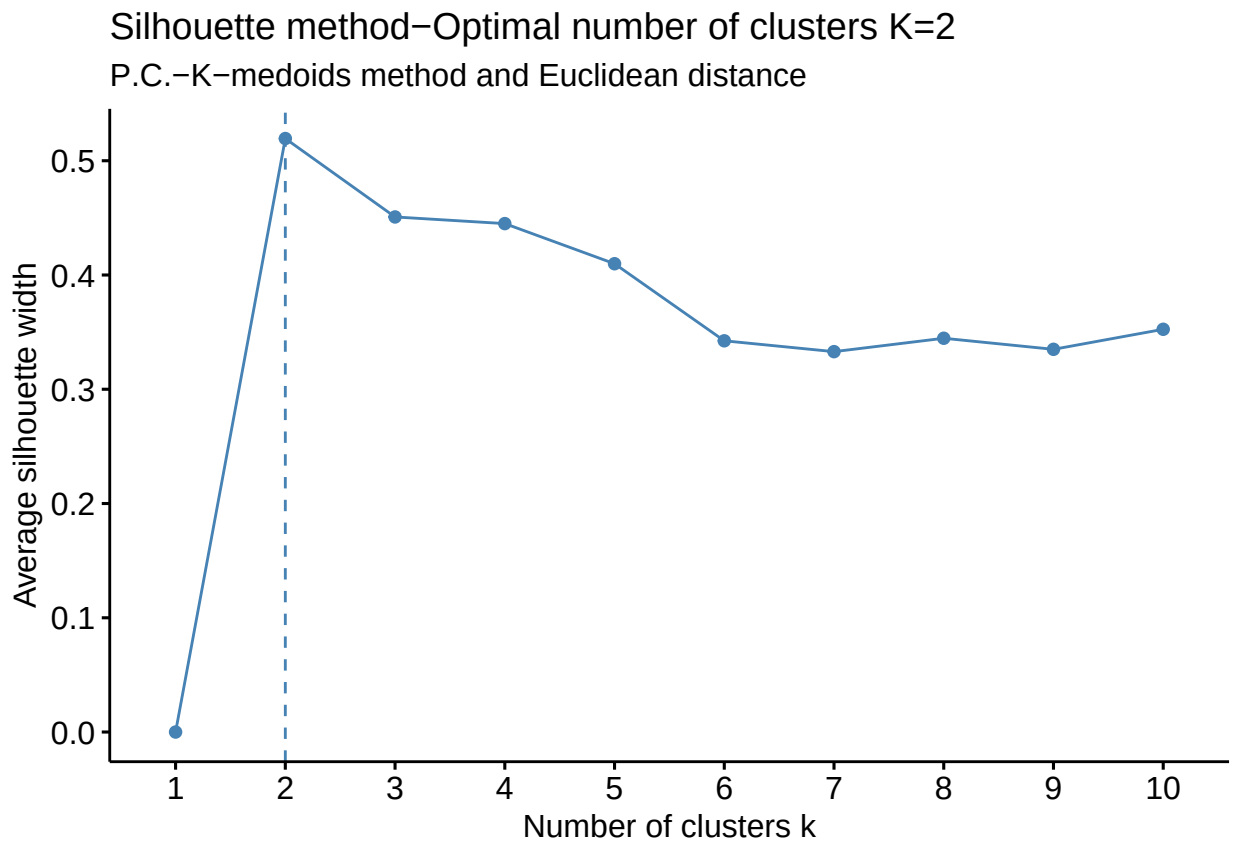
```
## [1] 1.386144
```

Partitioning around medoids (PAM) method and Manhattan distance:

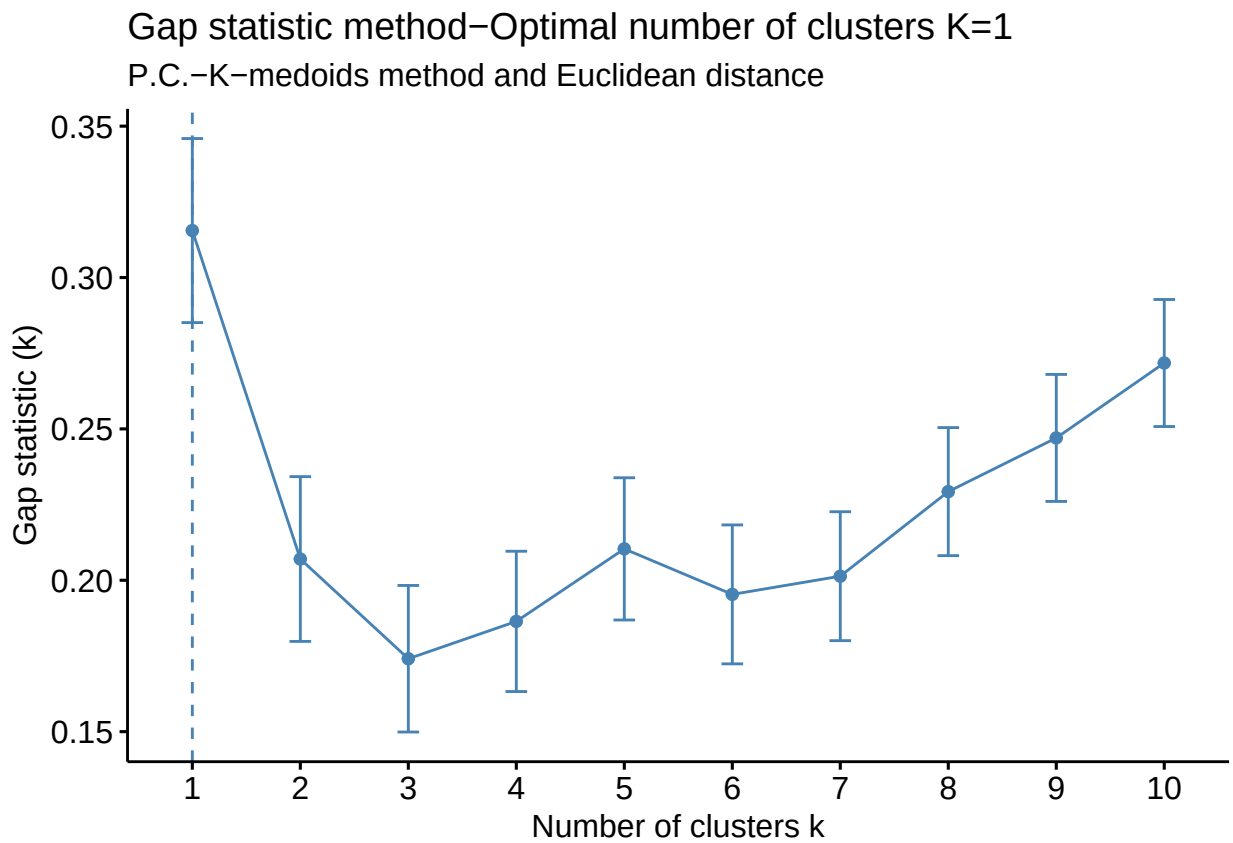
```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "wss",
diss = dist(df_scaled_crabs, method = "manhattan"))+
geom_vline(xintercept = 3, linetype = 2)+ labs(title = "Elbow method-Optimal number of clusters K=3",
subtitle="P.C.-K-medoids method and Manhattan distance", cex.sub= 0.5)
```




```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "silhouette")+
labs(title = "Silhouette method-Optimal number of clusters K=2",
subtitle="P.C.-K-medoids method and Euclidean distance", cex.sub= 0.5)
```



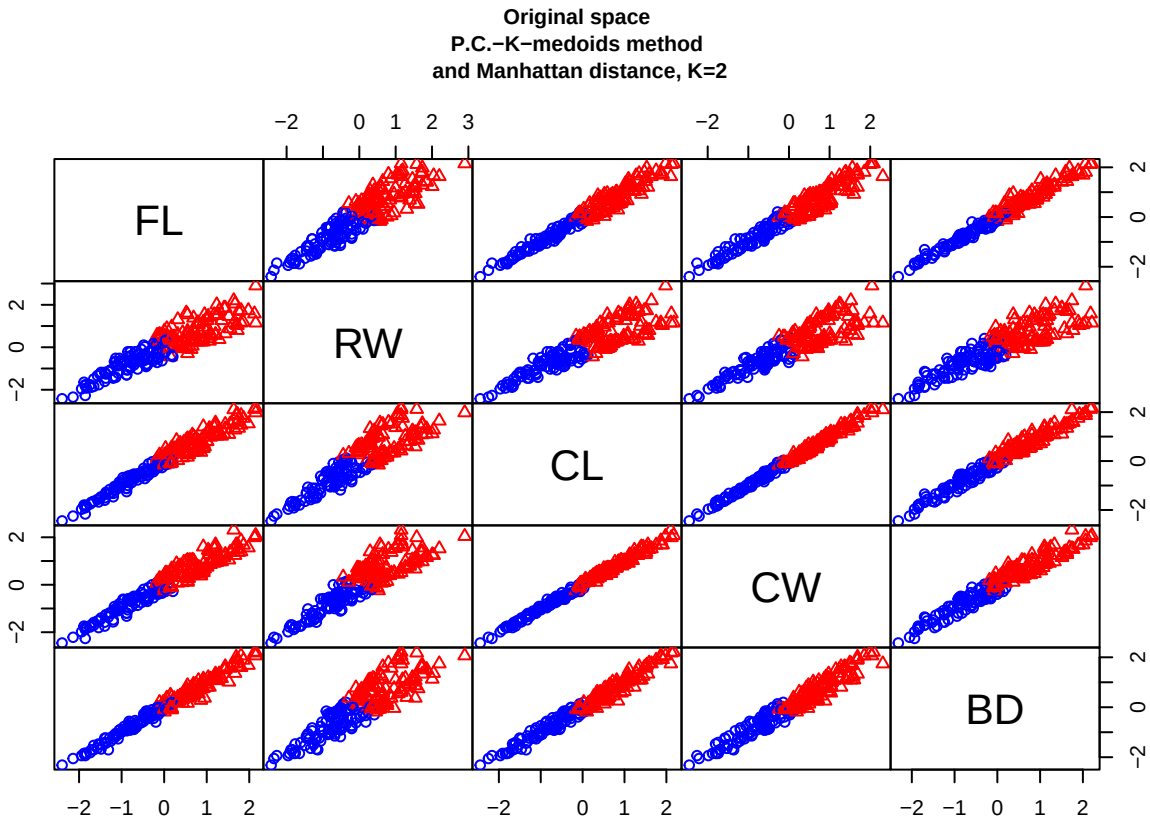

```
fviz_nbclust(df_scaled_crabs, cluster::pam, method = "gap_stat", nboot = 500)+
labs(title = "Gap statistic method-Optimal number of clusters K=1",
subtitle="P.C.-K-medoids method and Euclidean distance", cex.sub= 0.5)
```




```
pam.res$clusinfo
```

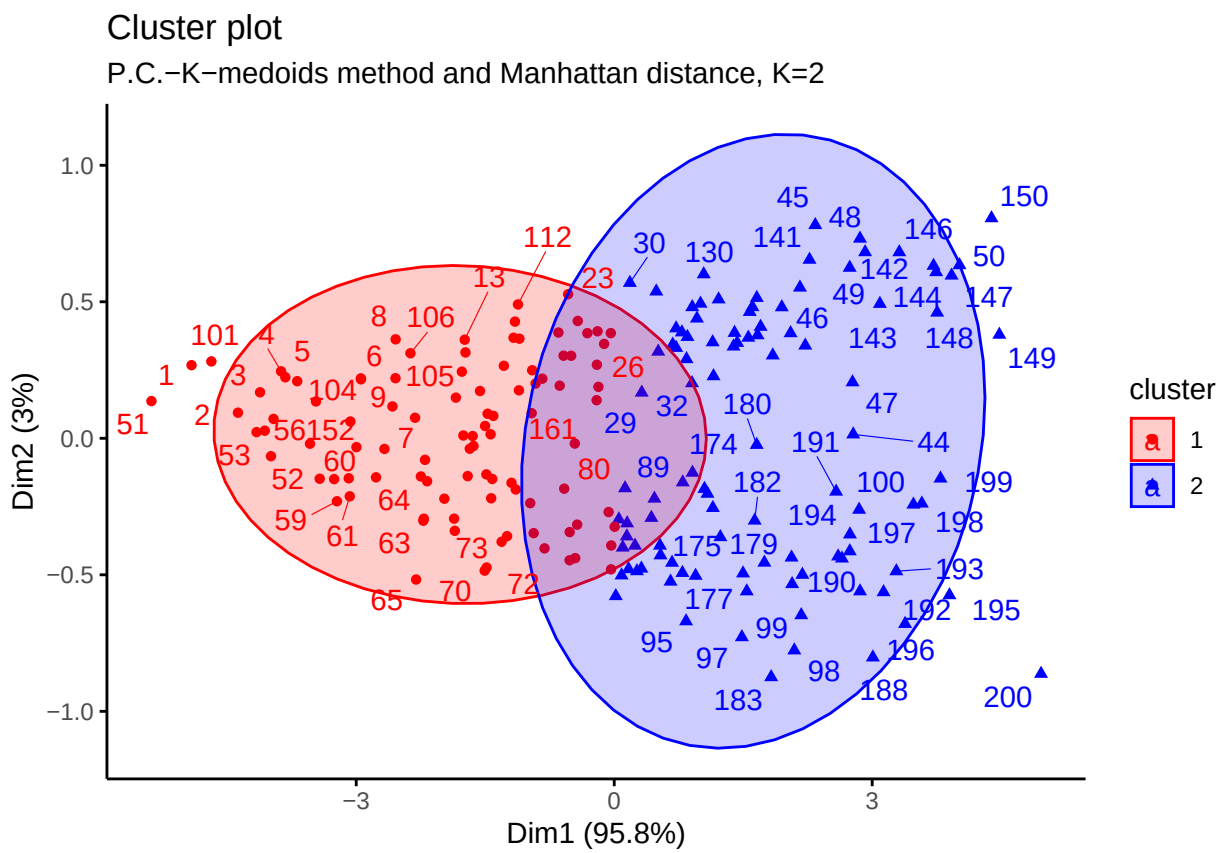
```
##      size max_diss  av_diss diameter separation
## [1,]   98 3.751577 1.123292 5.406655  0.1477102
## [2,]  102 3.412238 1.197831 4.998164  0.1477102
```

```
cm <- pam.res$cluster
pairs(df_scaled_crabs, gap=0, pch=cm, main="Original space\nP.C.-K-medoids method
 and Manhattan distance, K=2", cex.main= 0.7,
col=c("blue","red")[cm])
```




```r
fviz_cluster(pam.res, palette = c("red", "blue"), ellipse.type = "t",
repel = TRUE, ggtheme = theme_classic())+labs(
subtitle = "P.C.-K-medoids method and Manhattan distance, K=2", cex.sub= 0.5)
```

```
## Warning: ggrepel: 122 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



Internal validation measures:

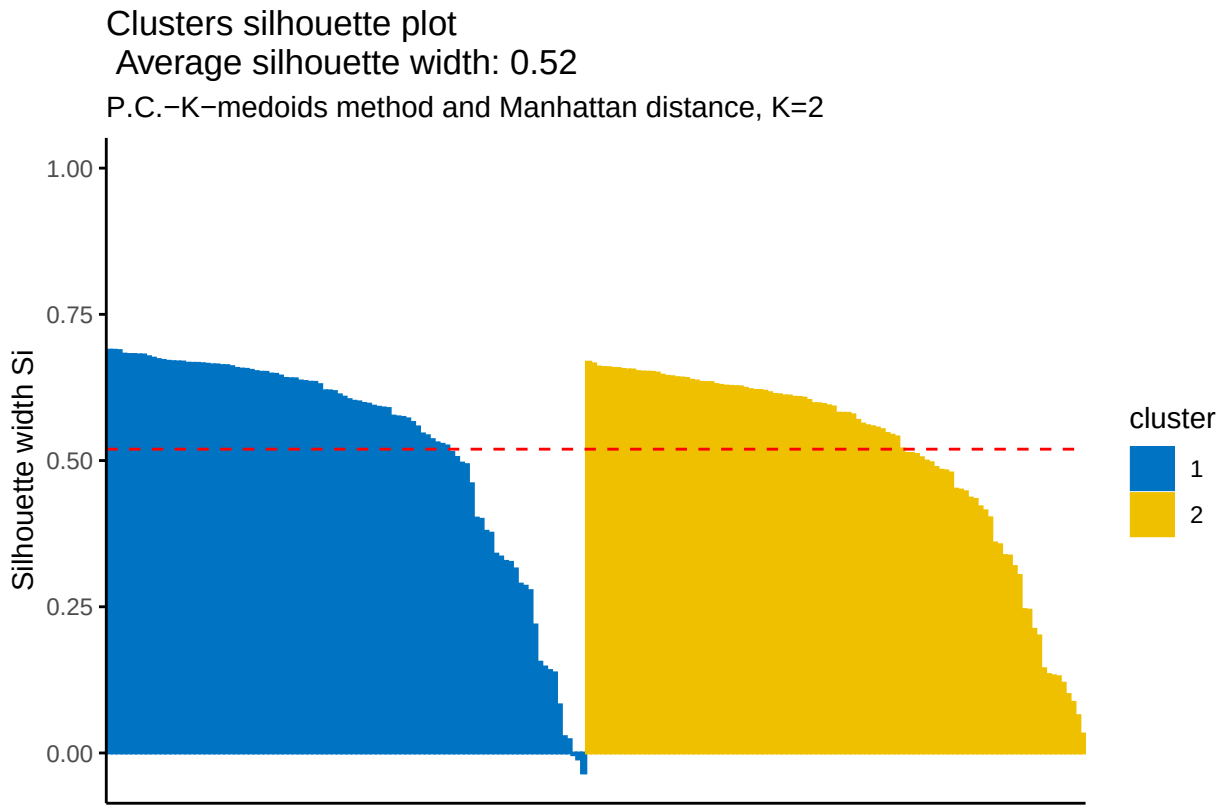
Silhouette width:

```
hclust<- eclust(crabs_p,k=2 ,"pam", graph = FALSE, hc_metric = "manhattan")
silinfo <- hclust$silinfo
silinfo$avg.width
```

```
## [1] 0.5451375
```

```
fviz_silhouette(pam.res, palette = "jco",
ggtheme = theme_classic())+labs(
subtitle = "P.C.-K-medoids method and Manhattan distance, K=2", cex.sub= 0.5)
```

```
##   cluster size ave.sil.width
## 1       1   98          0.53
## 2       2  102          0.51
```




```
silinfo$clus.avg.widths
```

```
## [1] 0.5341951 0.5563010
```

The value of the complete silhouette width suggests that, on average, the units are well-clustered. A higher complete silhouette width signifies that the data points within each cluster are relatively close to each other and well-separated from the points in other clusters. This indicates a good level of clustering quality and coherence among the units.

```
sil <- hclust$silinfo$widths[, 1:3]
neg_sil_index_aver.eu<- which(sil[, "sil_width"] < 0)
sil[neg_sil_index_aver.eu, , drop = FALSE]
```

```
##    cluster neighbor    sil_width
## 82       2        1 -0.005313539
```

2.Dunn index:

```
stats <- cluster.stats(dist(df_scaled_crabs), hclust$cluster)
stats$dunn
```

```
## [1] 0.02128465
```

External validation measures:

#Confusion matrix:

According to the Confusion matrix, the number of clusters is equal to the number of nominal values. In this case, the clusters found are 2, which corresponds to the fact that the nominal variable can take 2 possible values. The clustering results align with the nominal variable categories, indicating a consistent and meaningful clustering solution.

```
table(crabs$sex, hclust$cluster)
```

```
##
##      1  2
##   F 51 49
##   M 50 50
```

2.Corrected Rand Index:

```
sex <- as.numeric(crabs$sex)
stats<- cluster.stats(d = dist(df_scaled_crabs), sex, hclust$cluster)
stats$corrected.rand
```

```
## [1] -0.004949492
```

Soft clustering approach

Model based-clustering:

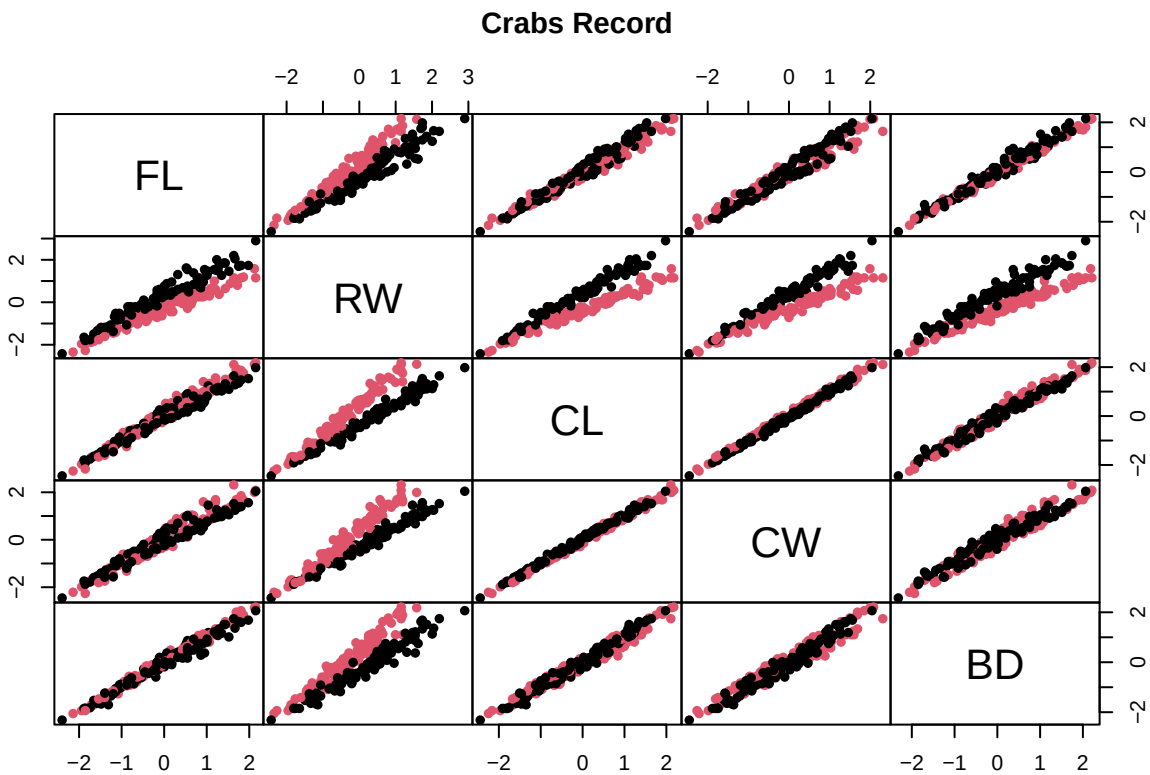
```
summary(crabs$sex)
```

```
##   F   M
## 100 100
```

```
head(crabs)
```

```
##   sp sex index   FL  RW   CL   CW  BD
## 1  B   M     1  8.1 6.7 16.1 19.0 7.0
## 2  B   M     2  8.8 7.7 18.1 20.8 7.4
## 3  B   M     3  9.2 7.8 19.0 22.4 7.7
## 4  B   M     4  9.6 7.9 20.1 23.1 8.2
## 5  B   M     5  9.8 8.0 20.3 23.0 8.2
## 6  B   M     6 10.8 9.0 23.0 26.5 9.8
```

```
X <- data.matrix(crabs_p)
sX <- scale(X)
pairs(sX, gap=0, pch = 16, col = as.numeric(crabs$sex), cex.main = 0.9 ,main="Crabs Record")
```

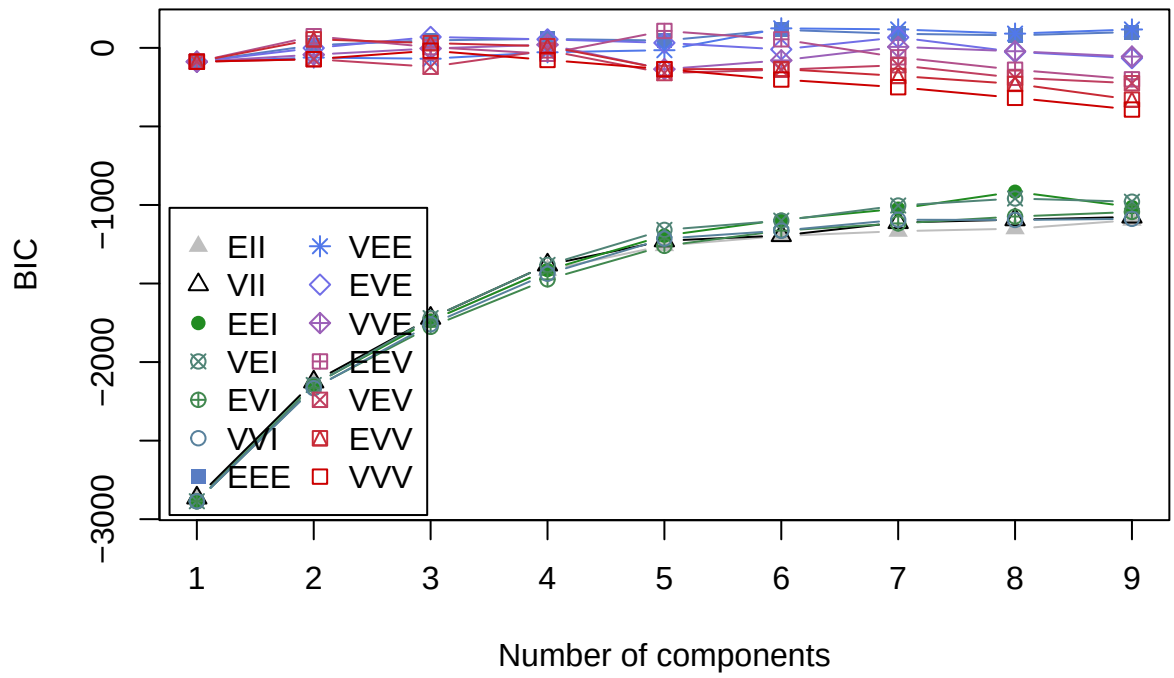


In order to investigate the connection between the categorical variable and the underlying clustering, the variable is removed and the data is standardized. The data is then presented visually through scatterplots that show pairs of variables, with different colors representing the two possible values. It appears challenging to discern distinct groups. To address this, various simplified Gaussian mixture models are applied to the standardized data using the "Mclust()" function in R.

```
mod <- Mclust(df_scaled_crabs)
summary(mod$BIC)
```

```
## Best BIC values:
##             VEE,6      VEE,7      VEE,9
## BIC      124.2538 117.541388 117.204593
## BIC diff   0.0000  -6.712384  -7.049179
```

```
plot(mod, what = "BIC", ylim = range(mod$BIC, na.rm = TRUE),
legendArgs = list(x = "bottomleft"))
```

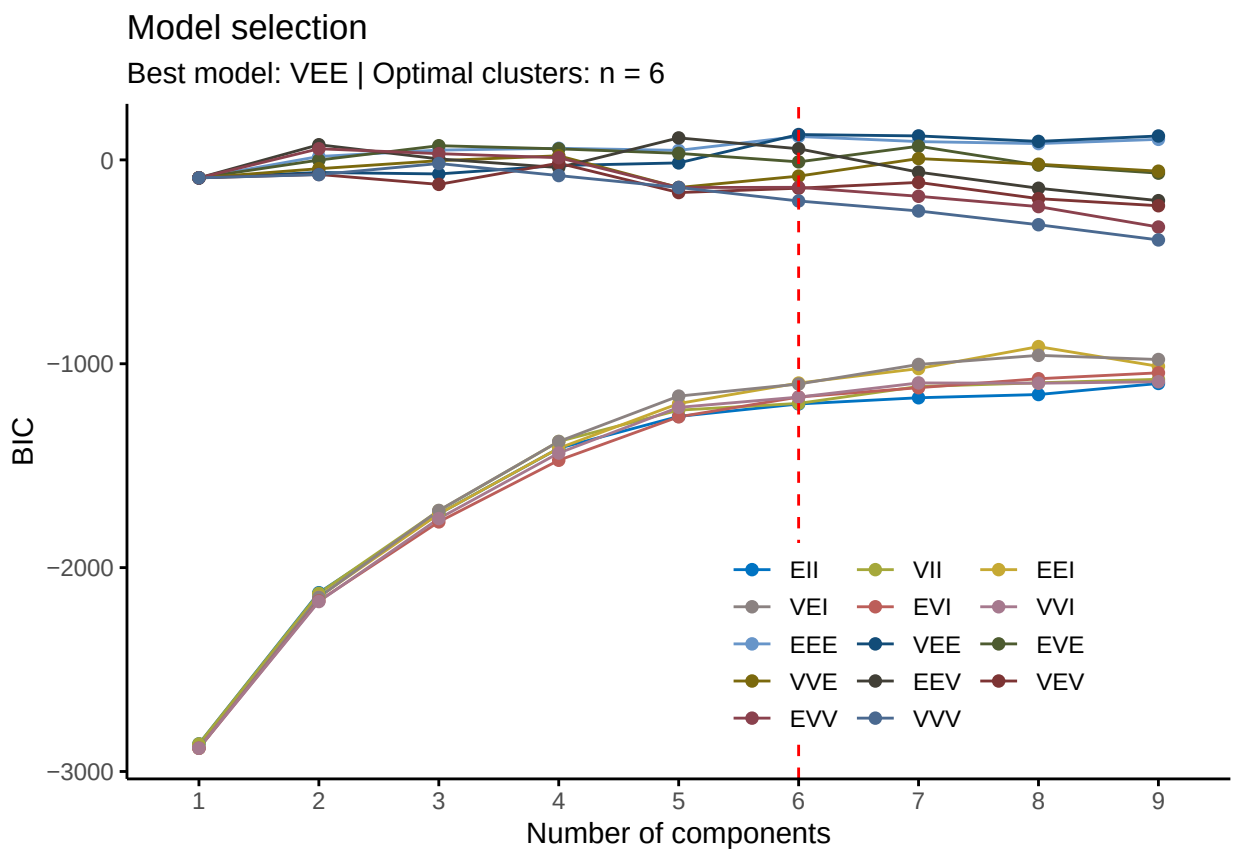



```r
summary(mod)
```

```
## ----------------------------------------------------
## Gaussian finite mixture model fitted by EM algorithm
## ----------------------------------------------------
##
## Mclust VEE (ellipsoidal, equal shape and orientation) model with 6 components:
##
##  log-likelihood   n df      BIC      ICL
##        207.8306 200 55 124.2538 107.6974
##
## Clustering table:
##  1  2  3  4  5  6
## 21 27 36 45 28 43
```

```r
fviz_mclust(mod, "BIC", palette = "jco")
```

```
## Warning: `gather_()` was deprecated in tidyr 1.2.0.
## i Please use `gather()` instead.
## i The deprecated feature was likely used in the factoextra package.
##   Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
head(round(mod$z, 6), 15)
```

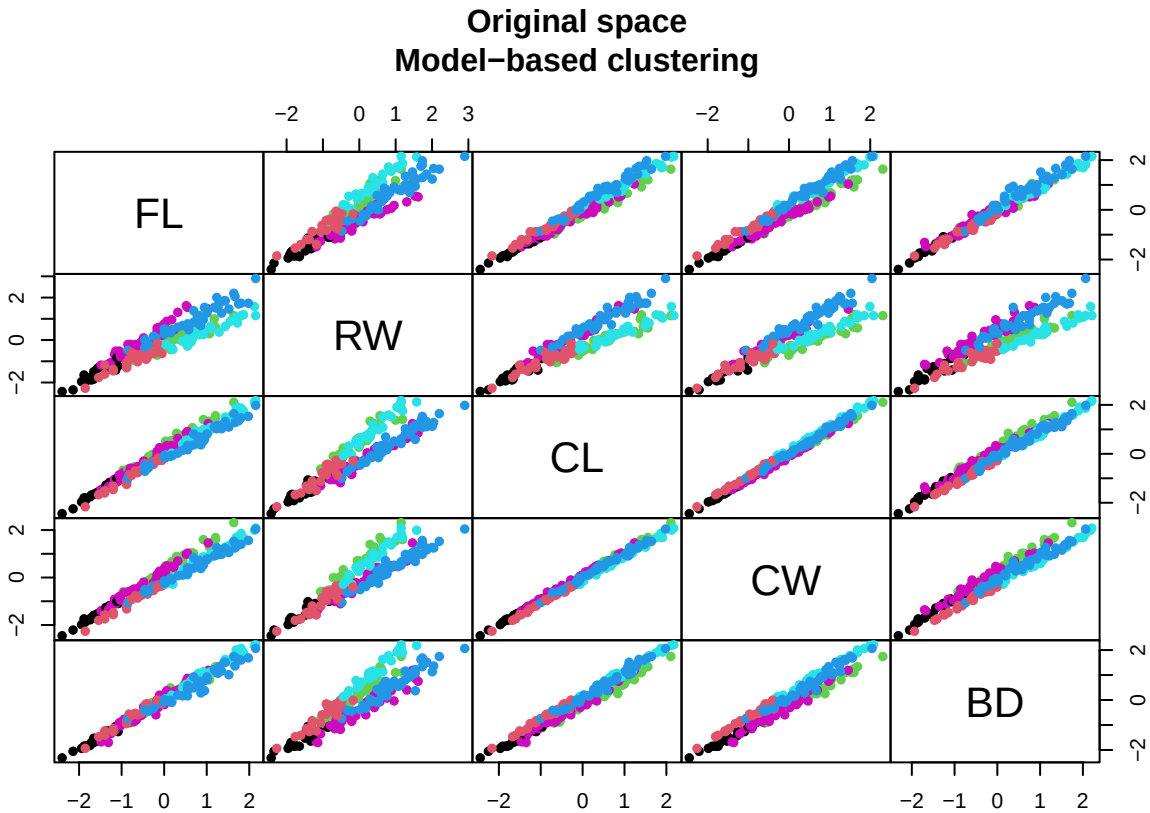
```
##         [,1]   [,2]     [,3] [,4] [,5]     [,6]
## 1   0.999140 4e-06 0.000066    0    0 0.000790
## 2   0.999198 0e+00 0.000015    0    0 0.000787
## 3   0.997724 0e+00 0.000605    0    0 0.001671
## 4   0.998641 0e+00 0.000942    0    0 0.000417
## 5   0.998513 3e-06 0.000756    0    0 0.000728
## 6   0.995316 0e+00 0.002696    0    0 0.001988
## 7   0.949197 0e+00 0.000792    0    0 0.050011
## 8   0.815528 0e+00 0.181389    0    0 0.003082
## 9   0.943131 0e+00 0.016039    0    0 0.040830
## 10  0.589187 0e+00 0.008547    0    0 0.402267
## 11  0.002760 0e+00 0.884404    0    0 0.112835
## 12  0.122394 0e+00 0.053473    0    0 0.824133
## 13  0.002166 0e+00 0.997454    0    0 0.000380
## 14  0.001371 0e+00 0.993610    0    0 0.005019
## 15  0.234681 0e+00 0.299888    0    0 0.465431
```

```r
head(mod$classification, 15)
```

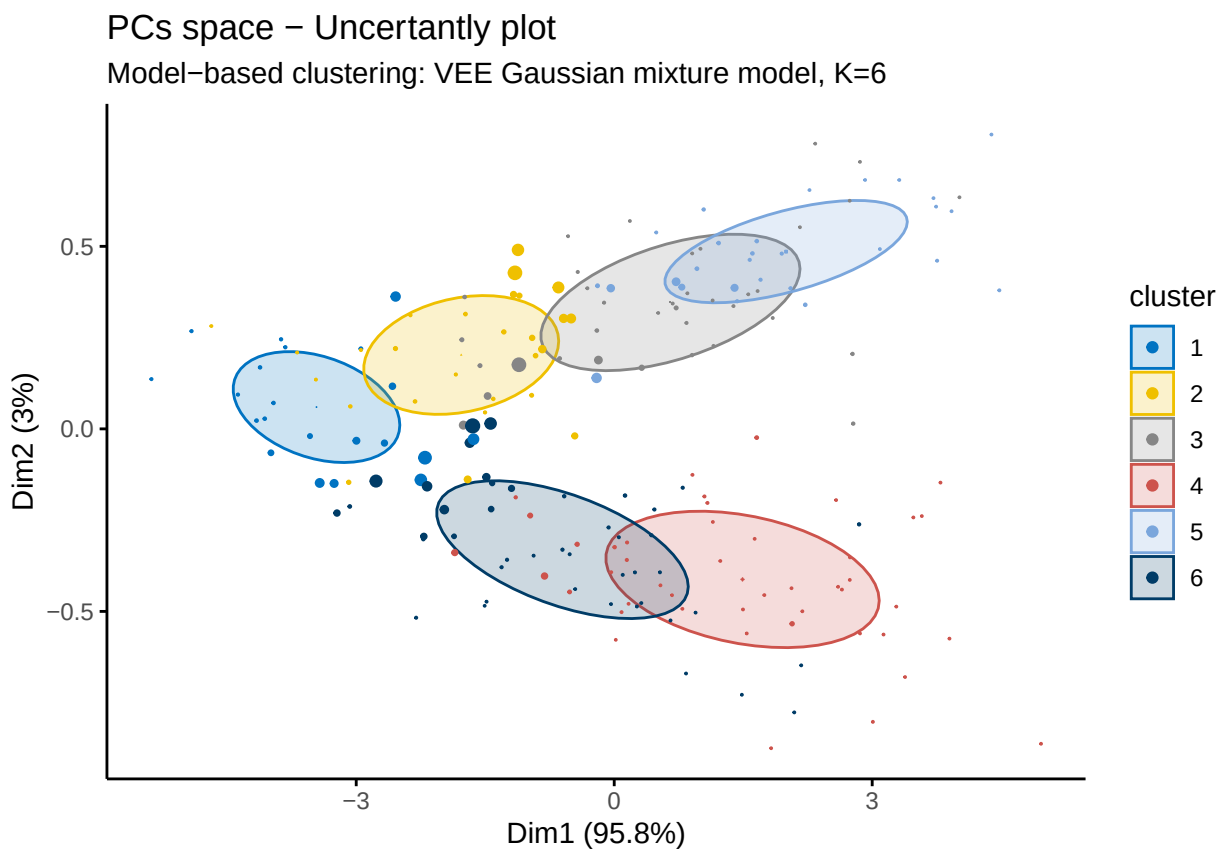
```
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
##  1  1  1  1  1  1  1  1  1  1  3  6  3  3  6
```

Based on the Bayesian Information Criterion (BIC), a penalized selection criterion, the top three Gaussian mixture models are identified as follows: VVI with 7 clusters, VVI with 8 clusters, and VVI with 9 clusters.

```r
pairs(sX, gap=0, pch = 16, col = mod$classification, cex.main = 1,main="Original space\nModel-based clus
```




```r
fviz_mclust(mod, "uncertainty", palette = "jco",
main = "PCs space - Uncertantly plot")+
labs(subtitle= "Model-based clustering: VEE Gaussian mixture model, K=6")
```



External validation measures:

Confusion matrix:

```
table(crabs$sex, mod$classification)
```

```
##
##      1  2  3  4  5  6
##   F 10  5  0 45  0 40
##   M 11 22 36  0 28  3
```

Correct Rand Index:

```
adjustedRandIndex(crabs$sex, mod$classification)
```

```
## [1] 0.2822713
```

The Correct Rand Index indicates a higher level of agreement between the nominal variable and the cluster solution.